Ciencia de Datos en la terminal Linux

Luis Gómez

Índice

Información del curso **8**
Requisitos computacionales . . . 8
Evaluaciones . . . 8
Fechas evaluaciones . . . 8
Contenidos . . . 8

1. Instalación de software **9**
1.1. Windows 10+ . . . 9
1.1.1. Instalación Ubuntu WSL . . . 9
1.1.2. Instalación de Docker . . . 9
1.1.3. Abrir distribución . . . 9
1.2. Linux y distribución WSL (basado en Debian) . . . 10
1.2.1. Instalación contenedor docker . . . 10
1.2.2. Crear alias para llamado simple (opcional) . . . 11

2. Introducción **12**
2.1. Ciencia de Datos . . . 12
2.1.1. Obtención de datos . . . 12
2.1.2. Depuración de datos . . . 12
2.1.3. Exploración de datos . . . 13
2.1.4. Modelación de datos . . . 13
2.1.5. Interpretación de datos . . . 14
2.2. Historia de la terminal . . . 14
2.2.1. Shell . . . 15
2.2.2. Ventajas del uso de la terminal . . . 15
2.3. La terminal . . . 15
2.3.1. El ambiente . . . 15
2.3.2. Ejecutar un comando . . . 16
2.3.3. Tipos de herramientas . . . 18
2.3.4. Combinando herramientas . . . 19
2.3.5. Redireccionando entrada y salida . . . 21

2.3.6. Trabajando con Archivos y Directorios . . . 23
2.3.7. Gestión de Salida . . . 25
2.3.8. Ayuda! . . . 26
2.4. Control de Versiones . . . 26
2.4.1. Git: La herramienta esencial . . . 27
2.4.2. GitHub/GitLab . . . 28
2.4.3. Mejores prácticas para proyectos de ciencia de datos . . . 29
2.4.4. Clonar . . . 29
2.4.5. Commit y Push . . . 30
2.4.6. Hacer aportes a un repositorio . . . 30
2.4.7. Sugerencias de mensajes commit . . . 31
2.4.8. Recursos adicionales . . . 32
2.4.9. Para mayor exploración . . . 32

3. Obtención de datos **33**
3.1. Copiando Archivos Locales en el Contenedor Docker . . . 33
3.2. Descargando desde Internet . . . 34
3.2.1. Presentando `curl` . . . 34
3.2.2. Guardado . . . 35
3.2.3. Otros protocolos . . . 35
3.2.4. Siguiendo redirecciones . . . 35
3.3. Descomprimiendo Archivos . . . 36
3.4. Conversión de hojas de cálculo de Microsoft Excel a CSV . . . 37
3.5. Consultando Bases de Datos Relacionales . . . 39
3.6. Llamando a las API web . . . 39
3.6.1. Autenticación . . . 40
3.6.2. API de transmisión (streaming) . . . 40
3.7. Para Mayor Exploración . . . 41

4. Creando Herramientas de Línea de Comandos **42**
4.1. Conversión de one-liner into shell scripts . . . 42
4.1.1. Paso 1: Crear Archivo . . . 44
4.2. Paso 2: Dar permisos de Ejecución . . . 45
4.2.1. Paso 3: Definir shebang . . . 46
4.2.2. Paso 4: Remover la entrada fija . . . 46
4.2.3. Paso 5: Añadir argumentos . . . 47
4.2.4. Paso 6: Extiender el PATH . . . 48
4.3. Creando Herramientas de Línea de Comandos con Python y R . . . 48
4.3.1. Portando El Script De Shell . . . 49
4.3.2. Procesando Datos de Transmisión desde la Entrada Estándar (stdin) . . . 50
4.4. Para Profundizar . . . 51

5. Depuración de datos **52**

5.1. Transformaciones . . . 52
5.2. Texto Plano . . . 54
5.2.1. Filtrando Líneas . . . 54
5.2.2. Extrayendo Valores . . . 56
5.2.3. Reemplazando y Eliminando Valores . . . 57
5.3. CSV . . . 58
5.3.1. Cuerpos, Encabezados y Columnas . . . 58
5.3.2. Realizando Consultas SQL en CSV . . . 60
5.3.3. Extrayendo y Reordenando Columnas . . . 61
5.3.4. Filtrando Filas . . . 61
5.3.5. Combinar Columnas . . . 62
5.3.6. Combinando Múltiples Archivos CSV . . . 63
5.4. Trabajando con XML/HTML y JSON . . . 64
5.5. Para Profundizar en el Tema . . . 66

6. Exploración de Datos **67**
6.1. Inspeccionando Datos y sus Propiedades . . . 67
6.1.1. Encabezado . . . 68
6.1.2. Inspeccionando los datos . . . 68
6.1.3. Nombres de atributos y Tipos de Datos . . . 69
6.1.4. Identificadores Únicos, Variables Continuas y Factores . . . 69
6.2. Calculando Estadísticas Descriptivas . . . 70
6.2.1. Estadísticas de Columna . . . 70
6.2.2. R One-Liners en la terminal . . . 71
6.3. Creando Visualizaciones . . . 73
6.3.1. Mostrando Imágenes desde la Línea de Comandos . . . 73
6.3.2. Graficas rápidas . . . 75
6.3.3. Creando Gráficos de Barras . . . 76
6.3.4. Creando Histogramas . . . 76
6.3.5. Creando Gráficos de Densidad . . . 76
6.3.6. Gráficas erróneas . . . 77
6.3.7. Creando Diagramas de Dispersión . . . 77
6.3.8. Creando Líneas de Tendencia . . . 77
6.3.9. Creando Box-Plot . . . 78
6.3.10. Añadiendo Etiquetas . . . 78
6.3.11. Yendo Más Allá de los Gráficos Básicos . . . 78
6.4. Visualización de imágenes en la terminal . . . 79
6.5. Copia de archivos hacia el servidor . . . 79
6.6. Para Profundizar . . . 80

7. Paralelización **81**
7.1. Procesamiento en serie . . . 81
7.1.1. Iterando sobre números . . . 82

7.1.2. Iterando Sobre Líneas . . . 82
7.1.3. Iterando sobre Archivos . . . 83
7.2. Procesamiento Paralelo . . . 83
7.2.1. Introduciendo GNU Parallel . . . 84
7.2.2. Especificando la Entrada . . . 85
7.2.3. Controlando el Número de Trabajos Concurrentes . . . 87
7.2.4. Log y Salida . . . 87
7.2.5. Creando Herramientas Paralelas . . . 89
7.3. Procesamiento Distribuido . . . 89
7.4. Para Profundizar en el Tema . . . 89

8. Modelación de Datos **90**
8.1. Vino . . . 90
8.2. Reducción de Dimensionalidad con Tapkee . . . 92
8.2.1. Introducción a Tapkee . . . 93
8.2.2. Mapeos Lineales y No Lineales . . . 93
8.3. Regresión con Vowpal Wabbit . . . 95
8.3.1. Preparando los Datos . . . 95
8.3.2. Entrenando el Modelo . . . 96
8.3.3. Probando el Modelo . . . 96
8.4. Clasificación con SciKit-Learn Laboratory . . . 97
8.4.1. Preparando los Datos . . . 98
8.4.2. Ejecutando el Experimento . . . 99
8.4.3. Analizando los Resultados . . . 99
8.5. Para Mayor Exploración . . . 100

9. Nushell **101**
9.1. Instalación de nushell . . . 101
9.2. Elementos básicos de nushell . . . 102
9.2.1. Obtener Ayuda . . . 104
9.3. Trabajando con tablas . . . 104
9.3.1. Ordenando los Datos . . . 104
9.3.2. Seleccionando Datos . . . 105
9.3.3. Obteniendo Datos de una Tabla . . . 105
9.3.4. Cambiando Datos en una Tabla . . . 105
9.3.5. Añadiendo una columna . . . 106
9.3.6. Actualizando una Columna . . . 107
9.3.7. Moviendo Columnas . . . 107
9.3.8. Renombrando Columnas . . . 108
9.3.9. Rechazando/Eliminando Columnas . . . 108
9.4. Moviéndose por el Sistema . . . 108
9.4.1. Viendo el Contenido de un Directorio . . . 108
9.4.2. Patrones Glob (comodines) . . . 108

9.5. Como entender nu . . . 110
9.5.1. Nushell no es bash . . . 110
9.5.2. Nushell como un lenguaje compilado . . . 111
9.5.3. Las variables son inmutables por defecto . . . 112
9.6. Formateando salida de comandos (parsing) . . . 112
9.6.1. `detect columns` (bastante automático) . . . 112
9.6.2. Usando `from ssv` . . . 112
9.6.3. Usando `parse` . . . 113
9.7. Descargando datos . . . 113
9.7.1. Descargando un archivo json . . . 113
9.7.2. Descargando desde multiples urls . . . 113
9.8. Trabajando con listas . . . 115
9.8.1. Creando listas . . . 115
9.8.2. Actualizando listas . . . 115
9.8.3. Eliminando o Agregando Elementos de la Lista . . . 116
9.8.4. Iterando sobre Listas . . . 117
9.8.5. Accediendo a la Lista . . . 117
9.8.6. Convirtiendo la Lista . . . 119
9.9. Trabajando con Registros (records) . . . 120
9.9.1. Creando registros . . . 120
9.9.2. Actualizando Registros . . . 120
9.9.3. Iterando sobre un Registro . . . 121
9.10. Navegando y Accediendo a Datos Estructurados . . . 121
9.10.1. Antecedentes . . . 121
9.10.2. Rutas de Celda . . . 122
9.10.3. Datos Anidados . . . 124
9.10.4. Usando `get` y `select` . . . 124
9.10.5. Nombres de Clave/Columna con espacios . . . 125
9.10.6. Manejo de Datos Faltantes . . . 125
9.10.7. Otros comandos para acceder a datos estructurados . . . 126
9.11. Trabajando con strings . . . 126
9.11.1. Formatos de String . . . 126
9.11.2. Strings con Comillas Simples . . . 127
9.11.3. Strings con Comillas Dobles . . . 127
9.11.4. Strings Raw . . . 128
9.11.5. Strings de Palabra Simple . . . 128
9.11.6. Strings con Comillas Invertidas . . . 129
9.11.7. Strings como comandos externos . . . 129
9.11.8. Añadir y anteponer a strings . . . 130
9.11.9. Interpolación de strings . . . 130
9.11.10. Dividir Strings . . . 131
9.11.11. El comando `str` . . . 131
9.11.12. Comparación de Strings . . . 133

9.11.13. Convertir Strings . . . 133
9.11.14. Colorear Strings . . . 133
9.12. El plugin polars de Nushell . . . 133
9.12.1. Casos de uso . . . 134
9.12.2. Conceptos básicos . . . 134
9.12.3. Uso básico . . . 135
9.12.4. Manipulación de datos . . . 137
9.12.5. Guardado y formatos . . . 140
9.12.6. Pivoteo de columnas . . . 140
9.12.7. Buenas prácticas y recomendaciones . . . 141
9.12.8. Conclusión . . . 142
9.13. Programación en Nushell . . . 142
9.13.1. Operadores . . . 142
9.13.2. Variables . . . 147
9.13.3. Comandos personalizados . . . 152
9.13.4. Nombres de Comandos . . . 152
9.13.5. Subcomandos . . . 153
9.13.6. Tipos de Parámetros . . . 153
9.13.7. Parámetros con un Valor Predeterminado . . . 154
9.13.8. Parámetros Posicionales Opcionales . . . 155
9.13.9. Opciones (*flags*) . . . 156
9.13.10. Parámetros rest . . . 157
9.13.11. Documentando un Comando . . . 158
9.13.12. Salida de la *pipeline* . . . 159
9.13.13. Entrada de la *pipeline* . . . 160
9.13.14. Control de flujo . . . 160
9.13.15. Ejemplo de desarrollo de comando . . . 167

10. Para seguir explorando **171**
10.1. La Línea de Comandos . . . 171
10.2. Programación de Shell . . . 171
10.3. Python, R y SQL . . . 171
10.4. APIs . . . 171
10.5. Aprendizaje Automático . . . 171

Índice de figuras

1. Proceso de la ciencia de datos. . . . 13
2. Primeros computadores. . . . 14
3. Primeras terminales. . . . 15
4. Primeras terminales de video. . . . 16
5. Primeras terminales conectadas a un computador. . . . 17

6. Streams estandar de herramientas de linea de comando 20
7. La salida de una herramienta se puede redirigir (pipe) hacia otra herramienta 20
8. La salida de una herramienta se puede redirigir a un archivo 21
9. Dos formas de usar el contenido de un archivo como entrada 22
10. Redirigiendo `stderr` a `/dev/null` 23
11. Redirigir y leer en una misma instrucción 23
12. Con 'tee', se puede escribir una salida intermedia en un archivo 25
13. Diagrama de procesamiento en paralelo. 85
14. Diagrama de salida en paralelo. 86
15. Diagrama de resultados en paralelo. 88

Índice de tablas

Información del curso

Requisitos computacionales

Conexión `ssh` o imagen docker instalada.

Evaluaciones

- **Trabajo Individual:** 30 % (1 semana)
- .**Trabajo Grupal:** 25 % (1 mes)
- **Presentación:** 45 %

Fechas evaluaciones

- **Trabajo Individual:** 22 octubre
- **Trabajo Grupal:** 26 noviembre
- **Presentación:** 1 diciembre
- **Eval. Intg.:** 3 diciembre

  Todas las entregas mediante repositorio de GitHub o GitLab.

Contenidos

1. Introducción a la terminal Linux.
2. Obtención, depuración, exploración y modelación de datos en la terminal.
3. Nushell para ciencia de datos.

1. Instalación de software

En esta sección se entregaran las instrucciones para instalar el software necesario para este curso, tanto en Windows 10+ como en Linux, para distribución basada en Debian como Ubuntu.

1.1. Windows 10+

1.1.1. Instalación Ubuntu WSL

1. Habilitar WSL1, ejecutando en Powershell (con permisos de administrador):

```
Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Windows-Subsystem-Linux
```

2. Habilitar VMP, ejecutando en Powershell:

```
Enable-WindowsOptionalFeature -Online -FeatureName VirtualMachinePlatform
```

3. Listar las distribuciones disponibles y seleccionar (recomendada Ubuntu 24.04):

```
wsl --list --online
```

4. Instalamos la distribución seleccionada:

```
wsl --install -d Ubuntu-24.04
```

4. Configuramos para que la distribución Linux en WSL use v2. Ejecutamos en Powershell:

```
wsl --set-version Ubuntu-24.04 2
```

Para hacerlo por defecto en todas las distribuciones (recomendado):

```
wsl --set-default-version 2
```

5. Actualizar kernel. Descarga desde WSL e instalar.

1.1.2. Instalación de Docker

1. Descargar instalador Docker para Windows desde: docker y ejecutar. Seleccionar Usar WSL 2.
2. Habilitar Iniciar Docker al iniciar sesión, en la configuración de Docker.
3. Reiniciar.

1.1.3. Abrir distribución

Para entrar a la distribución instalada, ejecutar en `cmd` o `powershell`:

```
wsl
```

1.2. Linux y distribución WSL (basado en Debian)

Una vez instalada una distribución WSL en Windows, o directamente una distribución Linux basada en Debian, reproducir las siguientes instrucciones en la terminal:

```
# actualizar sistema e instalacion paquetes basico
sudo apt -y update
sudo apt -y upgrade
sudo apt -y install aria2 p7zip-full curl git

# instalacion de docker y añadir repositorio:
#"deb [arch=amd64] https://download.docker.com/linux/ubuntu focal stable"
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -
sudo add-apt-repository \
    "deb [arch=amd64] https://download.docker.com/linux/ubuntu focal stable"
sudo apt install docker-ce docker-ce-cli containerd.io caca-utils -y
sudo usermod -aG docker $USER
newgrp docker
```

1.2.1. Instalación contenedor docker

Dentro de la distribución Linux elegida (vía WSL en Windows o distribución instalada directamente), ejecutar en la terminal lo siguiente:

```
# Descarga contenedor Docker
docker pull datasciencetoolbox/dsatcl2e

# Ejecución del contenedor y verificación
docker run --rm -it datasciencetoolbox/dsatcl2e

cowsay "Let's go\!"
exit

# Configuración directorio de datos
cd
mkdir DSITCL #o en ruta a elección
cd DSITCL
aria2c https://www.datascienceatthecommandline.com/2e/data.zip
7z x data.zip
rm data.zip

wget https://bit.ly/3WLMqm6 -O cache.7z
7z x cache.7z
rm cache.7z
```

```
# Para cargar contenedor docker con datos descargados
# reemplazar "$HOME/DSITCL" por la ruta de los datos si se utilizo otra
docker run --rm -it -p 8000:8000 -v "$HOME/DSITCL":/data datasciencetoolbox/dsatcl2e

cd /data
ls
#si se observan las carpetas ch02, etc. Se ejecuto correctamente.

exit
```

> Si no se crea un `alias`, el contenedor Docker siempre debe cargarse con el comando `docker run --rm -it -p 8000:8000 -v "$HOME/DSITCL":/data datasciencetoolbox/dsatcl2e`

1.2.2. Crear alias para llamado simple (opcional)

En la terminal de la distribución Linux elegida (vía WSL en Windows o distribución instalada directamente), ejecutar lo siguiente:

```
cd
nano .bash_aliases
```

Copiar lo siguiente:

```
alias dsitcl='docker run --rm -it -p 8000:8000 -v "$HOME/DSITCL":/data datasciencetoolbox/
```

Teclar `ctrl+x>y>Enter` para guardar. Luego ejecutar:

```
source .bash_aliases
```

Ahora podemos cargar el contenedor Docker mediante:

```
dsitcl
```

> Notar que se puede utilizar `Ctrl+d` para salir del contenedor.

2. Introducción

El objetivo de este curso es aumentar la productividad y eficiencia al trabajar en ciencia de datos, aprendiendo herramientas de linea de comando que son transversales a cualquier lenguaje de programación.

2.1. Ciencia de Datos

De acuerdo a la definición de Hilary Mason and Chris H. Wiggins [1], la *ciencia de datos* se define de acuerdo a 4 pasos:

1. Obtención de datos.
2. Depuración de datos.
3. Exploración de datos.
4. Modelación de datos.
5. Interpretación de datos.

En este curso veremos los primeros 4 pasos.

Aunque estos 5 pasos se discuten en forma lineal e incremental, en la practica es común el moverse de uno a otro o incluso realizarlos en paralelo (Figura 1). Es decir, la ciencia de datos es un proceso iterativo no lineal.

2.1.1. Obtención de datos

Sin datos, no hay ciencia de datos, por lo que el primer paso es obtenerlos. Para ello, se puede necesitar realizar uno o mas de las siguientes tareas:

- Descargar datos desde otra ubicación, por ejemplo, pagina web o servidor.
- Obtener datos desde una base de datos o a través de una API, por ejemplo, MySQL o Twitter.
- Extraer datos de un archivo, por ejemplo, html, excel, csv.
- Generar datos, ya sea mediante sensores o haciendo encuestas.

Los formatos mas usuales para almacenar datos son, texto plano (txt), csv, json, html, xml, excel.

2.1.2. Depuración de datos

No es poco usual que los datos obtenidos tengan valores faltantes, inconsistencias, errores, caracteres erróneos o columnas no interesantes. En tales casos, los datos deben depurarse o limpiarse antes de poder hacer cualquier cosa con ellos. La operaciones mas comunes incluyen:

- Filtrar lineas.
- Extraer algunas columnas.
- Reemplazar valores.
- Extraer palabras.

[1] A Taxonomy of Data Science, 2010.

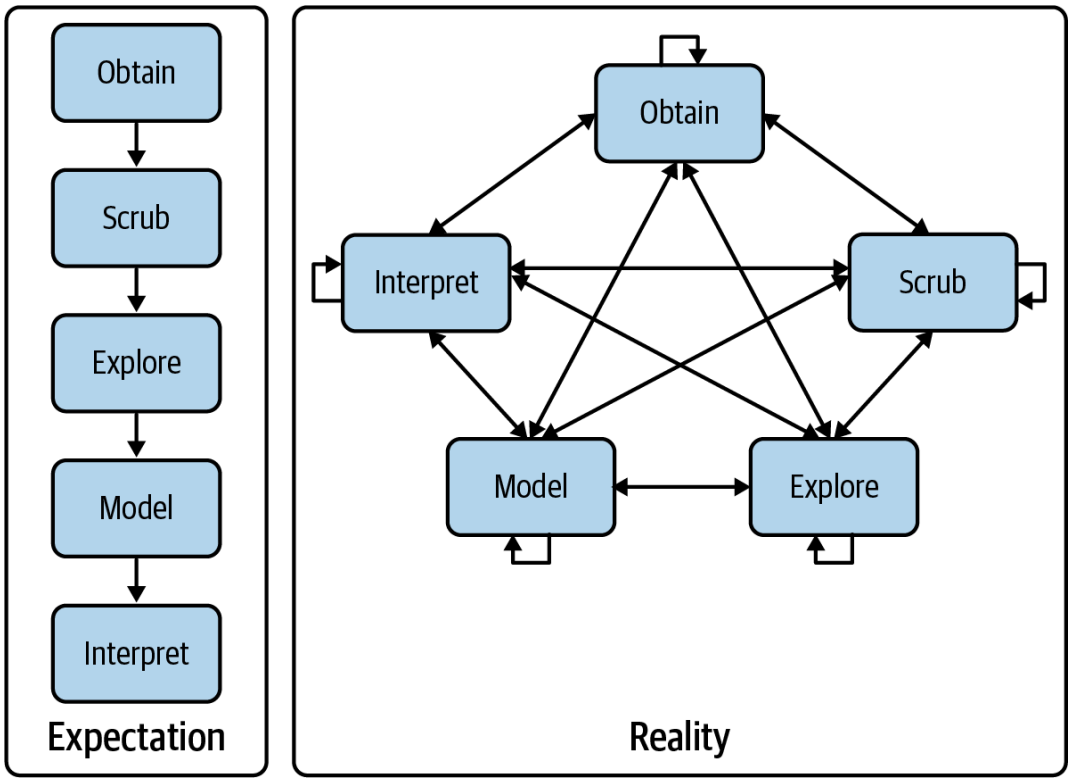


Figura 1: Proceso de la ciencia de datos.

- Lidiar con datos faltantes y duplicados.
- Convertir de un formato a otro.

Usualmente, un 80 % de cualquier trabajo con datos corresponde a la obtención y depuración. El otro 20 % es la exploración, modelación y visualización.

2.1.3. Exploración de datos

Una vez hecha la limpieza de datos, se procede a explorarlos. Es en esta etapa donde realmente se comprende de mejor forma los datos obtenidos. A través de la terminal se puede:

- Observar los datos.
- Derivar estadísticas.
- Crear visualizaciones.

2.1.4. Modelación de datos

Si se quiere explicar los datos o predecir lo que pasara, probablemente se necesita crear un modelo estadístico de los datos. Algunas técnicas para crear modelos incluyen clustering, clasificación, regresión y reducción de dimensionalidad.

La terminal no es apta para programar un nuevo modelo desde 0, pero es muy útil el ser capaz de

crear un modelo desde ella.

2.1.5. Interpretación de datos

El ultimo y probablemente mas importante paso es interpretarlos. Este paso involucra:

- Obtener conclusiones.
- Evaluación de resultados.
- Comunicación de resultados.

Esta etapa no involucra la terminal, y queda en manos del investigador.

2.2. Historia de la terminal

La *terminal* (*terminal emulator*) en un programa o interfaz que nos permite acceder a la *linea de comandos* (*command line*) a través de un tipo particular de sintaxis llamado *shell*.

Originalmente, una terminal era una pieza de *hardware* usado para ingresar y mostrar datos a/desde un computador.

Los primeros computadores, eran maquinas que ocupaban mucho espacio (ver Figura 2) y las primeras terminales eran tele-impresoras electrónicas (teletypewriter o tty) como se muestra en la (ver Figura 3).



Figura 2: Primeros computadores.

Conforme fue avanzando la tecnología, estas terminales se reemplazaron por terminales de video (ver Figura 4).

Notar que lo mostrado en la Figura 4 no es un computador, es solo la interfaz para poder conectarse al computador. En la Figura 5 se muestra conectada a uno.

En la actualidad no son necesarias estas terminales de *hardware*, por lo que se usan emuladores de terminales para realizar la comunicación directa con el sistema operativo sin necesidad de una interfaz gráfica (gui), esto ultimo siendo lo mas conocido (ventanas, botones, etc).



Figura 3: Primeras terminales.

2.2.1. Shell

La *shell* es la sintaxis particular que se utiliza para ejecutar comandos y programas en la terminal. Hay varios tipos de *shell*, como *bash*, *csh*, *zsh*, *nushell*, entre muchos otros, siendo la mas usual *bash* como la por defecto en la mayoría de distribuciones Linux.

En este curso, utilizaremos la *shell* llamada *zsh*, en la cual a cada comando lo precede un símbolo $, llamado *prompt*, e indica que la terminal esta lista para recibir comandos.

2.2.2. Ventajas del uso de la terminal

El uso de la terminal para ciencia de datos tienes varias ventajas, como por ejemplo:

- Es ágil: permite explorar datos de forma interactiva, obteniendo resultados de forma casi inmediata.
- Es integradora: se integra bien con otras tecnologías y software.
- Es escalable: se pueden automatizar muchas tareas mediante *scripts*, lo que da como resultado herramientas mas complejas.
- Es extensible: permite el uso de diferentes lenguajes de programación.
- Es ubicua: se encuentra en todos los sistemas operativos.

2.3. La terminal

En este capítulo visitaremos los conceptos fundamentales para trabajar con la terminal.

2.3.1. El ambiente

El ambiente en el que trabajaremos tiene 4 capas:



Figura 4: Primeras terminales de video.

1. **Herramientas de linea de comando**: el conjunto de herramientas y comandos que se ejecutaran.
2. **Terminal**: la aplicación en donde de ingresaran los comandos.
3. **Shell**: una vez se ingrese un comando en la terminal, esta lo envía a la *shell*. La *shell* lo interpreta de acuerdo a su sintaxis especifica y comunica las instrucciones al sistema operativo.
4. **Sistema operativo**: esta en contacto directo con la *cpu*, disco duro, *ram* y periféricos.

2.3.2. Ejecutar un comando

Los comandos se escriben directamente en la terminal. Para ejecutarlos se presiona `Enter`. Por ejemplo:

```
pwd
```

muestra el directorio actual. Al iniciar sesión, se comienza en el directorio llamado `home`.

```
ls
```

muestra el contenido del directorio actual.

```
mkdir carpeta
```

crea un directorio llamado `carpeta`. Verificamos con:

```
ls
```

Para ingresar a `carpeta`, ejecutamos:



Figura 5: Primeras terminales conectadas a un computador.

```
cd carpeta
```

```
pwd
```

Volvemos al directorio anterior:

```
cd ..
ls
```

Borramos `carpeta` con:

```
rm -r carpeta
```

```
ls
```

Si no se ha realizado, ir a la Sección 1.2.1.

Naveguemos al directorio de datos establecido anteriormente:

```
dsitcl
```

```
cd /data
ls
```

```
cd ch02
ls
```

```
cd
ls
```

Ejecutar `cd` a secas nos lleva de vuelta a `home`.

El comando:

```
cd /data/ch02/
head -n 5 movies.txt
```

Nos muestra las primeras 5 lineas del archivo `movies.txt` ubicado en el directorio `/data/ch02`. Al argumento `-n` se le llama *parámetro*.

> Si se obtiene un error de permisos al intentar hacer `cd`, salir del contenedor docker y ejecutar lo siguiente:

```
cd
chmod -R g+rwx DSITCL
```

O bien:

```
cd
chmod -R a+rwx DSITCL
```

> Verificamos ingresando al contenedor docker e intentando ingresar a la carpeta `/data/ch02`:

```
dsitcl
cd /data/ch02
ls
```

2.3.3. Tipos de herramientas

Hay al menos 5 tipos de herramientas de linea de comando:

1. **Binario ejecutable**: es un programa en el sentido clásico, se crea al compilar su código fuente a lenguaje de maquina.
2. **Incluido en la shell**: herramientas que proporciona la *shell* (`cd`, `pwd`, etc).
3. **Un script interpretado**: archivo de texto que es ejecutado por un binario ejecutable. Por ejemplo: scripts R, Python, bash, etc.

Por ejemplo, el archivo `fac.py`, contiene el siguiente script Python:

```
#!/usr/bin/env python

def factorial(x):
    result = 1
    for i in range(2, x + 1):
        result *= i
    return result

if __name__ == "__main__":
    import sys
```

```
    x = int(sys.argv[1])
    sys.stdout.write(f"{factorial(x)}\n")
```

Se puede visualizar con el comando:

```
bat fac.py
```

Y ejecutar como:

```
./fac.py 5
```

O:

```
python fac.py 5
```

4. **Una función de shell**: una función que es es ejecutada por la shell, similar a un script, pero tienden a ser mas pequeños. Por ejemplo:

```
fac() { (echo 1; seq $1) | paste -s -d\* - | bc; }
```

define una función llamada `fac`. Para ejecutarla, realizamos:

```
fac 5
```

Se recomienda definir las funciones que se utilizaran con frecuencia en el archivo `~/.zshrc`.

5. **Un alias**: son análogas a las macros. Nos permiten definir un comando largo con menos caracteres, tal como se hizo con el alias `dsitcl`. No permiten parámetros.

Las herramientas instaladas en el contenedor Docker son principalmente de los primeros 2 tipos. Los otros tipos nos permiten construir nuestras propias herramientas para ser mas productivos y eficientes.

Para encontrar a que tipo cada comando pertenece, se puede ejecutar:

```
type -a pwd

type -a cd

type -a fac

type -a l
```

Una lista mas comprehensiva se puede encontrar en el siguiente link: 60 comandos para la terminal Linux.

2.3.4. Combinando herramientas

Debido a que la mayoría de las herramientas de línea de comandos se adhieren a la filosofía Unix, están diseñadas para hacer solo una cosa y hacerla realmente bien. Por ejemplo, la herramienta de línea de comandos `grep` puede filtrar líneas, `wc` puede contar líneas y `sort` puede ordenar líneas.

El poder de la línea de comandos proviene de su capacidad para combinar estas pequeñas pero poderosas herramientas.

Este poder es posible gestionando los flujos (*streams*) de comunicación de estas herramientas. Estos suelen abreviarse como ***stdin***, ***stdout*** y ***stderr***.

Tanto la salida estándar como el error estándar se redirigen, por defecto, al terminal, de modo que tanto la salida normal como cualquier mensaje de error se imprimen en la pantalla. La Figura 6 ilustra esto tanto para `pwd` como para `rev`.

Si se ejecuta `rev`, se verá que no pasa nada. Eso es porque `rev` espera entrada y, por defecto, son todas las teclas presionadas en el teclado. Intenta escribir una oración y presiona **`Enter`**. `rev` responde inmediatamente con la entrada en reversa. Se puedes detene el envío de entrada presionando **`Ctrl-D`**, después de lo cual `rev` se detendrá.

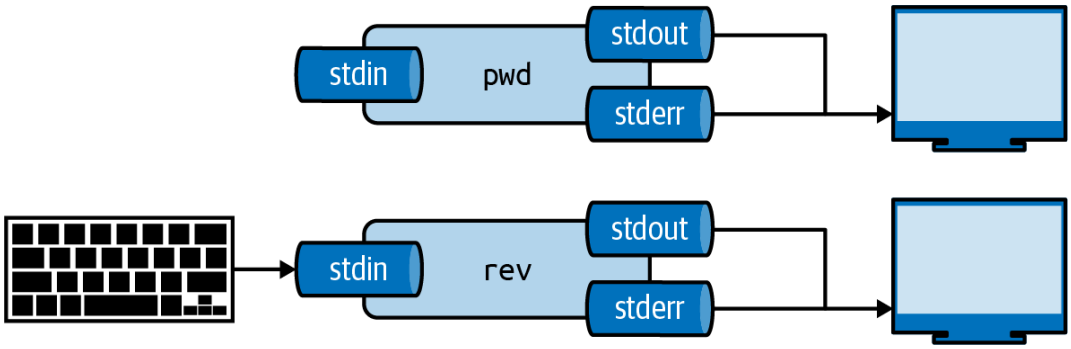


Figura 6: Streams estandar de herramientas de linea de comando

En la práctica, no se utilizará el teclado como fuente de entrada, sino la salida generada por otras herramientas y el contenido de archivos. Por ejemplo, con `curl` podemos descargar el libro *Las aventuras de Alicia en el País de las Maravillas* de Lewis Carrol y enviarlo (*pipe*) hacia la siguiente herramienta.

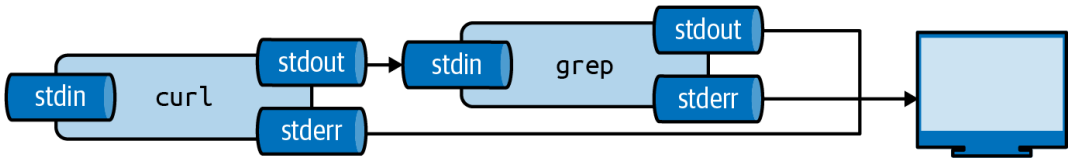


Figura 7: La salida de una herramienta se puede redirigir (pipe) hacia otra herramienta

Se puede enviar (*pipe*) la salida de `curl` hacia `grep` para filtrar líneas con algún patrón. Si se quisieran ver los capítulos listados en la tabla de contenidos, se pueden combinar `curl` y `grep` de la siguiente manera:

```
curl -s "https://www.gutenberg.org/files/11/11-0.txt"
curl -s "https://www.gutenberg.org/files/11/11-0.txt" | grep " CHAPTER"
```

Y si se quiere saber *cuántos* capítulos tiene el libro, se puede usar `wc`:

```
curl -s "https://www.gutenberg.org/files/11/11-0.txt" | grep " CHAPTER" | wc -l
```

La opción `-l` especifica que `wc` solo debe generar el número de líneas que se le pasan. De forma predeterminada, también devuelve el número de caracteres y palabras.

Se puede considerar el *pipe* como un copiar y pegar automático.

2.3.5. Redireccionando entrada y salida

Además de canalizar (*pipe*) la salida de una herramienta a otra herramienta, también puedes guardarla en un archivo. El archivo se guardará en el directorio actual, a menos que se proporcione una ruta completa. Esto se llama *redirección de salida*, y funciona de la siguiente manera:

```
curl "https://www.gutenberg.org/files/11/11-0.txt" | grep " CHAPTER" > chapters.txt
cat chapters.txt
```

Aquí, se guarda la salida de `grep` en un archivo llamado *chapters.txt* en el directorio */data/ch02*. Si este archivo aún no existe, se creará. Si este archivo ya existe, su contenido se sobrescribe. La Figura 8 ilustra cómo funciona conceptualmente la redirección de salida.

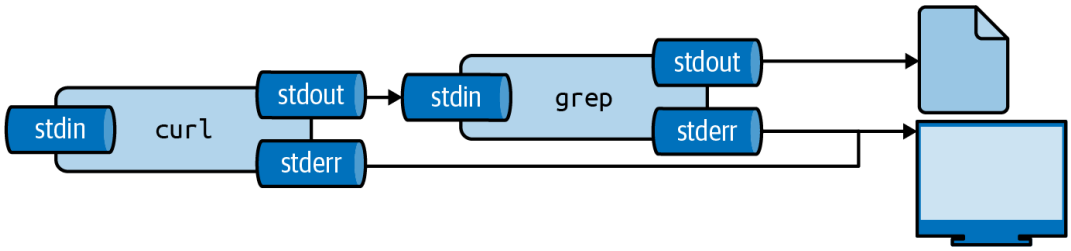


Figura 8: La salida de una herramienta se puede redirigir a un archivo

También se puede adjuntar la salida a un archivo con `>>`, lo que significa que la salida se agrega después del contenido original:

```
cd /data/ch02
echo -n "Hello" > greeting.txt
echo " World" >> greeting.txt
```

La herramienta `echo` entrega el valor que se especifique. La opción `-n`, que significa *nueva línea*, especifica que `echo` no debe generar una nueva línea final.

Guardar la salida a un archivo es útil si se necesita almacenar resultados intermedios, por ejemplo para continuar un análisis en una etapa posterior. Para utilizar el contenido del archivo *greeting.txt* nuevamente, podemos utilizar `cat`, que lee un archivo y lo imprime.

```
cat greeting.txt
cat greeting.txt | wc -w
```

La opción `-w` le indica a `wc` que solo cuente palabras.

El mismo resultado se puede lograr utilizando el signo menor que (<):

```
< greeting.txt wc -w
```

De esta forma, se está pasando el archivo directamente a la entrada estándar de `wc` sin ejecutar un proceso adicional. La Figura 9 ilustra cómo funcionan estas dos formas. El resultado final es el mismo.

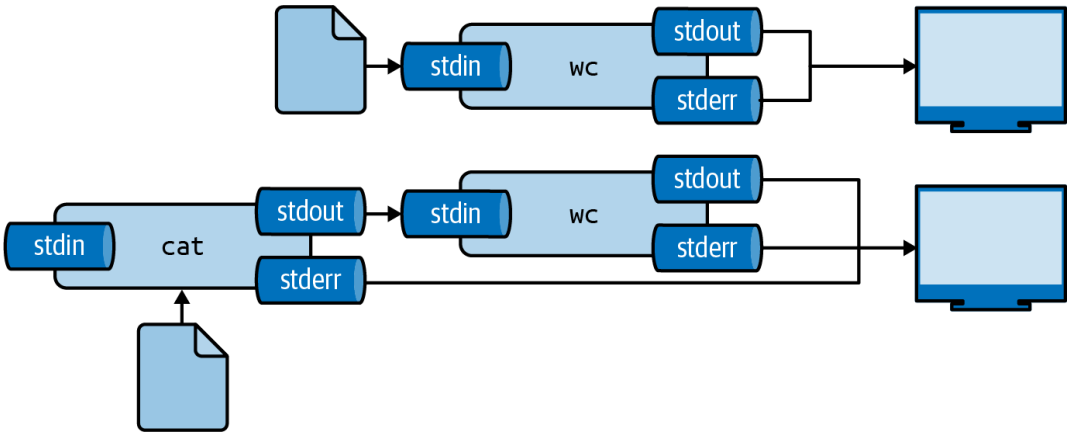


Figura 9: Dos formas de usar el contenido de un archivo como entrada

Como muchas herramientas de línea de comandos, `wc` permite que se especifiquen uno o más nombres de archivo como argumentos. Por ejemplo:

```
wc -w greeting.txt movies.txt
```

Se puede suprimir la salida de cualquier herramienta redirigiéndola a un archivo especial llamado */dev/null*. Se utiliza a menudo para suprimir los mensajes de error (ver Figura 10 para una ilustración). Lo siguiente hace que `cat` produzca un mensaje de error porque no puede encontrar el archivo *404.txt*:

```
cat movies.txt 404.txt
```

Se puede redirigir el error estándar a */dev/null* de la siguiente manera:

```
cat movies.txt 404.txt 2> /dev/null
```

> *2* se refiere al error estándar.

Se debe tener cuidado de no leer y escribir en el mismo archivo. Si se hace, se terminará con un archivo vacío. Esto se debe a que la herramienta cuya salida se redirige abre ese archivo inmediatamente para escribir y, por lo tanto, lo vacía. Hay dos soluciones para esto: (1) escribir en un archivo diferente y luego renombrarlo con `mv` o (2) usar `sponge`, que absorbe toda su entrada antes de escribir en un archivo. La Figura 11 ilustra cómo funciona.

Por ejemplo, si se usa `dseq` para generar un archivo *dates.txt* y se quisiera añadir números de línea usando `nl`. Ejecutando lo siguiente, el archivo *dates.txt* acabará vacío.

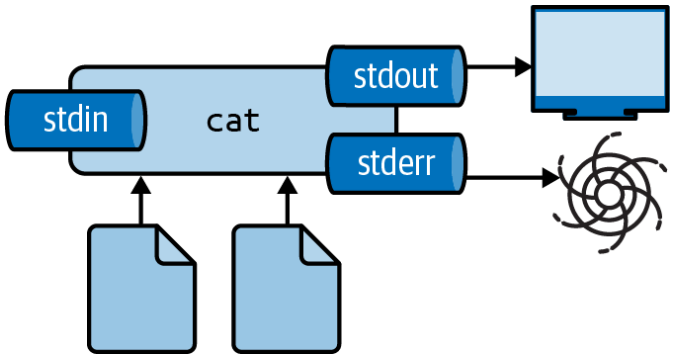


Figura 10: Redirigiendo `stderr` a `/dev/null`

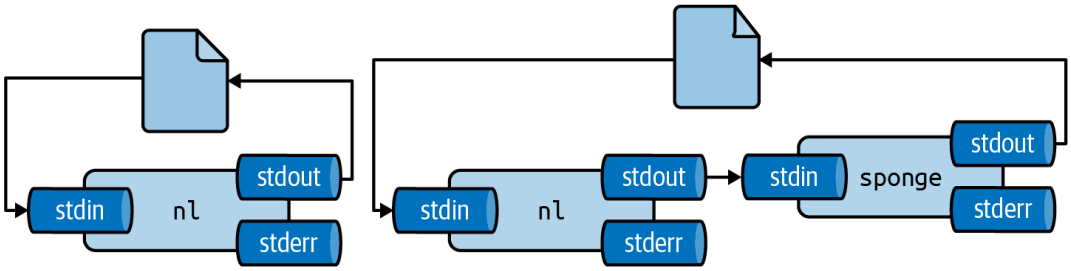


Figura 11: Redirigir y leer en una misma instrucción

```
dseq 5 > dates.txt
< dates.txt nl > dates.txt
bat dates.txt
```

En su lugar, se puede utilizar alguna de las soluciones alternativas descritas anteriormente:

```
dseq 5 > dates.txt
< dates.txt nl > dates-nl.txt
bat dates-nl.txt

dseq 5 > dates.txt
< dates.txt nl | sponge dates.txt
bat dates.txt
```

2.3.6. Trabajando con Archivos y Directorios

Como Estadístico, se trabaja con muchos datos. Estos datos a menudo se almacenan en archivos. Es importante saber cómo trabajar con archivos (y los directorios en los que residen) en la línea de comandos o terminal. Cada acción que se puede hacer usando una GUI (*Grafical User Interface*), se puede hacer con herramientas de línea de comandos (y mucho más). En esta sección se presentan las más importantes para listar, crear, mover, copiar, renombrar y eliminar archivos y directorios.

Listar el contenido de un directorio se puede hacer con `ls`. Si no se especifica un directorio, lista el

contenido del directorio actual. Es usualmente preferente que `ls` tenga un formato de lista larga y los directorios agrupados antes que los archivos. En lugar de escribir las opciones correspondientes cada vez, se puede utilizar el alias `l`.

```
ls /data/ch10
alias l
l /data/ch10
cd /data/ch10
```

Ya se ha visto cómo crear nuevos archivos redirigiendo la salida con `>` o `>>`. Si se necesitas mover un archivo a un directorio diferente, se puede usar `mv`:

```
touch hello.txt
mv hello.txt /data/ch02
```

También se puede renombrar archivos con `mv`:

```
cd /data/ch02
mv hello.txt bye.txt
```

También se puede renombrar o mover directorios enteros. Si ya no se necesita un archivo, se puede eliminar (o remover) con `rm`:

```
rm bye.txt
```

Si se quieres elimina un directorio entero con todo su contenido, se especifica la opción `-r`, que significa recursivo:

```
mkdir old
l
rm -r /data/ch02/old
l
```

Si se quiere copiar un archivo, se usa `cp`. Esto es útil para crear respaldos:

```
cp fac.py fac.py.bak
```

Se pueden crear directorios usando `mkdir`:

```
cd /data
mkdir logs
l
```

Todas las herramientas de línea de comandos anteriores aceptan la opción `-v`, que significa *verbose*, para que muestren lo que está pasando. Por ejemplo:

```
mkdir -v backup
cp -v * backup
```

Todas las herramientas excepto `mkdir` también aceptan la opción `-i`, que significa interactivo, y hace que las herramientas pidan confirmación. Por ejemplo:

```
touch logs.txt
rm -i logs.txt
```

2.3.7. Gestión de Salida

A veces, una herramienta o secuencia de herramientas produce demasiada salida. En lugar de alterar manualmente dicha salida, se puede hacer *pipe* hacia una herramienta auxiliar.

Una de esas herramientas es `trim`, que sirve para limitar la salida a una altura y anchura determinadas. Por ejemplo:

```
cat /data/ch07/tips.csv | trim 5 25
```

Otras herramientas similares son: `head`, `tail`, `fold`, `paste` y `column`.

Si la salida está separada por comas, se puede hacer *pipe* a través de `csvlook` para convertirla en una tabla de aspecto agradable. Si ejecutas `csvlook`, verás la tabla completa.

```
cat /data/ch07/tips.csv
csvlook /data/ch07/tips.csv
```

Se puede utilizar `bat` para mostrar el contenido de un archivo donde importan los números de línea y el resaltado de sintaxis. Por ejemplo, para mirar este código Python:

```
cat /data/ch04/stream.py
bat /data/ch04/stream.py
```

A veces es útil escribir una salida intermedia en un archivo. Esto permite inspeccionar cualquier paso de tu *pipeline* una vez que se haya completado. Se puede insertar la herramienta `tee` tantas veces como se necesite. A menudo la utilizo para inspeccionar una parte de la salida final, mientras escribo la salida completa en un archivo. En el siguiente ejemplo (ver Figura 12), la salida completa se escribe en `even.txt` y las primeras 5 líneas se imprimen usando `trim`:

```
seq 0 2 100 | trim 5
seq 0 2 100 | tee even.txt | trim 5
```

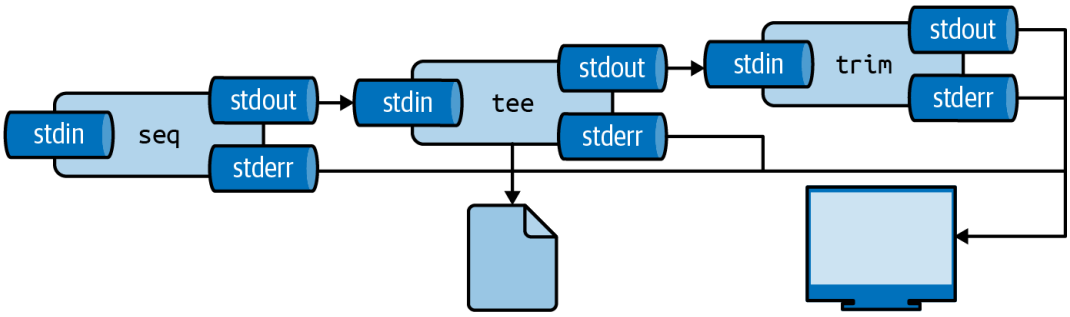


Figura 12: Con ‘tee‘, se puede escribir una salida intermedia en un archivo

2.3.8. Ayuda!

Al iniciarse con la línea de comandos, es usual necesites ayuda. Incluso los usuarios más experimentados necesitan ayuda en algún momento. Es imposible recordar todas las distintas herramientas de línea de comandos y sus posibles argumentos. Afortunadamente, la terminal ofrece varias formas de obtener ayuda.

Primeramente, toda herramienta o comando tiene una opción `-h` o `--help`, que entrega información breve acerca del comando. Por ejemplo:

```
cat --help
tee --help
tar --help
curl --help
```

El comando más importante para obtener ayuda es quizás `man`, que es la abreviatura de *manual*, que entrega información mas detallada acerca del uso de cada comando. Contiene información para la mayoría de las herramientas de la terminal. Por ejemplo (presionar `q` para salir del manual):

```
man cat
man tee
man tar
man curl
```

Las páginas de manual pueden ser bastante extensas y difíciles de leer, por lo que existe una herramienta que permite simplificar la obtención de ayuda, llamada `tldr`. Ésta es una colección de páginas de ayuda mantenidas por la comunidad para herramientas de línea de comandos, que pretende ser un complemento más simple y accesible a las páginas de manual tradicionales. Por ejemplo:

```
tldr --update #solo se ejecuta esta vez
tldr tar
```

Como se puede apreciar, en lugar de enumerar las muchas opciones alfabéticamente como suele hacer `man`, `tldr` entrega directamente una lista de ejemplos prácticos.

> El sitio web explainshell analiza un comando o una secuencia de comandos y proporciona una breve explicación de cada parte. Útil para comprender rápidamente un nuevo comando u opción sin tener que hojear las páginas de manual correspondientes.

2.4. Control de Versiones

Esta sección se introduce en el mundo del control de versiones con Git y cómo plataformas como GitHub y GitLab pueden potenciar el flujo de trabajo en ciencia de datos.

En el desarrollo de cualquier proyecto, especialmente en ciencia de datos, el código fuente evoluciona constantemente. Llevar un control de estos cambios resulta crucial para evitar errores, facilitar la

colaboración y garantizar la reproducibilidad de los análisis. Aquí es donde entra en juego el **control de versiones**.

El *control de versiones* es un sistema que registra los cambios realizados en un archivo o conjunto de archivos a lo largo del tiempo. Permite regresar a versiones anteriores, comparar diferentes versiones, identificar quién realizó un cambio específico y mucho más.

Este sistema es esencial para la ciencia de datos debido a:

- **Seguimiento de cambios:** Permite entender la evolución del código, facilitando la identificación y corrección de errores.
- **Colaboración eficiente:** En proyectos colaborativos, varios miembros pueden trabajar simultáneamente sin sobrescribir el trabajo del otro.
- **Restauración de versiones anteriores:** Permite volver a puntos específicos en el desarrollo, lo cual es útil si se introducen errores o se desea revertir un cambio.
- **Reproducibilidad:** Garantiza la consistencia del código y los resultados a lo largo del tiempo, lo que es fundamental en la investigación científica.

  Notar que en esta sección, trabajaremos fuera del contenedor docker.

2.4.1. Git: La herramienta esencial

Git es un sistema de control de versiones distribuido, ampliamente utilizado en la actualidad. A diferencia de sistemas centralizados, cada usuario posee una copia completa del historial del proyecto, lo que permite trabajar sin conexión y facilita la colaboración.

Conceptos básicos de Git:

- **Repositorio:** Un directorio que contiene todos los archivos del proyecto y su historial de cambios.
- **Rama (branch):** Una línea de desarrollo independiente dentro del repositorio. Permite trabajar en nuevas características o corregir errores sin afectar la rama principal.
- **Commit:** Una instantánea del estado del proyecto en un momento determinado. Cada commit contiene un mensaje que describe los cambios realizados.
- **Merge:** La acción de combinar los cambios de una rama a otra.

Estados de un archivo:

- **Modificado (modified):** Se han realizado cambios en el archivo, pero aún no se han guardado en el repositorio.
- **Preparado (staged):** Se han marcado los archivos modificados para incluirlos en el siguiente commit.
- **Confirmado (committed):** Los cambios se han guardado en el repositorio.

Comandos esenciales de Git:

- `git init`: Crea un nuevo repositorio Git en el directorio actual.
- `git clone <URL>`: Crea una copia local de un repositorio remoto existente.

- `git add <archivo>`: Agrega un archivo al área de preparación (staging area).
- `git commit -m "Mensaje del commit"`: Crea un nuevo commit con los archivos preparados.
- `git push`: Envía los commits locales al repositorio remoto.
- `git pull`: Descarga los cambios del repositorio remoto al repositorio local.
- `git branch`: Muestra las ramas existentes.
- `git checkout <nombre_de_la_rama>`: Cambia a la rama especificada.
- `git merge <nombre_de_la_rama>`: Combina la rama especificada con la rama actual.

Flujos de trabajo comunes con Git:

- **Flujo de trabajo centralizado:** Todos los desarrolladores trabajan en una sola rama principal. Es un flujo de trabajo simple, adecuado para proyectos pequeños.
- **Flujo de trabajo con ramas (feature branching):** Se crea una rama separada para cada nueva característica o corrección de errores. Permite un desarrollo más organizado y facilita la colaboración en proyectos grandes.

2.4.2. GitHub/GitLab

GitHub y **GitLab** son plataformas de alojamiento de código basadas en Git que ofrecen herramientas adicionales para la colaboración y gestión de proyectos.

Creación de un repositorio remoto:

1. Crear una cuenta gratuita en GitHub o GitLab.
2. Crear un nuevo repositorio e indicar un nombre y una descripción.

Conexión del repositorio local con el remoto:

1. En la configuración del repositorio remoto, copiar la URL de clonación.
2. En la terminal, navegar al directorio del repositorio local.
3. Ejecutar el comando `git remote add origin <URL_del_repositorio_remoto>`.

Colaboración en proyectos:

- **Clonar un repositorio existente:** Utilizar el comando `git clone <URL_del_repositorio>` para crear una copia local.
- **Bifurcación (fork) de repositorios:** Crear una copia personal de un repositorio para trabajar en él de forma independiente.
- **Pull requests:** Proponer cambios al repositorio original. Se realiza una solicitud al propietario del repositorio para que revise y combine los cambios.

Herramientas adicionales en GitHub/GitLab:

- **Gestión de issues:** Permite registrar errores, tareas y solicitudes de características.
- **Wiki:** Un espacio para documentar el proyecto de forma colaborativa.
- **Integración continua/despliegue continuo (CI/CD):** Permite automatizar pruebas, compilación y despliegue del código.

2.4.3. Mejores prácticas para proyectos de ciencia de datos

- **Organización de proyectos:** Utilizar una estructura de directorios clara para organizar el código, los datos y la documentación.
- **Archivos `.gitignore`:** Excluir del control de versiones archivos innecesarios, como datos sin procesar o archivos temporales.
- **Mensajes de commit informativos:** Describir claramente los cambios realizados en cada commit.
- **Gestión de dependencias:** Utilizar archivos de requisitos para especificar las librerías y versiones necesarias para ejecutar el código.

2.4.4. Clonar

Clonaremos un repositorio para mostrar el procedimiento, utilizando el repositorio `ddgr`, que permite búsquedas web en la terminal.

Fuera del contenedor docker (salir de él mediante `exit` o `Ctrl+d`), realizar lo siguiente:

Creamos directorios necesarios:

```
cd
mkdir -p bin
mkdir -p software
```

Clonamos el repositorio e instalamos:

```
cd software
git clone https://github.com/jarun/ddgr.git
cd ddgr
PREFIX=~/bin make install
```

Añadimos el directorio de instalación al `PATH` de sistema:

```
export PATH="${PATH}:~/bin"

cd ~/bin/bin
chmod +x ddgr
cd
```

Ejecutamos la herramienta:

```
ddgr como clonar un repositorio github
```

> Para salir presione q.
>
> Para poder visualizar los resultados de búsqueda, se debe tener configurado un navegador por defecto. Esto solo es posible fuera de la imagen docker.

2.4.5. Commit y Push

Para crear proyectos en Gitlab o Github, primero se debe crear una cuenta de usuario.

Github permite la existencia de solo un repositorio personal privado (en su versión gratis), mientras que Gitlab no tiene esa limitante.

Una vez creada la cuenta de usuario, se debe autentificar la maquina local mediante una clave `ssh`.

Creamos la clave (seleccione Si a todo):

```
ssh-keygen -t rsa
cat ~/.ssh/id_rsa.pub
```

Copie el contenido de ese archivo y cree una nueva clave ssh en la web de github/gitlab.

Ahora en la web de github/gitlab, creamos un nuevo repositorio vacío y seleccionamos un nombre.

En la máquina local, realizamos los siguiente para conectarla al repositorio.

```
git config --global init.defaultBranch main
cd ~/software
mkdir test #o algun nombre seleccionado
cd test
echo "# test" >> README.md
git init
git add README.md
git commit -m "first commit" #mensaje a eleccion
git branch -M main
git remote add origin git@github.com:your_user/your_repo_name.git
git push -u origin main
```

Esto sincroniza el directorio local con el repositorio.

Supongamos que hacemos algún cambio al contenido de este directorio. Por ejemplo, editamos el archivo `README.md`:

```
nano README.md
```

Para subir los cambios realizamos:

```
git add -A
git status
git commit -am "mensaje apropiado"
git push origin main
```

2.4.6. Hacer aportes a un repositorio

Para realizar aportes a un repositorio, o una *pull request* (PR), se debe realizar un siguiente:

1. **Crear una copia del repositorio o un *fork*.**

- En el navegador web, ir al repositorio de GitHubitlab al que desea contribuir.
- En la esquina superior derecha de la página del repositorio, hacer click en el bóton Fork. Esto crea una copia del repositorio en su cuenta de GitHub/Gitlab.

2. **Clonar su Repositorio Bifurcado.**

- En la página del *fork* en su repositorio copie la URL de clonación `ssh`.
- En la terminal, ejecutar:

```
git clone <copied_https_url>
```

Reemplace `<copied_https_url>` con la URL que copió en el paso anterior.

3. **Crear una Nueva Rama (*branch*).**

```
cd <repository_directory>
git checkout -b <your-branch-name>
```

Por ejemplo: `git checkout -b fix-typo-in-readme`

4. **Hacer sus Cambios y hacer el *commit*.**

- Realice los cambios deseados en el código o la documentación.
- Realice `git add`, `git commit` y `git push` según lo descrito en la sección anterior.

5. **Crear la Solicitud de Aporte (*Pull Request*).**

- **Navegar a GitHub:** Vuelva al repositorio original que bifurcó en GitHub.
- **Crear PR:** Por lo general, verá un aviso sobre su rama recientemente enviada. Haga clic en el botón Comparar y solicitar PR. Si no es así, vaya a la pestaña Solicitudes de extracción y haga clic en Nueva solicitud de extracción, luego seleccione su rama.
- **Revisar y Enviar:** Revise cuidadosamente sus cambios, agregue un título y una descripción a su solicitud de extracción explicando sus cambios y haga clic en Crear solicitud de extracción.

2.4.7. Sugerencias de mensajes commit

Algunas directrices al momento de elegir el mensaje del `commit`.

1. Elija una etiqueta. Algunas sugerencias:

- *build*: Cambios que afectan al sistema de compilación o a las dependencias externas (ejemplos de ámbitos: `gulp`, `broccoli`, `npm`).
- *chore*: Actualización de bibliotecas, derechos de autor u otra configuración del repositorio, incluye la actualización de dependencias.
- *ci*: Cambios en nuestros archivos de configuración y scripts de CI (ejemplos de ámbitos: Travis, Circle, GitHub Actions).
- *docs*: Cambios que no son de código, como corregir errores tipográficos o añadir nueva documentación.

- *feat*: Una confirmación de tipo *feat* introduce una nueva característica en la base de código.
- *fix*: Una confirmación de tipo *fix* corrige un error en su base de código.
- *perf*: Un cambio de código que mejora el rendimiento.
- *refactor*: Un cambio de código que no corrige un error ni añade una característica.
- *style*: Cambios que no afectan al significado del código (espacios en blanco, formato, punto y coma faltantes, etc.).
- *test*: Añadir pruebas que faltan o corregir las existentes.

2. Elija un título para la confirmación.
3. Resuma el *commit*, escribiendo una lista mínima de los cambio realizados.
4. El mensaje *commit* final debe tener el siguiente formato:

```
<etiqueta>: <título>

- Item 1
- Item 2
  etc
```

2.4.8. Recursos adicionales

- **Documentación oficial:**
  - Git: https://git-scm.com/docs
  - GitHub: https://docs.github.com/
  - GitLab: https://docs.gitlab.com/
- **Ejemplos de proyectos de ciencia de datos que utilizan Git y GitHub:**
  - TensorFlow
  - scikit-learn
  - Pandas

2.4.9. Para mayor exploración

Una lista completa de los comandos o herramientas utilizadas en este curso se encuentran en el siguiente link: https://jeroenjanssens.com/dsatcl/list-of-command-line-tools.

3. Obtención de datos

Este capítulo trata del primer paso del modelo OSEMN: la obtención de datos. Después de todo, sin datos, no hay mucha ciencia de datos que realizar. Asumiendo que los datos necesarios para resolver un problema de ciencia de datos ya existen, la primera tarea es obtener estos datos en un formato con el que se pueda trabajar.

Según la filosofía Unix, el texto es una interfaz universal. Casi todas las herramientas de la terminal toman texto como entrada, producen texto como salida o ambas cosas. Esta es la razón principal por la que las herramientas de línea de comandos pueden funcionar tan bien juntas. Sin embargo, incluso el texto puede presentarse en múltiples formas.

Los datos se pueden obtener de varias maneras: por ejemplo, descargándolos de un servidor, consultando una base de datos o conectándose a una API web. A veces, los datos vienen en formato comprimido o en formato binario, como una hoja de cálculo de Microsoft Excel.

En este capítulo, se presentarán varias herramientas que ayudan a abordar esto desde la línea de comandos, incluyendo: `curl`, `in2csv`, `sql2csv` y `tar`.

Mas precisamente, en este capítulo se aprenderá a:

- Copiar archivos locales en la imagen Docker.
- Descargar datos de Internet.
- Descomprimir archivos.
- Extraer datos de hojas de cálculo.
- Consultar bases de datos relacionales.
- Llamar a una API web.

Este capítulo comienza con los siguientes archivos:

```
cd /data/ch03
l
```

Si no existe esta carpeta o no tiene archivos, seguir las instrucciones de instalación al inicio de este documento.

3.1. Copiando Archivos Locales en el Contenedor Docker

Una situación común es ya poseer los archivos necesarios en nuestro propio computador. Esta sección explica cómo puedes introducir esos archivos en el contenedor Docker.

Si bien el contenedor Docker es un entorno virtual aislado, es posible transferir archivos dentro y fuera del contenedor Docker. El directorio local desde el que se ejecutaba `docker run` se mapea a un directorio en el contenedor Docker. Este directorio se llama */data*. Recordar que este no es el directorio de inicio, que es */home/dst*.

Si se tiene uno o más archivos en el computador local y se quiere aplicarle algunas herramientas de la línea de comandos, solo se debe copiar o mover los archivos a ese directorio mapeado. A modo

de ilustración, supongamos que tenemos un archivo llamado *logs.csv* en el directorio de Descargas.

Primero, dentro del contenedor Docker ejecutar:

```
cd /data
mkdir my-data
```

Si se está ejecutando Windows, abrir el símbolo del sistema o PowerShell y ejecutar los dos comandos siguientes:

VERIFICAR EN WINDOWS

```
cd %UserProfile%\Downloads
copy logs.csv MyDataScienceToolbox\
```

Si se está ejecutando Linux o macOS, abrir una terminal y ejecutar el siguiente comando en el sistema operativo (y no dentro del contenedor Docker):

```
cp ~/Downloads/logs.csv $HOME/DSITCL/my-data
```

3.2. Descargando desde Internet

Internet ofrece, sin duda, el mayor recurso de datos interesantes. La herramienta de línea de comandos `curl` puede considerarse la navaja suiza de la línea de comandos cuando se trata de descargar datos de Internet.

3.2.1. Presentando `curl`

Cuando se accede a una URL (*Uniform Resource Locator*), el navegador interpreta los datos que descarga. Por ejemplo, el navegador renderiza archivos HTML, reproduce archivos de vídeo automáticamente y muestra archivos PDF. Sin embargo, cuando se usa `curl` para acceder a una URL, descarga los datos y, por defecto, los imprime en la salida estándar. `curl` no realiza ninguna interpretación, pero afortunadamente otras herramientas de línea de comandos pueden utilizarse para procesar los datos posteriormente.

La invocación más sencilla de `curl` es especificar una URL como argumento de línea de comandos. Como ejemplo, descargaremos un artículo de Wikipedia:

```
curl "https://en.wikipedia.org/wiki/List_of_windmills_in_the_Netherlands"
```

Como se puede apreciar, `curl` descarga el HTML en bruto devuelto por el servidor de Wikipedia; no se realiza ninguna interpretación y todo el contenido se imprime inmediatamente en la salida estándar. Debido a la URL, se podría pensar que este artículo listaría todos los molinos de viento de los Países Bajos. Sin embargo, aparentemente hay tantos molinos de viento que cada provincia tiene su propia página.

Por defecto, `curl` muestra un medidor de progreso que muestra la velocidad de descarga y el tiempo estimado de finalización. Esta salida no se escribe en la salida estándar, sino en un canal separado, conocido como *error estándar*, por lo que no interfiere cuando se añade otra herramienta

al *pipeline*. Si bien esta información puede ser útil cuando se descargan archivos muy grandes, se puede especificar la opción `-s` para *silenciar* esta salida.

```
curl -s "https://en.wikipedia.org/wiki/List_of_windmills_in_Friesland" | pup -n 'table.wiki
```

Aparentemente hay 234 molinos de viento en la provincia de Friesland.

Se discutirá la herramienta `pup` mas adelante.

3.2.2. Guardado

Es posible que `curl` guarde la salida en un archivo agregando la opción `-O`. El nombre del archivo se basará en la última parte de la URL.

```
curl -s "https://en.wikipedia.org/wiki/List_of_windmills_in_Friesland" -o MolinosFriesland
l
```

O bien, redireccionando la salida:

```
curl -s "https://en.wikipedia.org/wiki/List_of_windmills_in_Friesland" > friesland.html
l
```

3.2.3. Otros protocolos

En total, `curl` admite más de 20 protocolos. Por ejemplo, para descargar desde un servidor FTP (*File Transfer Protocol*), se utiliza `curl` de la misma manera:

```
curl -s "ftp://ftp.gnu.org/welcome.msg"
```

Si la URL especificada es un directorio, `curl` listará el contenido de ese directorio. Cuando la URL está protegida por contraseña, puedes especificar un nombre de usuario y una contraseña con la opción `-u`.

El protocolo DICT permite acceder a varios diccionarios y buscar definiciones. A continuación la definición de *windmill* según el Diccionario internacional colaborativo de inglés:

```
curl -s "dict://dict.org/d:windmill"
```

Sin embargo, al descargar datos de Internet, lo más probable es que el protocolo sea HTTP, por lo que la URL comenzará con *http://* o *https://*.

3.2.4. Siguiendo redirecciones

Cuando se accede a una URL acortada, como las que empiezan por *http://bit.ly/* o *http://t.co/*, el navegador redirige automáticamente a la ubicación correcta. Sin embargo, con `curl`, se necesita especificar la opción `-L` o `--location` para que haga la redirección. Si no se hace, se puede obtener algo como:

```
curl -s "https://bit.ly/2XBxvwK"
curl -s -L "https://bit.ly/2XBxvwK"
```

A veces no se obtiene nada, como cuando se sigue la siguiente URL:

```
curl -s "https://youtu.be/dQw4w9WgXcQ"
```

Especificando la opción `-I` o `--head`, `curl` obtiene solo la cabecera HTTP de la respuesta, lo que permite inspeccionar el código de estado y otra información devuelta por el servidor.

```
curl -sI "https://youtu.be/dQw4w9WgXcQ"
```

La primera línea muestra el protocolo seguido del código de estado HTTP, que en este caso es 303. También se puede ver la ubicación a la que redirige esta URL.

Inspeccionar la cabecera y obtener el código de estado es una herramienta de depuración útil en caso de que `curl` no dé el resultado esperado. Otros códigos de estado HTTP comunes incluyen 404 (no encontrado) y 403 (prohibido). Wikipedia tiene una página que enumera todos los códigos de estado HTTP.

En resumen, `curl` es una herramienta útil de línea de comandos para descargar datos de Internet. Sus tres opciones más comunes son `-s` para silenciar el medidor de progreso, `-u` para especificar un nombre de usuario y contraseña, y `-L` para seguir automáticamente las redirecciones.

Para obtener más información, consultar su página de manual:

```
man curl
```

3.3. Descomprimiendo Archivos

Si el conjunto de datos original es muy grande o es una colección de muchos archivos, el archivo puede ser un archivo comprimido. Los conjuntos de datos que contienen muchos valores repetidos (como las palabras en un archivo de texto o las claves en un archivo JSON) son especialmente adecuados para la compresión.

Las extensiones de archivo comunes de archivos comprimidos son: *.tar.gz*, *.zip*, y *.rar*. Para descomprimirlos, se usan las herramientas de línea de comandos `tar`, `unzip` y `unrar`, respectivamente, o incluso `7z`.

Consideremos archivos *tar.gz* (en inglés *gzipped tarball*) como ejemplo. Para extraer un archivo llamado *logs.tar.gz*, se usa:

```
cd /data/ch03
ls
tar -xzf logs.tar.gz
ls
rm E1*
```

Es común combinar estas tres opciones cortas, pero también se pueden especificar por separado como `-x -z -f`. De hecho, muchas herramientas de comandos permiten combinar opciones que consisten en un solo carácter.

En este caso, las tres opciones `-x`, `-z` y `-f` especifican que `tar` debe *extraer* archivos de un archivo, usar *gzip* como el algoritmo de descompresión y usar el archivo *logs.tar.gz*. Sin embargo, como aún no se está familiarizado con este archivo, primero se debe examinar su contenido. Esto se puede hacer con la opción `-t` (en lugar de la opción `-x`):

```
tar -tzf logs.tar.gz
```

Este archivo contiene muchos archivos y no están dentro de un directorio. Para mantener limpio el directorio actual, primero creamos un nuevo directorio usando `mkdir` y extraemos esos archivos allí usando la opción `-C`.

```
mkdir logs
tar -xzf logs.tar.gz -C logs
```

Verificamos el número de archivos y algunos de sus contenidos:

```
l logs | wc -l
cat logs/*
```

Los métodos de depuración y exploración de estos archivos de registro se verán mas adelante.

3.4. Conversión de hojas de cálculo de Microsoft Excel a CSV

Para muchas personas, Microsoft Excel ofrece una forma intuitiva de trabajar con pequeños conjuntos de datos y realizar cálculos en ellos. Como resultado, muchos datos se incrustan en hojas de cálculo de Microsoft Excel. Estas hojas de cálculo se almacenan, según la extensión del nombre de archivo, en un formato binario propietario (*.xls*) o como una colección de archivos XML comprimidos (*.xlsx*). En ambos casos, la mayoría de las herramientas de línea de comandos no pueden utilizar los datos fácilmente.

Inicialmente se podría tener la tentación de convertir una hoja de cálculo a CSV abriéndola en Microsoft Excel o una variante de código abierto como LibreOffice Calc, y exportándola manualmente a CSV. Si bien esto funciona como una solución única, la desventaja es que no se adapta bien a múltiples archivos y no se puede automatizar. Además, cuando se está trabajando en un servidor, es probable que no exista una aplicación de este tipo disponible.

Afortunadamente, hay una herramienta de línea de comandos llamada `in2csv`, que convierte hojas de cálculo de Microsoft Excel en archivos CSV.

CSV significa valores separados por comas (*Comma Separated Values*). Trabajar con CSV puede ser complicado porque carece de una especificación formal.

Por lo general, se define el formato CSV según lo siguiente:

1. Cada registro está ubicado en una línea separada, delimitada por un salto de línea (). Tomemos, por ejemplo, el siguiente archivo CSV con información crucial sobre las Tortugas Ninja Mutantes Adolescentes:

```
bat -A tmnt-basic.csv
```

La opción `-A` hace que `bat` muestre todos los caracteres no imprimibles como espacios, tabulaciones y nuevas líneas.

2. El último registro del archivo puede o no tener un salto de línea final. Por ejemplo:

```
bat -A tmnt-missing-newline.csv
```

3. Puede haber un encabezado que aparece como la primera línea del archivo con el mismo formato que las líneas de registro normales. Este encabezado contendrá nombres correspondientes a los campos en el archivo y debe contener el mismo número de campos que los registros en el resto del archivo. Por ejemplo:

```
bat -A tmnt-with-header.csv
```

Como se puede observar, CSV por defecto no es demasiado legible. Se puede hacer *pipe* de los datos hacia una herramienta llamada `csvlook`, que los formateará en una tabla. Si los datos CSV no tienen encabezado, como *tmnt-missing-newline.csv*, entonces debe agregar la opción `-H`, de lo contrario, la primera línea se interpretará como el encabezado.

```
csvlook tmnt-with-header.csv
csvlook tmnt-basic.csv
csvlook -H tmnt-missing-newline.csv
```

La opción `-H` especifica que el archivo CSV no tiene encabezado.

Demostremos el uso de `in2csv` usando una hoja de cálculo que contiene información de vehículos. Para extraer sus datos, utilizar `in2csv` de la siguiente manera:

```
curl -L "https://bit.ly/3WnqUpB" > cars.xlsx
in2csv cars.xlsx | tee cars.csv | trim
csvlook cars.csv | trim
```

Podemos buscar una marca en particular de la siguiente forma:

```
csvgrep cars.csv --columns make --regex '^toyota$' | csvlook -I | trim
```

El valor después de las opciones `--regex` es una expresión regular (o *regex*). Es una sintaxis especial para definir patrones. Aquí, solo queremos que coincidan los vehículos que coincidan exactamente con *toyota*, así que uso el circunflejo (`^`) y el signo de pesp (`$`) para que coincidan con el inicio y el final de los valores en la columna ***make***.

Por cierto, las herramientas `in2csv`, `csvgrep` y `csvlook` son parte de CSVkit, que es una colección de herramientas de línea de comandos para trabajar con datos CSV.

El formato del archivo se determina automáticamente por la extensión, *.xlsx* en este caso. Si se tuviera que hacer *pipe* de los datos a `in2csv`, se tendría que especificar el formato explícitamente.

Una hoja de cálculo puede contener varias hojas de trabajo. `in2csv` extrae, por defecto, la primera hoja de trabajo. Para extraer una hoja de trabajo diferente, debe pasar el nombre de la hoja de

trabajo a la opción `--sheet`. Si no se está seguro de cómo se llama la hoja de trabajo, se puede utilizar la opción `--names`, que imprime los nombres de todas las hojas de trabajo.

Aquí vemos que *cars.xlsx* tiene solo una hoja, llamada *autos*.

```
in2csv --names cars.xlsx
```

3.5. Consultando Bases de Datos Relacionales

Muchas compañías almacenan sus datos en una base de datos relacional. Al igual que con las hojas de cálculo, podemos trabajar con este tipo de datos desde la línea de comandos.

Ejemplos de bases de datos relacionales son MySQL, PostgreSQL y SQLite. Todas estas bases de datos tienen una forma ligeramente diferente de interactuar con ellas. Algunas proveen una herramienta de línea de comandos o una interfaz de línea de comandos, mientras que otras no. Además, no son muy consistentes cuando se trata de su uso y salida.

Afortunadamente, hay una herramienta de línea de comandos llamada `sql2csv`, que es parte de la suite CSVkit. Funciona con muchas bases de datos diferentes a través de una interfaz común, incluyendo MySQL, Oracle, PostgreSQL, SQLite, Microsoft SQL Server y Sybase. La salida de `sql2csv` es, como su nombre lo sugiere, en formato CSV.

Podemos obtener datos de bases de datos relacionales ejecutando una consulta ***SELECT*** sobre ellas. (`sql2csv` también soporta consultas ***INSERT***, ***UPDATE*** y ***DELETE***, pero ese no es el propósito de este capítulo.)

`sql2csv` necesita dos argumentos: `--db`, que especifica la URL de la base de datos, cuya forma típica es: `dialecto+controlador://usuario:contraseña@host:puerto/basededatos` y `--query`, que contiene la consulta ***SELECT***. Por ejemplo, dada una base de datos SQLite que contiene los conjuntos de datos estándar de `R`, se puede seleccionar todas las filas de la tabla *mtcars* y ordenarlas por la columna *mpg* de la siguiente manera:

```
sql2csv --db 'sqlite:///r-datasets.db' --query 'SELECT row_names FROM mtcars ORDER BY mpg'
```

Esta base de datos SQLite es un archivo local, así que en este caso no se necesita especificar ningún usuario, contraseña o host.

3.6. Llamando a las API web

En la sección anterior se explicó cómo descargar archivos desde Internet. Otra forma en que los datos pueden venir de Internet es a través de una API web, que significa *Interfaz de programación de aplicaciones* (*Application Programming Interface*). El número de APIs que se ofrecen está creciendo a un ritmo cada vez mayor, lo que significa muchos datos interesantes para trabajar.

Las API web no están pensadas para presentarse en un diseño agradable, como los sitios web. En cambio, la mayoría de las API web devuelven datos en un formato estructurado, como JSON o XML.

Tener datos en forma estructurada tiene la ventaja de que los datos pueden ser procesados fácilmente por otras herramientas, como `jq`. Por ejemplo, la API de *Ice and Fire*, que contiene mucha información sobre el mundo ficticio de George R.R. Martin, en el que tienen lugar el libro y los programas de televisión de Juego de Tronos, devuelve datos en la siguiente estructura JSON:

```
curl -s "https://anapioficeandfire.com/api/characters/583" | jq '.'
```

Se hace *pipe* de los datos hacia una herramienta de línea de comandos `jq` solo para mostrarlos de una manera agradable. `jq` tiene muchas más posibilidades de depuración y exploración que se explorarán posteriormente.

3.6.1. Autenticación

Algunas API web requieren de autenticación (es decir, ingresar credenciales de acceso) antes de que se pueda utilizar. Hay varias formas de hacer esto. Algunas API web utilizan claves API mientras que otras utilizan el protocolo OAuth.

News API, una fuente independiente de titulares y artículos de noticias, es un gran ejemplo. Esto sucede cuando se intenta acceder a esta API sin una clave API:

```
curl -s "http://newsapi.org/v2/everything?q=linux" | jq .
```

La parte después del signo de interrogación, por cierto, es donde se entrega cualquier parámetro de consulta. Ese es también el lugar donde se necesita especificar una clave API. Un ejemplo con uso de clave sería (almacenada en `/data/.secret/newsapi.org_apikey`):

```
curl -s "http://newsapi.org/v2/everything?q=linux&apiKey=$(cat /data/.secret/newsapi.org_ap
```

Para que el código anterior se ejecute correctamente, se debe obtener una clave API en el sitio web de News API, copiar su valor y ejecutar:

```
cd /data
mkdir .secret
nano newsapi.org_apikey
```

Copiar el valor de la API key y guardar. Luego ejecutar nuevamente el código anterior.

3.6.2. API de transmisión (streaming)

Algunas API web devuelven datos de forma continua. Esto significa que una vez que se conectan a ellas, los datos continuarán llegando hasta que se cierre la conexión. Un ejemplo bien conocido es el *firehose* de Twitter, que transmite constantemente todos los tweets que se envían por todo el mundo. Afortunadamente, la mayoría de las herramientas de línea de comandos también funcionan de forma continua.

Descargamos una muestra de 10 segundos de una de las API de transmisión de Wikimedia, por ejemplo:

```
curl -s "https://stream.wikimedia.org/v2/stream/recentchange" | sample -s 10 > wikimedia-s
```

Esta API en particular devuelve todos los cambios que se han realizado en Wikipedia y otras propiedades de Wikimedia. La herramienta de línea de comandos `sample` se utiliza para cerrar la conexión después de 10 segundos. La conexión también se puede cerrar manualmente presionando `Ctrl-C` para enviar una interrupción. La salida se guarda en un archivo *wikimedia-stream-sample*. Se puede ver el contenido usando `trim`:

```
cat wikimedia-stream-sample | trim
```

Con un poco de `sed` y `jq` se pueden depurar estos datos para obtener una idea de los cambios que ocurren en la versión en inglés de Wikipedia:

```
cat wikimedia-stream-sample | sed -n 's/^data: //p' |
jq 'select(.type == "edit" and .server_name == "en.wikipedia.org") | .title'
```

- La expresión `sed` solo imprime líneas que comienzan con *`data:`* e imprime la parte después del punto y coma, que resulta ser JSON.
- La expresión `jq` imprime la clave *`title`* de los objetos JSON que tienen un determinado *`type`* y *`server_name`*.

3.7. Para Mayor Exploración

- El repositorio de GitHub *Awesome Public Datasets* enumera cientos de conjuntos de datos de alta calidad que están disponibles públicamente.
- El repositorio de GitHub *Public APIs* enumera muchas API gratuitas. City Bikes y The One API se encuentran entre mis favoritas.
- Escribir consultas SQL para obtener datos de una base de datos relacional es una habilidad importante. Las primeras 15 lecciones del libro *SQL in 10 Minutes a Day* de Ben Forta enseñan la instrucción *`SELECT`* y sus capacidades de filtrado, agrupación y clasificación.

4. Creando Herramientas de Línea de Comandos

A lo largo de este curso, se presentaran muchos comandos y *pipelines* que básicamente caben en una línea. Estos se conocen como *one-liners* o *pipelines*. Ser capaz de realizar tareas complejas con solo un *one-liner* es lo que hace que la línea de comandos sea poderosa. Es una experiencia muy diferente a escribir y usar programas tradicionales.

Algunas tareas se realizan solo una vez y otras se realizan con más frecuencia. Algunas tareas son muy específicas y otras pueden generalizarse. Si se necesita repetir un determinado *one-liner* de forma regular, conviene convertirlo en una herramienta de línea de comandos por sí misma. Entonces, tanto los *one-liners* como las herramientas de línea de comandos tienen sus usos. Reconocer la oportunidad requiere práctica y habilidad. Las ventajas de una herramienta de línea de comandos son que no hay que recordar todo el *one-liner* y que mejora la legibilidad si se incluye en algún otro *pipeline*. En ese sentido, se puede pensar en una herramienta de línea de comandos como similar a una función en un lenguaje de programación.

El beneficio de trabajar con un lenguaje de programación, sin embargo, es que el código está en uno o más archivos. Esto significa que se puede editar y reutilizar ese código fácilmente. Si el código tiene parámetros, incluso se puede generalizar y volver a aplicar a problemas que siguen un patrón similar.

Las herramientas de línea de comandos tienen lo mejor de ambos mundos: se pueden usar desde la línea de comandos, aceptan parámetros y solo tienen que crearse una vez. En este capítulo, primero, se explica cómo convertir esos *one-liners* en herramientas de línea de comandos reutilizables. Al agregar parámetros a los comandos, se pueden agregar la misma flexibilidad que ofrece un lenguaje de programación. Posteriormente, se muestra cómo crear herramientas de línea de comandos reutilizables a partir de código escrito en un lenguaje de programación. Siguiendo la filosofía Unix, el código se puede combinar con otras herramientas de línea de comandos, que pueden estar escritas en un lenguaje completamente diferente. En este capítulo, nos enfocaremos en tres lenguajes de programación: `Bash`, `Python` y `R`.

Este capitulo comienza con los siguientes archivos:

```
cd /data/ch04
l
```

4.1. Conversión de one-liner into shell scripts

Digamos que nos gustaría obtener las palabras más frecuentes utilizadas en un texto. Tomemos por ejemplo el libro *Las aventuras de Alicia en el país de las maravillas* de Lewis Carroll, que está, como muchos otros libros, disponible gratuitamente en el Proyecto Gutenberg.

```
curl -sL "https://www.gutenberg.org/files/11/11-0.txt" | trim
```

La siguiente secuencia de herramientas o *pipeline* debería hacer el trabajo:

```
curl -sL "https://www.gutenberg.org/files/11/11-0.txt" |
```

```
tr '[:upper:]' '[:lower:]' |
grep -oE "[a-z\']{2,}" |
sort |
uniq -c |
sort -nr |
head -n 10
```

Cada linea realiza lo siguiente:

1. Descargar un ebook usando `curl`.
2. Convertir todo el texto a minúsculas usando `tr`.
3. Extraer todas las palabras usando `grep` y poner cada palabra en una línea separada.
4. Ordenar estas palabras en orden alfabético usando `sort`.
5. Eliminar todos los duplicados y contar cuántas veces aparece cada palabra en la lista usando `uniq`.
6. Ordenar esta lista de palabras únicas por su cuenta en orden descendente usando `sort`.
7. Mantener solo las 10 primeras líneas (es decir, palabras) usando `head`.

Esas palabras efectivamente aparecen con mayor frecuencia en el texto. Porque esas palabras (aparte de la palabra alice) aparecen con mucha frecuencia en muchos textos en inglés, tienen muy poco significado. De hecho, estas son conocidas como *stopwords*. Si nos deshacemos de esas, nos quedamos con las palabras más frecuentes que están relacionadas con este texto.

Aquí hay una lista de *stopwords* del idioma inglés:

```
curl -sL "https://raw.githubusercontent.com/stopwords-iso/stopwords-en/master/stopwords-en
sort | tee stopwords | trim 20
```

Con `grep` podemos filtrar estas palabras justo antes de empezar a contar:

```
curl -sL "https://www.gutenberg.org/files/11/11-0.txt" |
tr '[:upper:]' '[:lower:]' |
grep -oE "[a-z\']{2,}" |
sort |
grep -Fvwf stopwords |
uniq -c |
sort -nr |
head -n 10
```

> `grep -Fvwf stopwords` significa: obtener los patrones de un archivo (*stopwords* en este caso), uno por línea, con `-f`. Interpretar esos patrones como cadenas fijas con `-F`. Seleccionar solo aquellas líneas que contengan coincidencias que formen palabras completas con `-w`. Seleccionar líneas no coincidentes con `-v`.
>
> Cada herramienta de línea de comandos utilizada en este *one-liner* ofrece una página de manual. Así que en caso de que se quiera saber más sobre, por ejemplo, `grep`, se puede ejecutar `man grep` desde la línea de comandos. Las herramientas de línea de

comandos `tr`, `grep`, `uniq` y `sort` se discutirán con más detalle en el siguiente capítulo.

No hay nada malo en ejecutar este *one-liner* solo una vez. Sin embargo, si se quisiera tener las 10 palabras principales de cada libro electrónico en el Proyecto Gutenberg, o se quisiera las 10 palabras principales de un sitio web de noticias cada hora, ejecutarlo cada vez no seria eficiente. En esos casos, es mejor tener este *one-liner* como un bloque de construcción separado que pueda ser parte de algo más grande.

Para agregar algo de flexibilidad a este *one-liner* en términos de parámetros, vamos a convertirlo en un script de shell (*shell script*). Esto permite tomar el *one-liner* como punto de partida y mejorarlo gradualmente.

Para convertir este *one-liner* en una herramienta de línea de comandos reutilizable, se requieren 6 pasos:

1. Copiar y pegar el *one-liner* en un archivo.
2. Agregar permisos de ejecución.
3. Definir un llamado *shebang*.
4. Eliminar la parte fija de entrada.
5. Agregar un parámetro.
6. Opcionalmente extender el *PATH*.

4.1.1. Paso 1: Crear Archivo

El primer paso es crear un nuevo archivo. Se puedes abrir un editor de texto y copiar y pegar el *one-liner*.

Nombremos el archivo *top-words-1.sh* para indicar que este es el primer paso hacia nuestra nueva herramienta de línea de comandos. Si es necesario permanecer en la línea de comandos, se puede usar el comando integrado `fc`, que significa *fix command*, que permite corregir o *editar* el último comando ejecutado:

```
fc
```

Ejecutar `fc` invoca el editor de texto predeterminado, que se almacena en la variable de entorno *EDITOR*. En el contenedor Docker, esto se establece en `nano`, un editor de texto sencillo. Como se puede observar, este archivo contiene el *one-liner*.

Presione **`Ctrl-O`** para guardar el archivo, escriba `top-words-1.sh`, luego presione **`Enter`** y luego presione `y` para confirmar. Finalmente presione **`Ctrl-X`** para salir de `nano`.

Verificamos el archivo:

```
l
bat top-words-1.sh
```

Estamos usando la extensión de archivo *.sh* para dejar en claro que estamos creando un *shell script*. Sin embargo, las herramientas de línea de comandos no necesitan tener una extensión. De hecho, las herramientas de línea de comandos rara vez tienen extensiones.

Ejecutamos el archivo de la siguiente manera:

```
bash top-words-1.sh
```

Esto nos ahorra de escribir la línea de código de nuevo la próxima vez.

Sin embargo, debido a que el archivo no se puede ejecutar por sí solo, todavía no es una *verdadera* herramienta de línea de comandos.

4.2. Paso 2: Dar permisos de Ejecución

La razón por la que no podemos ejecutar el archivo directamente es que no se tienen los permisos de acceso correctos. Los usuarios, necesita tener permiso para ejecutar el archivo.

Para comparar las diferencias entre pasos, copiamos el archivo a *top-words-2.sh* usando

```
cp -v top-words-{1,2}.sh
l
```

Para cambiar los permisos de acceso de un archivo, necesitamos usar una herramienta de línea de comandos llamada `chmod`, que significa *cambiar modo*. Cambia los bits del modo de archivo de un archivo específico. El siguiente comando le otorga al usuario, permiso para ejecutar *top-words-2.sh*:

```
chmod u+x top-words-2.sh
l
```

El argumento `u+x` consta de tres caracteres:

1. `u` indica que queremos cambiar los permisos para el usuario propietario del archivo, que es usted, porque usted creó el archivo;
2. `+` indica que queremos agregar un permiso; y
3. `x`, que indica los permisos para ejecutar.

Veamos ahora los permisos de acceso de ambos archivos:

```
l top-words-{1,2}.sh
```

La primera columna muestra los permisos de acceso para cada archivo.

Para *top-words-2.sh*, esto es `-rwxrw-r--`.

El primer carácter `-` (guion) indica el tipo de archivo. Un `-` significa archivo regular y un `d` significa directorio.

Los siguientes tres caracteres, `rwx`, indican los permisos de acceso para el usuario propietario del archivo. Los `r` y `w` significan *leer* y *escribir*, respectivamente. Como se puede observar, *top-words-1.sh* tiene un `-` en lugar de un `x`, lo que significa que no se puede *ejecutar* ese archivo. Los siguientes tres caracteres `rw-` indican los permisos de acceso para todos los miembros del grupo propietario del archivo.

Finalmente, los últimos tres caracteres de la columna, `r--`, indican los permisos de acceso para todos los demás usuarios.

Ahora se puede ejecutar el archivo de la siguiente manera:

```
./top-words-2.sh
```

Si intenta ejecutar un archivo para el que no se tienen los permisos de acceso correctos, como con *top-words-1.sh*, se obtiene el siguiente mensaje de error:

```
./top-words-1.sh
```

4.2.1. Paso 3: Definir shebang

Aunque ya se puede ejecutar el archivo por sí solo, se debe agregar el llamado *shebang* al archivo. El *shebang* es una línea especial en el script que le indica al sistema qué ejecutable debe usar para interpretar los comandos.

No es una buena idea omitirlo, como se hizo anteriormente, porque cada *shell* tiene un ejecutable predeterminado diferente. El *shell Z*, el que se esta usando a lo largo del curso, usa el ejecutable */bin/sh* por defecto si no se define un *shebang*.

En este caso, es mas útil que `bash` interprete los comandos, ya que nos dará un poco más de funcionalidad que `sh`.

Creamos una copia y lo editamos:

```
cp -v top-words-{2,3}.sh
nano top-words-3.sh
```

Escribimos ***#!/usr/bin/env/bash*** al inicio del archivo y presionamos **Enter**. Luego presiona **Ctrl-X** para guardar y salir.

Confirmamos los cambios:

```
bat top-words-3.sh
```

A veces nos encontraremos con scripts que tienen un shebang en la forma de ***!/usr/bin/bash*** o ***!/usr/bin/python*** (en el caso de Python, como se verá en la siguiente sección). Aunque esto generalmente funciona, si los ejecutables `bash` o `python` están instalados en una ubicación diferente a */usr/bin*, entonces el script ya no funcionará. Por ello, es mejor usar la forma que se presento aquí, a saber, ***!/usr/bin/env bash*** y ***!/usr/bin/env python***, porque el ejecutable `env` sabe dónde están instalados `bash` y `python`.

En resumen, usar `env` hace que los scripts sean más portátiles.

4.2.2. Paso 4: Remover la entrada fija

Ahora tenemos una herramienta valida que podemos ejecutar desde la terminal. Pero se puede mejorar, haciéndola mas reutilizable.

Notamos que el primer comando del archivo es `curl`, que descarga el texto desde el que deseamos obtener las 10 palabras mas usadas. De esta forma, los datos y las operaciones están en un solo lugar.

Pero, como hacerlo si quisieramos obtener las 10 palabras mas usadas de otro e-book, o cualquier otro texto. Los datos de entrada están fijos en el script, sería mucho mejor de separar los datos de la herramienta.

Si asumimos que el usuario proveerá el texto a analizar, la herramienta se volverá aplicable de forma mas general. Así, la solución es remover el comando `curl` del script, quedando de la siguiente forma:

```
cp -v top-words-{3,4}.sh
sed -i '2d' top-words-4.sh
bat top-words-4.sh
```

Esto funciona porque el script comienza con un comando que necesita datos desde `stdin`, como `tr`. La utilizamos de la siguiente manera:

```
curl -sL 'https://www.gutenberg.org/files/11/11-0.txt' | ./top-words-4.sh
curl -sL 'https://www.gutenberg.org/files/12/12-0.txt' | ./top-words-4.sh
man bash | ./top-words-4.sh
```

4.2.3. Paso 5: Añadir argumentos

Hay un paso mas para hacer al script aun mas reutilizable, añadir parámetros. En él, hay argumentos fijos como `-nr` para `sort` o `-n 10` para `head` y sería bastante útil, por ejemplo, poder dar diferentes valores al argumento de `head` para definir arbitrariamente que lista de las palabras mas usadas se requieren. Esto esta implementado en el archivo *top-words-5.sh*:

```
cp /data/.cache/top-words top-words-5.sh
bat top-words-5.sh
```

Aquí, la variable `NUM_WORDS` se establece al valor de `$1`, que es una variable especial en Bash, que contiene el valor del primer argumento que se entrega desde la terminal. Si no se entrega un argumento, tomará por defecto el valor 10.

Notar que para usar el valor de `NUM_WORDS`, se debe utilizar un `$`: `$NUM_WORDS`.

Podríamos haber usado también *$1* directamente como un argumento para `head` y no crear una variable extra como *NUM_WORDS*. Sin embargo, con scripts más grandes y unos cuantos argumentos de línea de comandos más como *$2* y *$3*, el código se vuelve más legible cuando se usan variables con nombre.

Ahora, si quisiéramos ver las 20 palabras mas usadas en un texto, utilizamos el script como sigue:

```
curl -sL "https://www.gutenberg.org/files/11/11-0.txt" > alice.txt
< alice.txt ./top-words-5.sh 20
```

Si el usuario no especifica un numero, se mostraran las 10 palabras mas usadas:

```
< alice.txt ./top-words-5.sh
```

4.2.4. Paso 6: Extiender el PATH

Después de los cinco pasos anteriores, finalmente se ha terminado de construir una herramienta de línea de comandos reutilizable. Sin embargo, hay un paso más que puede ser muy útil. En este paso opcional, se asegura de que se pueda ejecutar las herramientas de línea de comandos desde cualquier lugar.

Actualmente, cuando se desea ejecutar una herramienta de línea de comandos, se debe navegar hasta el directorio en el que se encuentra o incluir el nombre de ruta completo como se muestra en el paso 2. Esto está bien si la herramienta de línea de comandos está diseñada específicamente para, digamos, un determinado proyecto. Sin embargo, si la herramienta de línea de comandos pudiera aplicarse en múltiples situaciones, entonces es útil poder ejecutarla desde cualquier lugar, al igual que las herramientas de línea de comandos que vienen con Ubuntu.

Para lograr esto, Bash necesita saber dónde buscar las herramientas de línea de comandos. Lo hace recorriendo una lista de directorios que se almacenan en una variable de entorno llamada *PATH*. En un contenedor Docker nuevo, el *PATH* se ve así:

```
echo $PATH
```

Los directorios están delimitados por dos puntos. Podemos imprimirlo como una lista de directorios *traduciendo* los dos puntos a nuevas líneas:

```
echo $PATH | tr ':' '\n'
```

Para cambiar el *PATH* permanentemente, se deberá editar el archivo *.bashrc* o *.profile* ubicado en el directorio de inicio. Si se colocan todas sus herramientas de línea de comandos personalizadas en un directorio, digamos, *~/tools*, entonces solo se cambia el *PATH* una vez. Ahora, ya no se necesita agregar *./*, sino que puede usar el nombre del archivo. Además, ya no se necesita recordar dónde se encuentra la herramienta de línea de comandos.

```
cp -v top-words{-5.sh,}
export PATH="${PATH}:/data/ch04"
chmod +x top-words
export PATH="${PATH}:/home/dst/tools"
echo $PATH
curl -# "https://www.gutenberg.org/files/11/11-0.txt" | top-words 10
```

4.3. Creando Herramientas de Línea de Comandos con Python y R

La herramienta de línea de comandos que se creo en la sección anterior fue escrita en Bash. (Claro, no se emplearon todas las características del lenguaje de programación Bash, pero el intérprete es `bash`.) Como la línea de comandos es independiente del lenguaje, no necesariamente se tiene que usar Bash para crear herramientas de línea de comandos.

En esta sección se va a demostrar que las herramientas de línea de comandos también se pueden crear en otros lenguajes de programación. Nos centraremos en Python y R porque son los dos lenguajes de programación más populares dentro de la comunidad de ciencia de datos. No se hará una introducción completa a ninguno de los dos lenguajes, por lo que se asume que se tiene algo de familiaridad con Python y/o R. Otros lenguajes de programación como Java, Go y Julia, siguen un patrón similar cuando se trata de crear herramientas de línea de comandos.

Hay tres razones principales para crear herramientas de línea de comandos en otro lenguaje de programación que no sea Bash.

- Primero, es posible que ya tenga algún código que le gustaría poder usar desde la línea de comandos.
- En segundo lugar, la herramienta de línea de comandos terminaría abarcando más de cien líneas de código Bash.
- En tercer lugar, la herramienta de línea de comandos debe ser más segura y robusta (Bash carece de muchas características, como la comprobación de tipos).

Los seis pasos que se mostraron en la sección anterior se aplican aproximadamente igual a la creación de herramientas de línea de comandos en otros lenguajes de programación también. Sin embargo, el primer paso no sería copiar y pegar desde la línea de comandos, sino copiar y pegar el código relevante en un nuevo archivo. Las herramientas de línea de comandos escritas en Python y R deben especificar `python` y `Rscript`, respectivamente, como intérprete después del *shebang*.

Cuando se trata de crear herramientas de línea de comandos usando Python y R, hay dos aspectos más que merecen especial atención.

- Primero, el procesamiento de la entrada estándar (*stdin*), que es natural para los scripts de shell, debe tenerse en cuenta explícitamente en Python y R.
- En segundo lugar, como las herramientas de línea de comandos escritas en Python y R tienden a ser más complejas, es posible que también queramos ofrecer al usuario la posibilidad de especificar argumentos de línea de comandos más elaborados.

4.3.1. Portando El Script De Shell

Como punto de partida, portamos el script de shell recién creado tanto a Python como a R. En otras palabras, £qué código de Python y R nos da las palabras más utilizadas desde la *stdin*?

En Python, el código se vería así:

```
cd /data/ch04
bat top-words.py
```

Note que este ejemplo de Python no usa ningún librería externa. Si se quiere hacer procesamiento de texto avanzado, entonces es recomiendable utilizar el paquete `NLTK`. Si se va a trabajar con una gran cantidad de datos numéricos, entonces es recomandable usar el paquete `Pandas`.

Y en R el código se vería como:

```
bat top-words.R
```

Comprobemos que las tres implementaciones (es decir, Bash, Python y R) devuelven las mismas 5 palabras principales:

```
chmod +x *.py *.R
time < alice.txt ./top-words-5.sh 5
time < alice.txt ./top-words.py 5
time < alice.txt ./top-words.R 5
```

Es impresionante que se pueda lograr la misma tarea con múltiples lenguajes. Veamos las diferencias entre los enfoques.

Primero, lo que es inmediatamente obvio es la diferencia en la cantidad de código. Para esta tarea específica, tanto Python como R requieren mucho más código que Bash. Esto ilustra que, para algunas tareas, es mejor usar la línea de comandos. Para otras tareas, es posible que sea preferible usar un lenguaje de programación.

A medida que se gane más experiencia en la línea de comandos, se comenzará a reconocer cuándo usar qué enfoque. Cuando todo es una herramienta de línea de comandos, incluso se puede dividir la tarea en subtareas y combinar una herramienta de línea de comandos de Bash con, por ejemplo, una herramienta de línea de comandos de Python. Cualquiera que sea el enfoque que mejor funcione para la tarea en cuestión.

4.3.2. Procesando Datos de Transmisión desde la Entrada Estándar (stdin)

En los dos fragmentos de código anteriores, tanto Python como R leyeron la entrada estándar completa a la vez.

En la línea de comandos, la mayoría de las herramientas canalizan los datos a la siguiente herramienta de línea de comandos en forma de transmisión (*streaming*). Hay algunas herramientas de línea de comandos que requieren los datos completos antes de escribir cualquier dato en la salida estándar, como `sort`. Esto significa que la canalización (*pipeline*) está bloqueada por dichas herramientas. Esto no tiene por qué ser un problema cuando los datos de entrada son finitos, como un archivo. Sin embargo, cuando los datos de entrada son un flujo continuo, dichas herramientas de línea de comandos son inútiles.

Afortunadamente, Python y R admiten el procesamiento de datos vía *streaming*. Puede aplicar una función línea por línea, por ejemplo.

Aquí hay dos ejemplos mínimos que demuestran cómo funciona esto en Python y R, respectivamente.

Tanto la herramienta de Python como la de R resuelven el, problema de *Fizz Buzz*, que se define de la siguiente manera:

> Imprime los números del 1 al 100, excepto que si el número es divisible por 3, en su lugar imprime fizz; si el número es divisible por 5, en su lugar imprime buzz; y si el

número es divisible por 15, en su lugar imprime fizzbuzz.

Aquí está el código de Python:

```
bat fizzbuzz.py
```

Y aquí está el código R:

```
bat fizzbuzz.R
```

Probemos ambas herramientas (para ahorrar espacio, canalizamos la salida a `column`):

```
seq 30 | sample -d 100 | ./fizzbuzz.py | column -x
seq 30 | ./fizzbuzz.R | column -x
```

4.4. Para Profundizar

- Añadir documentación de ayuda a una herramienta se vuelve importante cuando la herramienta tiene muchas opciones para recordar, y aún más cuando se quiere compartir la herramienta con otros. `docopt` es un framework independiente del lenguaje para proporcionar ayuda y definir las posibles opciones que la herramienta acepta. Hay implementaciones disponibles en casi cualquier lenguaje de programación incluyendo Bash, Python, y R.
- Si se quieres aprender más sobre programación en Bash, se recomienda *Classic Shell Programming* por Arnold Robbins y Nelson Beebe y *Bash Cookbook* por Carl Albing y JP Vossen.
- Escribir un script de Bash robusto y seguro es bastante complicado. ShellCheck es una herramienta online que revisará el código Bash en busca de errores y vulnerabilidades. También hay una herramienta de línea de comandos disponible.
- El libro *Ten Essays on Fizz Buzz* por Joel Grus es una colección perspicaz y divertida de diez maneras diferentes de resolver Fizz Buzz con Python.
- Si se utiliza *Sublime Text* para escribir código, se puede instalar el paquete *lsp* que permite tener visualización y resaltado de código en linea para cualquier lenguaje.

5. Depuración de datos

Hace dos capítulos, en el primer paso del modelo OSEMN para la ciencia de datos, se vio cómo *obtener* datos desde una variedad de fuentes. Este capítulo trata sobre el segundo paso: *depurar* o *limpiar* datos. Es bastante infrecuente que se pueda continuar inmediatamente con la *exploración* o incluso el *modelado* de los datos. Hay muchas de razones por las que los datos primero necesitan algo de depuración o limpieza.

Para empezar, es posible que los datos no estén en el formato deseado. Por ejemplo, es posible que se hayan obtenido algunos datos JSON desde una API, pero se necesita que estén en formato CSV para crear una visualización. Otros formatos comunes incluyen texto sin formato, HTML y XML. La mayoría de las herramientas de línea de comandos solo funcionan con uno o dos formatos, por lo que es importante que se puedan convertir datos de un formato a otro.

Una vez que los datos están en el formato deseado, aún podría haber problemas como valores faltantes, inconsistencias, caracteres extraños o partes innecesarias. Se pueden solucionar aplicando filtros, reemplazando valores y combinando varios archivos. La línea de comandos es especialmente adecuada para este tipo de transformaciones, porque hay muchas herramientas especializadas disponibles, la mayoría de las cuales pueden manejar grandes cantidades de datos.

En este capítulo, analizaremos herramientas clásicas como `grep` y `awk`, y herramientas más nuevas como `jq` y `pup`.

A veces, se puede usar la misma herramienta de línea de comandos para realizar varias operaciones o varias herramientas para realizar la misma operación.

En este capítulo, se aprenderá cómo:

- Convertir datos de un formato a otro
- Aplicar consultas SQL directamente a CSV
- Filtrar líneas
- Extraer y reemplazar valores
- Dividir, fusionar y extraer columnas
- Combinar múltiples archivos

Este capítulo comienza con los siguientes archivos:

```
cd /data/ch05
ls
```

Cualquier otro archivo se descarga o se genera utilizando herramientas de línea de comandos.

5.1. Transformaciones

En el Capítulo 2 se indicó que, en la práctica, los pasos del modelo OSEMN rara vez se seguirán de forma lineal. En este sentido, aunque la limpieza es el segundo paso del modelo OSEMN, es necesario destacar que no son solo los datos obtenidos los que necesitan limpieza. Las transformaciones que se revisarán en este capítulo pueden ser útiles en cualquier etapa del modelo OSEMN.

Generalmente, si una herramienta de línea de comandos genera una salida que puede ser utilizada inmediatamente por la siguiente herramienta, se pueden encadenar las dos herramientas utilizando el operador *pipe* (`|`). De lo contrario, es necesario aplicar primero una transformación a los datos insertando una herramienta intermedia en la canalización.

Consideremos el siguiente ejemplo, supongamos que hemos obtenido los primeros 100 elementos de una secuencia de *fizzbuzz* (ver Capítulo 4) y nos gustaría visualizar con qué frecuencia aparecen las palabras *fizz*, *buzz* y *fizzbuzz* utilizando un gráfico de barras.

Primero se obtienen los datos generando la secuencia y la almacenamos en el archivo *fb.seq*:

```
seq 100 |
/data/ch04/fizzbuzz.py |
tee fb.seq | trim
```

> La herramienta `fizzbuzz.py` proviene del Capítulo 4.

Luego usamos `grep` para mantener las líneas que coincidan con el patrón *fizz* o *buzz* y cuentas cuántas veces aparece cada palabra usando `sort` y `uniq`:

```
grep -E "fizz|buzz" fb.seq |
sort | uniq -c | sort -nr > fb.cnt
bat -A fb.cnt
```

- `grep -E "fizz|buzz" fb.seq` es una expresión regular que coincide con *fizzbuzz*.
- Usando `sort` y `uniq` de esta manera es una forma común de contar líneas y ordenarlas en orden descendente. Es la opción `-c` la que agrega los recuentos.

Notar que `sort` se usa dos veces: primero porque `uniq` asume que sus datos de entrada están ordenados y segundo para ordenar los recuentos numéricamente. En cierto modo, esta es una transformación intermedia, aunque sutil.

El siguiente paso sería visualizar los recuentos usando `rush`. Sin embargo, dado que `rush` espera que los datos de entrada estén en formato CSV, esto requiere una transformación menos sutil primero. `awk` puede agregar un encabezado, intercambiar los dos campos e insertar comas en un solo comando:

```
< fb.cnt awk 'BEGIN { print "value,count" } { print $2","$1 }' > fb.csv
bat fb.csv
csvlook fb.csv
```

Ahora usamos `rush` para crear un gráfico de barras. Consulta @ref(fig:fb-image) para ver el resultado.

```
rush plot -x value -y count --geom col --height 2 fb.csv > fb.png
```

- `rush` se cubrirá en detalle en el Capítulo 7.
- `display` puede no funcionar en todos los sistemas operativos. Como alternativa se puede utilizar `cacaview` o `img2txt` fuera del contenedor docker (si es que `caca-utils` esta instalado

en su sistema).

Aunque este ejemplo es un poco artificial, revela un patrón que es común cuando se trabaja en la línea de comandos. Las herramientas clave, como las que obtienen datos, crean una visualización o entrenan un modelo, a menudo requieren transformaciones intermedias para poder encadenarse en una *pipeline*. En ese sentido, escribir una *pipeline* es como resolver un rompecabezas, donde las piezas clave a menudo requieren piezas auxiliares para encajar.

5.2. Texto Plano

Hablando formalmente, *texto plano* (*plain text*) se refiere a una secuencia de caracteres legibles por humanos y, opcionalmente, algunos tipos específicos de caracteres de control, como tabulaciones y saltos de línea. Por ejemplo, registros, libros electrónicos, correos electrónicos y código fuente.

El texto plano tiene muchos beneficios sobre los datos binarios, incluyendo:

- Puede abrirse, editarse y guardarse usando cualquier editor de texto.
- Es autodescriptivo e independiente de la aplicación que lo creó.
- Sobrevivirá a otras formas de datos, porque no se requiere conocimiento adicional ni aplicaciones para procesarlo.

Pero lo más importante, la filosofía Unix considera el texto plano como la interfaz universal entre herramientas de línea de comandos. Es decir, la mayoría de las herramientas aceptan texto plano como entrada y producen texto plano como salida. Otros formatos reconocidos, como CSV, JSON, XML y HTML, también son en realidad texto plano.

Por ahora, asumimos que el texto plano no tiene una estructura tabular clara (como lo hace CSV) o una estructura anidada (como lo hacen JSON, XML y HTML).

5.2.1. Filtrando Líneas

La primera operación de limpieza es filtrar líneas. Esto significa que de los datos de entrada, cada línea será evaluada si será mantenida o descartada.

5.2.1.1. Basado en la Ubicación La forma más sencilla de filtrar líneas es en función de su ubicación. Esto puede resultar útil cuando se desea inspeccionar, por ejemplo, las 10 primeras líneas de un archivo, o cuando se extrae una fila específica de la salida de otra herramienta de línea de comandos. Para ilustrar cómo filtrar según la ubicación, vamos a crear un archivo ficticio que contenga 10 líneas:

```
seq -f "Line%g" 10 | tee lines
```

Se puede imprimir las primeras 3 líneas usando `head`, `sed`, o `awk`:

```
< lines head -n 3
< lines sed -n '1,3p'
< lines awk 'NR <= 3'
```

En `awk`, *NR* se refiere al número total de registros de entrada vistos hasta el momento.

Del mismo modo, se puede imprimir las últimas 3 líneas usando `tail`:

```
< lines tail -n 3
```

La eliminación de las 3 primeras líneas es la siguiente:

```
< lines tail -n +4
< lines sed '1,3d'
< lines sed -n '1,3!p'
```

Notar que con `tail` se tiene que especificar el número de líneas más uno. Dado que se comienza desde la que se quiere empezar a imprimir.

Eliminar las últimas 3 líneas se puede hacer con `head`:

```
< lines head -n -3
```

Se puede imprimir líneas específicas usando `sed`, `awk`, o una combinación de `head` y `tail`. A continuación se imprimen las líneas 4, 5 y 6:

```
cat lines | sed -n '4,6p'
cat lines | awk '(NR>=4) && (NR<=6)'
cat lines | head -n 6 | tail -n 3
```

Se puede imprimir líneas impares con `sed` especificando un inicio y un paso, o con `awk` usando el operador módulo:

```
cat lines | sed -n '0~2p'
cat lines | awk '(NR+1)%2'
```

5.2.1.2. Basado en un Patrón A veces se quiere mantener o descartar líneas según su contenido. Con `grep`, la herramienta de línea de comandos canónica para filtrar líneas, se puede imprimir cada línea que coincida con un determinado patrón o expresión regular. Por ejemplo, para extraer todos los títulos de los capítulos de *Las Aventuras de Alicia en el País de las Maravillas*:

```
cat alice.txt | grep chapter
```

La opción `-i` especifica que la coincidencia debe ser insensible a mayúsculas y minúsculas.

También se puede especificar una expresión regular. Por ejemplo, si solo se quisiera imprimir los encabezados que comienzan con *The*:

```
cat alice.txt | grep -E '^CHAPTER (.*)\. The'
```

Notar que se tiene que especificar la opción `-E` para habilitar expresiones regulares. De lo contrario, `grep` interpreta el patrón como una cadena literal que lo más probable es que no arroje ninguna coincidencia:

```
cat alice.txt | grep '^CHAPTER (.*)\. The'
```

Con la opción `-v` se invierten las coincidencias, de modo que `grep` imprime las líneas que *no* coinciden con el patrón. La expresión regular a continuación coincide con las líneas que contienen espacios en blanco, solamente. Entonces, con la inversa, y usando `wc -l`, se puede contar el número de líneas no vacías:

```
cat alice.txt | grep -Ev '^\s$' | wc -l
```

5.2.1.3. Basado en Aleatoriedad

Cuando se está en el proceso de formular una *pipelaine* de datos y se tiene muchos datos, la depuración puede ser engorrosa. En ese caso, generar una muestra más pequeña a partir de los datos puede ser útil. El propósito principal de `sample` es obtener un subconjunto de los datos generando solo un cierto porcentaje de la entrada línea por línea.

```
RANDOM=1337
seq -f "Line%g" 1000 | sample -r 1%
```

Aquí, cada línea de entrada tiene un uno por ciento de posibilidades de imprimirse. Este porcentaje también se puede especificar como una fracción (es decir, `1/100`) o como una probabilidad (es decir, `0.01`).

`sample` tiene otros dos propósitos, que pueden ser útiles cuando se está depurando una *pipeline*. Primero, es posible agregar algo de retraso a la salida. Esto es útil cuando la entrada es un flujo constante o *streaming* (por ejemplo, el flujo de Wikipedia que vimos en el Capítulo 3), y los datos llegan demasiado rápido para ver qué está sucediendo. En segundo lugar, se puede colocar un temporizador en `sample`, para que no se tenga que finalizar el proceso en curso manualmente. Por ejemplo, para agregar un retraso de 1 segundo entre cada línea que se imprime y solo ejecutar durante 5 segundos:

```
seq -f "Line%g" 1000 | sample -r 1% -d 1000 -s 5 | ts
```

La herramienta `ts` agrega una marca de tiempo delante de cada línea.

Para evitar cálculos innecesarios, se intenta colocar `sample` lo antes posible en la *pipeline*. De hecho, este argumento es válido para cualquier herramienta de línea de comandos que reduzca datos, como `head` y `tail`. Una vez que estés seguro de que la *pipeline* funciona, se remueve de ella.

5.2.2. Extrayendo Valores

Para extraer los encabezados de capítulo reales de nuestro ejemplo anterior, se puede tomar un enfoque simple al canalizar la salida de `grep` a `cut`:

```
grep -i chapter alice.txt | cut -d ' ' -f 3-
```

Aquí, cada línea que se pasa a `cut` se divide de espacios en campos, y luego se imprime desde el tercer campo hasta el último campo. El número total de campos puede ser diferente por línea de entrada. Con `sed` puede realizar la misma tarea de una manera mucho más compleja:

```
sed -rn 's/^CHAPTER ([IVXLCDM]{1,})\. (.*)$/\2/p' alice.txt
```

Este enfoque utiliza una expresión regular y una referencia posterior. Aquí, `sed` también se hace cargo del trabajo realizado por `grep`. Solo se recomienda usar un enfoque complejo cuando uno más simple no funcionaría. Por ejemplo, si *chapter* alguna vez fue parte del texto en sí y no solo se usó para indicar el comienzo de un nuevo capítulo.

Por supuesto, hay muchos niveles de complejidad que habrían solucionado esto, pero esto es para ilustrar un enfoque extremadamente estricto. En la práctica, el desafío es crear una *pipeline* que logre un buen equilibrio entre complejidad y flexibilidad.

Cabe notar que, `cut` también puede dividir en posiciones de caracteres. Esto es útil cuando desea extraer (o eliminar) el mismo conjunto de caracteres por línea de entrada:

```
grep -i chapter alice.txt | cut -c 9-
```

`grep` tiene una gran característica que genera cada coincidencia en una línea separada usando la opción `-o`:

```
cat alice.txt | grep -oE '\w{2,}' | trim
```

Pero, £qué pasaría si quisiera crear un conjunto de datos de todas las palabras que comienzan con una *a* y terminan con una *e*?:

```
cat alice.txt | tr '[:upper:]' '[:lower:]' |
grep -oE '\w{2,}' |
grep -E '^a.*e$' |
sort | uniq | sort -n
```

Aquí se utiliza `tr` para poner el texto en minúsculas.

5.2.3. Reemplazando y Eliminando Valores

Se puede usar la herramienta de línea de comandos `tr`, que significa *translate*, para reemplazar o eliminar caracteres individuales. Por ejemplo, los espacios se pueden reemplazar por guiones bajos de la siguiente manera:

```
echo 'hello world!' | tr ' ' '_'
```

Si se necesita reemplazar más de un carácter:

```
echo 'hello world!' | tr ' !' '_?'
```

`tr` también se puede usar para eliminar caracteres individuales especificando el argumento `-d`:

```
echo 'hello world!' | tr -d ' !'
echo 'hello world!' | tr -d -c '[a-z]'
```

En este caso, estos dos comandos logran lo mismo. El segundo comando, sin embargo, usa dos características adicionales:

- Especifica un *rango* de caracteres (todas las letras minúsculas) usando los corchetes y el guión (`[-]`),

- y la opción `-c` indica que se debe usar el complemento de eso.

En otras palabras, este comando solo conserva las letras minúsculas. Incluso se puede usar `tr` para convertir texto a mayúsculas:

```
echo 'hello world!' | tr '[a-z]' '[A-Z]'
echo 'hello world!' | tr '[:lower:]' '[:upper:]'
```

Sin embargo, si se necesitas traducir caracteres que no son ASCII, entonces `tr` puede no funcionar porque opera solo en caracteres de un solo byte. En esos casos, se debe usar `sed`:

```
echo 'hello world!' | tr '[a-z]' '[A-Z]'
echo 'hallo wêreld!' | tr '[a-z]' '[A-Z]'
echo 'hallo wêreld!' | tr '[:lower:]' '[:upper:]'
echo 'hallo wêreld!' | sed 's/[[:lower:]]*/\U&/g'
echo 'helló világ' | tr '[:lower:]' '[:upper:]'
echo 'helló világ' | sed 's/[[:lower:]]*/\U&/g'
```

Si se necesita operar en más que caracteres individuales, entonces puedes ser mas útil `sed`. Ya vimos un ejemplo de `sed` con la extracción de los títulos de los capítulos de *alice.txt*. Extraer, eliminar y reemplazar es en realidad la misma operación en `sed`. Simplemente se especifican diferentes expresiones regulares. Por ejemplo, para cambiar una palabra, eliminar espacios repetidos y eliminar espacios iniciales:

```
echo ' hello     world!' |
sed -re 's/hello/bye/' |
sed -re 's/\s+/ /g' |
sed -re 's/\s+//'
```

1. Reemplaza *hello* con *bye*.
2. Reemplaza cualquier espacio en blanco con un espacio. La opción o *flag* `g` significa global, lo que significa que la misma sustitución se puede aplicar más de una vez en la misma línea.
3. Esto solo elimina los espacios iniciales porque no se especificó la opción `g`.

Nuevamente, al igual que con el ejemplo de `grep` anterior, estos tres comandos `sed` se pueden combinar en uno:

```
echo ' hello     world!' |
sed -re 's/hello/bye/;s/\s+/ /g;s/\s+//'
```

Pero, no es simple de leer.

5.3. CSV

5.3.1. Cuerpos, Encabezados y Columnas

Las herramientas de línea de comandos que se han usado para limpiar texto plano, como `tr` y `grep`, no siempre se pueden aplicar a archivos CSV. La razón es que estas herramientas de línea

de comandos no tienen noción de encabezados, cuerpos y columnas. £Cómo filtrar líneas usando `grep` pero siempre incluir el encabezado en la salida? £O cómo convertir a mayúsculas los valores de una columna específica usando `tr` y dejar las otras columnas intactas?

Hay soluciones alternativas de varias etapas para esto, pero son muy engorrosas. Para aprovechar las herramientas ordinarias de línea de comandos para CSV, presentamos tres herramientas, llamadas: `body`, `header` y `cols`.

Con `body` se puede aplicar cualquier herramienta de línea de comandos al cuerpo de un archivo CSV, es decir, todo excepto el encabezado. Por ejemplo:

```
echo -e "value\n7\n2\n5\n3" | body sort -n
```

Asume que el encabezado del archivo CSV solo abarca una fila. Funciona así:

- Toma una línea de la entrada estándar y la almacena como una variable llamada *$header*.
- Imprime el encabezado.
- Ejecuta todos los argumentos de línea de comandos pasados a `body` en los datos restantes en la entrada estándar.

Otro ejemplo, contar las líneas del siguiente archivo CSV:

```
seq 5 | header -a count
```

Con `wc -l`, se puede contar el número de todas las líneas:

```
seq 5 | header -a count | wc -l
```

Si solo se quiere considerar las líneas del cuerpo (es decir, todo excepto el encabezado), agregamos `body`:

```
seq 5 | header -a count | body wc -l
```

Notar que el encabezado no se utiliza y se vuelve a imprimir en la salida.

Por otro lado, `header`, permite manipular el encabezado de un archivo CSV. Si no se proporcionan argumentos, se imprime el encabezado del archivo CSV:

```
csvlook tips.csv | trim
cat tips.csv | header
```

Notar que es equivalente a `head -n 1`. Si el encabezado abarca más de una fila, lo cual no se recomienda, se puede especificar `-n 2`. También se puede agregar un encabezado a un archivo CSV:

```
seq 5 | header -a count
```

Esto es equivalente a `echo çount" | cat - <(seq 5)`.

La eliminación de un encabezado se realiza con la opción `-d`:

```
cat iris.csv | header -d | trim
```

Esto es similar a `tail -n +2`, pero es un poco más fácil de recordar. Reemplazar un encabezado, que básicamente es primero eliminar un encabezado y luego agregar uno nuevo, se logra especificando la opción `-r`:

```
seq 5 | header -a line | body wc -l | header -r count
```

Finalmente. se puede aplicar un comando solo al encabezado, similar a lo que la herramienta de línea de comandos `body` le hace al cuerpo. Por ejemplo:

```
seq 5 | header -a line | header -e "tr '[a-z]' '[A-Z]'"
```

Por su parte, `cols`, permite aplicar un determinado comando solo a un subconjunto de las columnas. Por ejemplo, si se quiere convertir a mayúsculas los valores en la columna de día en el conjunto de datos *tips* (sin afectar las otras columnas y el encabezado), se usaría `cols` en combinación con `body`, de la siguiente manera:

```
cat tips.csv | cols -c day body "tr '[a-z]' '[A-Z]'" | head -n 5 | csvlook
```

En síntesis, si bien generalmente es preferible utilizar herramientas de línea de comandos que están hechas específicamente para datos CSV, `body`, `header` y `cols` también permiten aplicar las herramientas clásicas de línea de comandos a archivos CSV si es necesario.

5.3.2. Realizando Consultas SQL en CSV

En caso de que las herramientas de línea de comandos mencionadas en este capítulo no proporcionen suficiente flexibilidad, entonces existe otro enfoque para depurar los datos desde la línea de comandos. La herramienta `csvsql` permite ejecutar consultas SQL directamente en archivos CSV. SQL es un lenguaje poderoso para definir operaciones para depurar datos; es una forma muy diferente a la de usar herramientas de línea de comandos individuales.

> Si los datos provienen originalmente de una base de datos relacional, entonces, si es posible, se intenta ejecutar consultas SQL en esa base de datos y posteriormente se extraen los datos como CSV. Como se mostró en el Capítulo 3, se puede usar la herramienta de línea de comandos `sql2csv` para esto. Cuando primero se exportan datos de la base de datos a un archivo CSV y luego se aplica SQL, no solo es más lento, sino que también existe la posibilidad de que los tipos de columna no se infieran correctamente de los datos CSV.

En las tareas de depuración a continuación, se incluirán varias soluciones que involucran `csvsql`. Un comando básico es este:

```
seq 5 | header -a val | csvsql --query "SELECT SUM(val) AS sum FROM stdin"
```

Si se pasa la entrada estándar (*stdin*) a `csvsql`, entonces la tabla se denomina *stdin*. Los tipos de columna se infieren automáticamente a partir de los datos. Como se verá más adelante, en la sección de combinación de archivos CSV, también se pueden especificar varios archivos CSV.

Notar que `csvsql` emplea el lenguaje SQLite de SQL, que tiene algunas diferencias sutiles con respecto al estándar SQL. Si bien SQL es generalmente más detallado que las otras soluciones,

también es mucho más flexible. Si ya se sabe cómo abordar un problema de depuración con SQL, se puede aplicar directamente desde la terminal.

5.3.3. Extrayendo y Reordenando Columnas

Las columnas se pueden extraer y reordenar usando la herramienta de línea de comandos: `csvcut`. Por ejemplo, para mantener solo las columnas en el conjunto de datos de Iris que contienen valores numéricos *y* reordenar las dos columnas del medio:

```
cat iris.csv | csvcut -c sepal_length,petal_length,sepal_width,petal_width | csvlook | tri
```

Alternativamente, también se puede especificar las columnas que se desean omitir con la opción `-C`, que significa *complemento*:

```
cat iris.csv | csvcut -C species | csvlook | trim
```

Aquí, las columnas incluidas se mantienen en el mismo orden. En lugar de los nombres de las columnas, también se pueden especificar los índices de las columnas, que comienzan en 1. Esto permite, por ejemplo, seleccionar solo las columnas impares:

```
echo 'a,b,c,d,e,f,g,h,i\n1,2,3,4,5,6,7,8,9' |
csvcut -c $(seq 1 2 9 | paste -sd,)
```

Si se está seguro de que no hay comas en ninguno de los valores, también se puede usar `cut` para extraer columnas. Tener en cuenta que `cut` no reordena las columnas, como se demuestra con el siguiente comando:

```
echo 'a,b,c,d,e,f,g,h,i\n1,2,3,4,5,6,7,8,9' | cut -d, -f 5,1,3
```

Como se puede observar, no importa en qué orden se especifiquen las columnas con la opción `-f`; con `cut` siempre aparecerán en el orden original.

Para completar, veamos también el enfoque SQL para extraer y reordenar las columnas numéricas del conjunto de datos de Iris:

```
cat iris.csv | csvsql --query "SELECT sepal_length, petal_length, "\
"sepal_width, petal_width FROM stdin" | head -n 5 | csvlook
```

5.3.4. Filtrando Filas

La diferencia entre filtrar filas en un archivo CSV en lugar de filtrar líneas en un archivo de texto plano es que es posible que se desee basar este filtrado en valores en una determinada columna, solamente. Filtrar por ubicación es esencialmente lo mismo, pero se debe tener en cuenta que la primera línea de un archivo CSV suele ser el encabezado. Recuerdar que siempre se puede usar `body` si desea mantener el encabezado:

```
seq 5 | sed -n '3,5p'
seq 5 | header -a count | body sed -n '3,5p'
```

Cuando se trata de filtrar en un cierto patrón dentro de una columna determinada, se puede utilizar `csvgrep`, `awk` o `csvsql`. Por ejemplo, para excluir todas las facturas cuyo tamaño de grupo era inferior a 5:

```
csvgrep -c size -i -r "[1-4]" tips.csv | csvlook
```

Tanto `awk` como `csvsql` también pueden hacer comparaciones numéricas. Por ejemplo, para obtener todas las facturas superiores a 40 USD en sábado o domingo:

```
cat tips.csv | awk -F, 'NR==1 || ($1 > 40.0) && ($5 ~ /^S/)' | csvlook
```

La solución `csvsql` es más detallada, pero también es más robusta, ya que utiliza los nombres de las columnas en lugar de sus índices:

```
csvsql --query "SELECT * FROM tips WHERE bill > 40 AND day LIKE 'S%'" tips.csv | csvlook
```

Notar que la flexibilidad de la cláusula *WHERE* en una consulta SQL no se puede igualar fácilmente con otras herramientas de línea de comandos, porque SQL puede operar en fechas y conjuntos, y formar combinaciones complejas de cláusulas.

5.3.5. Combinar Columnas

Combinar columnas es útil cuando los valores de interés están distribuidos en varias columnas. Esto puede suceder con fechas (donde el año, el mes y el día podrían ser columnas separadas) o nombres (donde el nombre y el apellido son columnas separadas). Consideremos la segunda situación.

El CSV de entrada es una lista de compositores. Supongamos que la tarea es combinar el nombre y el apellido en un nombre completo. Se mostrarán cuatro enfoques diferentes para esta tarea: `sed`, `awk`, `cols` + `tr` y `csvsql`.

El CSV de entrada es el siguiente:

```
csvlook -I names.csv
```

El primer enfoque, `sed`, utiliza dos declaraciones. La primera es reemplazar el encabezado y la segunda es una expresión regular con referencias inversas aplicadas a la segunda fila en adelante:

```
cat names.csv |
sed -re '1s/.*/id,full_name,born/g;2,$s/(.*),(.*),(.*),(.*)/\1,\3 \2,\4/g' |
csvlook -I
```

El enfoque `awk` tiene la siguiente forma:

```
cat names.csv |
awk -F, 'BEGIN{OFS=","; print "id,full_name,born"} {if(NR > 1) {print $1,$3" "$2,$4}}' |
csvlook -I
```

El enfoque `cols` en combinación con `tr`:

```
cat names.csv |
cols -c first_name,last_name tr \",\" \" \" \" |
```

```
header -r full_name,id,born |
csvcut -c id,full_name,born |
csvlook -I
```

Notar que `csvsql` emplea SQLite como base de datos para ejecutar la consulta y que `||` representa la concatenación:

```
cat names.csv | csvsql --query "SELECT id, first_name || ' ' || last_name "\
"AS full_name, born FROM stdin" | csvlook -I
```

£Qué pasa si *last_name* contuviera una coma? Consideremos el siguiente CSV:

```
cat names-comma.csv
```

Probamos las alternativas anteriores:

```
cat names-comma.csv |
sed -re '1s/.*/id,full_name,born/g;2,$s/(.*),(.*),(.*),(.*)/\1,\3 \2,\4/g' | tail -n 1
```

```
cat names-comma.csv |
awk -F, 'BEGIN{OFS=","; print "id,full_name,born"} {if(NR > 1) {print $1,$3" "$2,$4}}' |
tail -n 1
```

```
< names-comma.csv | cols -c first_name,last_name tr \",\" \" \" |
header -r full_name,id,born | csvcut -c id,full_name,born | tail -n 1
```

Todos fallan de distintas manera. Para `csvsql`:

```
cat names-comma.csv |
csvsql --query "SELECT id, first_name || ' ' || last_name AS full_name, born FROM stdin" |
tail -n 1
```

se logra el objetivo.

Tambien se puede ejecutar código R directamente en la terminal usando `rush`, que veremos mas adelante:

```
cat names-comma.csv |
rush run -t 'unite(df, full_name, first_name, last_name, sep = " ")' - |
tail -n 1
```

5.3.6. Combinando Múltiples Archivos CSV

5.3.6.1. Concatenar Horizontalmente Supongamos que tenemos tres archivos CSV que se requiere unir uno al lado del otro. Usamos `tee` para guardar el resultado de `csvcut` en la mitad de la *pipeline*:

```
cat tips.csv | csvcut -c bill,tip | tee bills.csv | head -n 3 | csvlook
cat tips.csv | csvcut -c day,time | tee datetime.csv |
head -n 3 | csvlook -I
```

```
cat tips.csv | csvcut -c sex,smoker,size | tee customers.csv |
head -n 3 | csvlook
```

Asumiendo que las filas se alinean, se puede usar `paste` para unirlos:

```
paste -d, {bills,customers,datetime}.csv | head -n 3 | csvlook -I
```

Aquí, el argumento `-d` instruye a `paste` para usar una coma como delimitador.

5.3.6.2. Unión A veces los datos no se pueden combinar mediante concatenación vertical u horizontal. En algunos casos, especialmente en bases de datos relacionales, los datos se distribuyen en múltiples tablas (o archivos) para minimizar la redundancia.

Supongamos quisiéramos extender el conjunto de datos de Iris con más información sobre los tres tipos de flores de Iris, por ejemplo, el identificador USDA. Consideremos el siguiente CSV:

```
csvlook irismeta.csv
```

Lo que este conjunto de datos y el conjunto de datos de Iris tienen en común es la columna *species*. Se puedes usar `csvjoin` para unir los dos conjuntos de datos:

```
csvjoin -c species iris.csv irismeta.csv |
csvcut -c sepal_length,sepal_width,species,usda_id |
sed -n '1p;49,54p' | csvlook
```

Por supuesto, también se puedes usar el enfoque SQL usando `csvsql`, que es un poco más largo (pero potencialmente mucho más flexible):

```
csvsql --query 'SELECT i.sepal_length, i.sepal_width, i.species, m.usda_id
FROM iris i JOIN irismeta m ON (i.species = m.species)' iris.csv irismeta.csv |
sed -n '1p;49,54p' | csvlook
```

5.4. Trabajando con XML/HTML y JSON

En esta sección se mostrarán un par de herramientas de línea de comandos que pueden convertir datos de un formato a otro. Hay dos razones para convertir datos.

Primero, a menudo, los datos deben estar en forma tabular, como una tabla de base de datos o una hoja de cálculo, porque muchos algoritmos de visualización y aprendizaje automático dependen de ello. CSV está inherentemente en forma tabular, pero los datos JSON y HTML/XML pueden tener una estructura profundamente anidada.

Segundo, muchas herramientas de línea de comandos, especialmente las clásicas como `cut` y `grep`, operan en texto plano. Esto se debe a que el texto se considera una interfaz universal entre las herramientas de línea de comandos. Además, los otros formatos son más jóvenes. Cada uno de estos formatos puede tratarse como texto plano, lo que nos permite aplicar dichas herramientas de línea de comandos también a los otros formatos.

A veces se puede aplicar directamente las herramientas clásicas a datos estructurados. Por ejemplo, al tratar los datos JSON a continuación como texto plano, se puede cambiar el atributo *gender* a *sex* usando `sed`:

```
sed -e 's/"gender":/"sex":/g' users.json | jq | trim
```

Como muchas otras herramientas de línea de comandos, `sed` no hace uso de la estructura de los datos. Es mejor usar una herramienta que haga uso de la estructura de los datos, o primero convertir los datos a un formato tabular como CSV y luego aplicar la herramienta de línea de comandos apropiada.

Consideremos el siguiente ejemplo para demostrar la conversión de XML/HTML y JSON a CSV. Para ello, se utilizará: `curl`, `pup`, `xml2json`, `jq` y `json2csv`.

Wikpedia posee una gran cantidad de información. Mucha de esta información está ordenada en tablas, que pueden considerarse conjuntos de datos. Por ejemplo, esta página contiene una lista de países y territorios junto con la longitud de su frontera, su área y la relación entre los dos.

Supongamos que queremos analizar estos datos. En esta sección, se ilustrarán todos los pasos necesarios y sus comandos correspondientes, sin explicarlos en demasiado detalle, ya que se verán posteriormente.

El conjunto de datos que de interés está incrustado en HTML. El objetivo es terminar con una representación de este conjunto de datos con el que se pueda trabajar.

El primer paso es descargar el HTML usando `curl`:

```
url='https://en.wikipedia.org/wiki/List_of_countries_and_dependencies_by_area'
curl -sL $url > wiki.html
```

El HTML se guarda en un archivo llamado *wiki.html*.

```
< wiki.html trim
```

Supongamos que de alguna forma hemos podido determinar que el elemento HTML raíz que nos interesa es un *<table>* con la clase *wikitable*. Esto nos permite mirar la parte que nos interesa usando `grep` (la opción `-A` especifica el número de líneas que se desea imprimir después de la línea coincidente):

```
grep wikitable -A 21 wiki.html
```

El siguiente paso es extraer los elementos necesarios del archivo HTML. Para se puede usar `pup`:

```
< wiki.html pup 'table.wikitable tbody' | tee table.html | trim
```

La expresión pasada a `pup` es un selector CSS. La sintaxis generalmente se usa para diseñar páginas web, pero también se puede utilizar para seleccionar ciertos elementos de HTML. En este caso, se desea seleccionar el *tbody* de la *table* que tiene la clase *wikitable*.

Lo siguiente es `xml2json`, que convierte XML (y HTML) a JSON.

```
< table.html xml2json > table.json
jq . table.json | trim 20
```

La razón de convertir el HTML a JSON es porque hay una herramienta muy poderosa llamada `jq` que opera con datos JSON. El siguiente comando extrae ciertas partes de los datos JSON y les da una nueva forma en un formato con el que podemos trabajar:

```
cat table.json | jq -r '.tbody.tr[1:][] | [.td[]["$t"]] | @csv' |
header -a rank,country,border,surface,ratio > countries.csv
csvlook countries.csv | trim
```

Los datos ahora están en una forma con la que se puede trabajar. Esos fueron bastantes pasos para pasar de una página de Wikipedia a un conjunto de datos CSV. Sin embargo, cuando se combinan todos los comandos anteriores en uno, se puede observar que en realidad es realmente conciso y expresivo.

```
csvlook --max-column-width 28 countries.csv
```

Eso concluye la demostración de la conversión de XML/HTML a JSON a CSV. Mientras que `jq` puede realizar muchas más operaciones, y aunque existen herramientas especializadas para trabajar con datos XML, se recomienda convertir los datos a formato CSV lo más rápido posible tiende. De esta manera, se puede dedicar más tiempo a dominar las herramientas genéricas de línea de comandos, en lugar de herramientas muy específicas.

5.5. Para Profundizar en el Tema

- `awk` es una herramienta y un lenguaje de programación muy potente. Se recomienda profundizar mas en él. Dos buenos recursos son el libro *sed & awk* de Doherty y Robbins y la Guía de Usuario de GNU Awk en línea.
- En este capítulo se han utilizado expresiones regulares en un par de ocasiones. Un tutorial sobre ellas está, lamentablemente, fuera del alcance de este curso. Dado que las expresiones regulares se pueden utilizar en muchas herramientas diferentes, se recomienda aprender mas sobre ellas también. Un buen libro es *Regular Expressions Cookbook* de Jan Goyvaerts y Steven Levithan.

6. Exploración de Datos

Ahora que se han obtenido y limpiado los datos, podemos continuar con el tercer paso del modelo OSEMN, que es explorarlos.

Explorar es el paso donde se familiariza con los datos. Estar familiarizado con los datos es esencial cuando se quiere extraer algún valor de ellos. Por ejemplo, saber qué tipo de características tienen los datos significa saber cuáles vale la pena explorar más a fondo y cuáles se pueden utilizar para responder a cualquier pregunta que se tenga.

Explorar los datos se puede hacer desde tres perspectivas. La primera perspectiva es inspeccionar los datos y sus propiedades. Aquí, Se quiere averiguar cosas como el aspecto de los datos brutos, cuántos puntos de datos tiene el conjunto de datos y qué características tiene el conjunto de datos.

La segunda es calcular estadísticas descriptivas. Esta perspectiva es útil para aprender más sobre las características individuales. La salida suele ser breve y textual y, por lo tanto, puede imprimirse en la línea de comandos.

La tercera perspectiva es crear visualizaciones de los datos. Desde esta perspectiva se puede obtener información sobre cómo interactúan las distintas variables. Se presentará una forma de crear visualizaciones que se puedan imprimir en la línea de comandos. Sin embargo, las visualizaciones son más adecuadas para ser mostradas en una interfaz gráfica de usuario. Una ventaja de las visualizaciones de datos sobre las estadísticas descriptivas es que son más flexibles y que pueden transmitir mucha más información.

En este capítulo, aprenderá a:

- Inspeccionar los datos y sus propiedades
- Calcular estadísticas descriptivas
- Crear visualizaciones de datos dentro y fuera de la línea de comandos

Este capítulo comienza con los siguientes archivos:

```
cd /data/ch07
ls
```

Cualquier otro archivo se descarga o se genera cuando se necesite.

6.1. Inspeccionando Datos y sus Propiedades

En esta sección se demostrará cómo inspeccionar un conjunto de datos y sus propiedades. Debido a que las próximas técnicas de visualización y modelado esperan que los datos tengan una forma estructurada, se asumirá que los datos están en formato CSV. Se pueden utilizar las técnicas descritas en el Capítulo 5 para convertir los datos a CSV si es necesario.

Para simplificar, también se asumirá que los datos tienen un encabezado.

Una vez que sepa que tiene un encabezado, se puede continuar respondiendo las siguientes preguntas:

- £Cuántos puntos de datos y características tiene el conjunto de datos?
- £Cómo se ven los datos sin procesar?
- £Qué tipo de características tiene el conjunto de datos?
- £Se pueden tratar algunas de estas características como categóricas?

6.1.1. Encabezado

Se puede verificar si un archivo tiene un encabezado imprimiendo las primeras líneas usando `head`:

```
head -n 5 venture.csv
```

Si las líneas se ajustan, agregue números de línea usando `nl`:

```
head -n 3 venture.csv | nl
```

Alternativamente, puede usar `trim`:

```
< venture.csv trim 5
```

En este caso, está claro que la primera línea es un encabezado porque contiene solo nombres en mayúsculas y las líneas siguientes contienen números. Este es de hecho un proceso bastante subjetivo y depende se debe decidir si la primera línea es un encabezado o ya es el primer punto de datos.

Cuando el conjunto de datos no contiene un encabezado, se debe utilizar la herramienta `header` (que se describe en el Capítulo 5) para corregirlo.

6.1.2. Inspeccionando los datos

Si se quiere inspeccionar los datos sin procesar, entonces probablemente no sea una buena idea usar `cat`, porque entonces todos los datos se imprimirán de una sola vez. Se recomiendo usar `less`, que permite inspeccionar interactivamente tus datos en la línea de comandos. Se pueden evitar que las líneas largas (como con *venture.csv*) se ajusten especificando la opción `-S`:

```
less -S venture.csv
```

Los signos de mayor que a la derecha indican que se puede desplazarte horizontalmente. Se puedes desplazar hacia arriba y hacia abajo presionando **`Up`** y **`Down`**. Se presiona **`Space`** para desplazarse hacia abajo una pantalla completa. El desplazamiento horizontal se realiza presionando **`Left`** y **`Right`**. Se presiona **`g`** y **`G`** para ir al principio y al final del archivo, respectivamente. Para salir de `less` se presiona **`q`**. La página del manual de `less` enumera todas las combinaciones de teclas disponibles.

Una ventaja de `less` es que no carga todo el archivo en la memoria, lo que significa que es rápido incluso para ver archivos grandes.

6.1.3. Nombres de atributos y Tipos de Datos

Los nombres de las columnas (características o atributos) pueden indicar el significado de la característica. Se puedes usar la siguiente combinación de `head` y `tr` para esto:

```
cat venture.csv | head -n 1 | tr , '\n'
```

Este comando básico asume que el archivo está delimitado por comas. Un enfoque más robusto es usar `csvcut`:

```
csvcut -n venture.csv
```

Además de los nombres de las columnas, sería muy útil saber qué tipo de valores contiene cada columna, como una cadena de caracteres, un valor numérico o una fecha. Consideremos el siguiente conjunto de datos de juguete:

```
bat -A datatypes.csv
```

Que `csvlook` interpreta de la siguiente manera:

```
csvlook datatypes.csv
```

Ya se ha utilizado `csvsql` en el Capítulo 5 para ejecutar consultas SQL directamente en datos CSV. Cuando no se pasan argumentos de línea de comandos, genera la sentencia SQL necesaria que se necesitaría si se fuera a insertar estos datos en una base de datos real. También se puede usar la salida para inspeccionar cuáles son los tipos de columna inferidos. Si una columna tiene la cadena *NOT NULL* impresa después del tipo de datos, entonces esa columna no contiene valores faltantes.

```
csvsql datatypes.csv
```

Esta salida es especialmente útil cuando se utilizan otras herramientas dentro del conjunto `csvkit`, como `csvgrep`, `csvsort` y `csvsql`. Para *venture.csv*, las columnas se infieren de la siguiente manera:

```
csvsql venture.csv
```

6.1.4. Identificadores Únicos, Variables Continuas y Factores

Conocer el tipo de datos de cada característica no es suficiente. También es esencial saber qué representa cada característica. Tener conocimiento sobre el dominio es muy útil aquí, pero también podemos obtener algo de contexto mirando los datos en sí.

Tanto una cadena como un entero podrían ser un identificador único o podrían representar una categoría. En el último caso, esto podría usarse para asignar un color a la visualización. Pero si un entero denota, por ejemplo, un código postal, entonces no tiene sentido calcular el promedio.

Para determinar si una característica debe tratarse como un identificador único o una variable categórica, se puede contar el número de valores únicos para una columna específica:

```
wc -l tips.csv
cat tips.csv | csvcut -c day | header -d | sort | uniq | wc -l
```

Se puede usar `csvstat`, que es parte de `csvkit`, para obtener el número de valores únicos para cada columna:

```
csvstat tips.csv --unique
csvstat venture.csv --unique
```

Si solo hay un valor único (como con *OBS_STATUS*), entonces existe la posibilidad de que pueda descartar esa columna porque no proporciona ninguna información. Si se desea descartar automáticamente todas esas columnas, entonces se podría usar lo siguiente:

```
cat venture.csv |
csvcut -C $(
  csvstat venture.csv --unique |
  grep ': 1$' |
  cut -d. -f 1 |
  tr -d ' ' |
  paste -sd,
) | trim
```

Esta *pipeline* realiza lo siguiente:

1. La opción `-C` deselecciona columnas dadas sus ubicaciones (o nombres), que se proporciona con la sustitución de comandos
2. Obtiene el número de valores únicos para cada columna en *venture.csv*
3. Mantiene solo las columnas que contienen un valor único
4. Extrae la ubicación de la columna
5. Recorta cualquier espacio en blanco
6. Coloca todas las ubicaciones de las columnas en una línea separada por comas
7. Muestra solo las primeras 10 líneas

En términos generales, si el número de valores únicos es bajo en comparación con el número total de filas, entonces esa característica podría tratarse como una categórica (como *GEO* en el caso de *venture.csv*). Si el número es igual al número de filas, podría ser un identificador único, pero también podría ser un valor numérico.

6.2. Calculando Estadísticas Descriptivas

6.2.1. Estadísticas de Columna

La herramienta de línea de comandos `csvstat` da mucha información. Para cada característica (columna), muestra:

- El tipo de datos
- Si tiene valores faltantes (nulos)
- El número de valores únicos
- Varias estadísticas descriptivas (mínimo, máximo, suma, media, desviación estándar y mediana) para aquellas características para las que es apropiado

Usamos `csvstat` de la siguiente manera:

```
csvstat venture.csv | trim 32
```

Es posible canalizar esto a través de `less` también. Si solo se está interesado en una estadística específica, también se puede usar una de las siguientes opciones:

- `--max` (máximo)
- `--min` (mínimo)
- `--sum` (suma)
- `--mean` (media)
- `--median` (mediana)
- `--stdev` (desviación estándar)
- `--nulls` (si la columna contiene nulos)
- `--unique` (valores únicos)
- `--freq` (valores frecuentes)
- `--len` (longitud máxima del valor)

Por ejemplo:

```
csvstat venture.csv --freq | trim
```

Se puede seleccionar un subconjunto de características con la opción `-c`, que acepta tanto enteros como nombres de columna:

```
csvstat venture.csv -c 3,GEO
```

> Consider que `csvstat`, al igual que `csvsql`, emplea heurísticas para determinar el tipo de datos y, por lo tanto, es posible que no siempre lo haga correctamente. Se recomienda por lo tanto siempre hacer una inspección manual como se describe en la subsección anterior.

Como un buen extra, `csvstat` genera, al final, el número de puntos de datos (filas). Los saltos de línea y las comas dentro de los valores se manejan correctamente. Para ver solo la última línea, se puede usar `tail`. Alternativamente, se puede usar `xsv`, que solo devuelve el número real de filas.

```
csvstat venture.csv | tail -n 1
xsv count venture.csv
```

> Considerar que estas dos opciones son diferentes de usar `wc -l`, que cuenta el número de nuevas líneas (y, por lo tanto, también cuenta el encabezado).

6.2.2. R One-Liners en la terminal

En esta sección se presentará una herramienta de línea de comandos llamada `rush`, que permite aprovechar el entorno de programación estadística `R` directamente desde la línea de comandos.

`R` es un paquete de software estadístico muy potente para hacer ciencia de datos. Es un lenguaje de programación interpretado, tiene una extensa colección de paquetes y ofrece su propio REPL, que

permite, de forma similar a la línea de comandos, trabajar con los datos. Notar que una vez que iniciado R, se está en una sesión interactiva que está separada de la línea de comandos de Unix.

Consideremos el archivo CSV llamado *tips.csv*, y nos gustaría calcular el porcentaje de propina y guardar el resultado. Para lograr esto en `R` primero ejecutaríamos, `R`:

```
R --quiet
```

> La opción `--quiet` suprime el mensaje de inicio.

Y luego ejecutamos el siguiente código:

```
library(tidyverse)                              #<1>
df <- read_csv("tips.csv")                      #<2>
df <- mutate(df, percent = tip / bill * 100)    #<3>
write_csv(df, "percent.csv")                    #<4>
q("no")                                         #<5>
```

1. Carga los paquetes necesarios.
2. Lee el archivo CSV y se asigna a una variable.
3. Calcula una nueva columna *percent*.
4. Guarda el resultado en el disco.
5. Salir de `R`.

Después, se puedes continuar con el archivo guardado *percent.csv* en la terminal:

```
cat percent.csv | trim 5
```

Notar que solo la tercera línea está asociada con lo que se quiere lograr específicamente. Las otras líneas son código repetitivo necesario. Escribir este código repetitivo para lograr algo simple es engorroso e interrumpe el flujo de trabajo. A veces, solo se quiere hacer una o dos cosas a la vez con los datos. Para aprovechar el poder de R directamente en la terminal existe `rush`. Realicemos la misma tarea que antes, pero ahora usando `rush`:

```
rm percent.csv
rush run -t 'mutate(df, percent = tip / bill * 100)' tips.csv > percent.csv
cat percent.csv | trim 5
```

Estos pequeños one-liners son posibles porque `rush` se encarga de todo el código repetitivo. En este caso, se está usando el subcomando `run`. También está el subcomando `plot`, que se usará en la siguiente sección para producir visualizaciones de datos rápidamente. Si se está pasando algún dato de entrada, entonces, de forma predeterminada, `rush` asume que está en formato CSV con un encabezado y una coma como delimitador. Además, los nombres de las columnas se sanean para que sean más fáciles de trabajar. Se pueden anular estos valores predeterminados usando las opciones `--no-header` (o `-H`), `--delimiter` (o `-d`) y `--no-clean-names` (o `-C`), respectivamente. La ayuda ofrece una buena descripción general de las opciones disponibles para el subcomando `run`:

```
rush run --help
```

Por detrás, `rush` genera un script `R` y posteriormente lo ejecuta. Se puedes ver este script generado especificando la opción `--dry-run` (o `-n`):

```
rush run -n --tidyverse 'mutate(df, percent = tip / bill * 100)' tips.csv
```

Este script generado:

- Escribe el shebang (`#!`; ver Capítulo 4) necesario para ejecutar un script `R` desde la línea de comandos.
- Importa los paquetes *tidyverse* y *glue*.
- Carga *tips.csv* como un marco de datos, limpia los nombres de las columnas y lo asigna a una variable *df*.
- Ejecuta la expresión especificada.
- Imprime el resultado en la salida estándar.

Se podrías redirigir este script generado a un archivo y convertirlo fácilmente en una nueva herramienta de línea de comandos debido al *shebang*.

La salida de `rush` no tiene que estar necesariamente en formato CSV. A continuación, se calcula el porcentaje de propina promedio, el tamaño máximo del grupo, los valores únicos de la columna de tiempo, la correlación entre la factura y la propina. Finalmente, se extrae una columna completa:

```
cat percent.csv | rush run 'mean(df$percent)' -
cat percent.csv | rush run 'max(df$size)' -
cat percent.csv | rush run 'unique(df$time)' -
cat percent.csv | rush run 'cor(df$bill, df$tip)' -
cat percent.csv | rush run 'df$tip' - | trim
```

Ese último guión significa que `rush` debería leer desde la entrada estándar o *stdin*.

Todo el conocimiento que ya tienes sobre `R` ahora se puede utilizar desde la línea de comandos. Con `rush`, incluso se puede crear visualizaciones sofisticadas, como se mostrará en la siguiente sección.

6.3. Creando Visualizaciones

En esta sección, se mostrará cómo crear visualizaciones de datos en la línea de comandos. Usando `rush plot` se podrán creando gráficos de barras, diagramas de dispersión y diagramas de caja.

6.3.1. Mostrando Imágenes desde la Línea de Comandos

Mostrar imágenes en la línea de comandos complejo. Creamos la siguiente imagen a partir del dataset *tips.csv*:

```
rush plot --x bill --y tip --color size --facets '~day' tips.csv > tips.png
l
```

Dependiendo de la configuración, hay diferentes opciones disponibles para mostrar imágenes. Hay por lo menos cuatro opciones, cada una con sus propias ventajas y desventajas:

1. como una representación textual,
2. como una imagen en línea,
3. usando un visor de imágenes, y
4. usando un navegador.

Vamos a repasarlas rápidamente.

La opción 1 es mostrar la imagen dentro de la terminal. Esta salida es generada por `rush` cuando la salida estándar no se redirige a un archivo. Está basada en caracteres ASCII y secuencias de escape ANSI, por lo que está disponible en cualquier terminal.

```
rush plot --x bill --y tip --color size --facets '~day' tips.csv
```

Si solo se observan caracteres ASCII, significa que el medio en el que se está ejecutando el comando no soporta las secuencias de escape ANSI responsables de los colores.

La opción 2 muestra imágenes dentro de la terminal, pero solo en la llamada *iTerm2*, que solo está disponible para macOS y utiliza el Protocolo de Imágenes en Línea a través de un pequeño script (que he llamado `display`). Este script no está incluido con la imagen de Docker, pero puedes instalarlo fácilmente:

```
curl -s "https://iterm2.com/utilities/imgcat" > display && chmod u+x display
```

Si no se está utilizando iTerm2 en macOS, puede haber otras opciones disponibles para mostrar imágenes en línea.

La opción 3 es abrir manualmente la imagen (*tips.png* en este ejemplo) en un visor de imágenes. Cuando se está trabajando localmente, esta opción siempre funciona. En Ubuntu, se puede ejecutar en la terminal (fuera del contenedor docker):

```
xdg-open tips.png
```

Cuando se está trabajando dentro de un contenedor Docker, solo se puede acceder a la imagen generada desde el sistema operativo cuando se ha mapeado un directorio local usando la opción `-v`. Una ventaja de esta opción es que la mayoría de los visores de imágenes actualizan automáticamente la visualización cuando la imagen ha cambiado, lo que permite iteraciones rápidas a medida que se ajusta la visualización.

La opción 4 es abrir la imagen en un navegador. Cualquier navegador servirá, pero se necesitan otros dos requisitos previos para que esta opción funcione. En primer lugar, se necesita haber hecho accesible un puerto (puerto 8000 en este ejemplo) en el contenedor Docker usando la opción `-p`. En segundo lugar, se necesita iniciar un servidor web. Para ello, el contenedor Docker tiene una pequeña herramienta llamada `servewd`, que sirve el directorio de trabajo actual usando Python:

```
bat $(which servewd)
```

Solo se necesita ejecutar `servewd` una vez desde un directorio (por ejemplo, */data/*) y se ejecutará en segundo plano. Una vez que se hayas graficado, se puedes visitar *localhost:8000* en el navegador

y acceder al contenido de ese directorio y todos sus subdirectorios. El puerto por defecto es 8000, pero se puede cambiarlo especificándolo como un argumento a `servewd`:

```
cd /data
servewd &
```

Solo se debe asegurar de que este puerto sea accesible. Debido a que `servewd` se ejecuta en segundo plano, necesitas detenerlo de la siguiente manera:

```
pkill -f http.server
```

La opción 4 también puede funcionar en una máquina remota.

6.3.2. Graficas rápidas

Cuando se trata de crear visualizaciones de datos, hay una varias opciones. Una de las mas versátiles es `ggplot2` de R. La gramática subyacente de los gráficos se acompaña de una API consistente que le permite crear de forma rápida e iterativa diferentes tipos de visualizaciones de datos sin tener que consultar la documentación.

La visualización de datos de la figura anterior podría haberse creado de la siguiente manera:

```
rush run --library ggplot2 'ggplot(df, aes(x = bill, y = tip, color = size)) + geom_point()
```

Sin embargo, es bastante diferente al que utilizamos anteriormente:

```
rush plot --x bill --y tip --color size --facets '~day' tips.csv > tips.png
```

Si bien la sintaxis de `ggplot2` es relativamente concisa, especialmente considerando la flexibilidad que ofrece, hay un atajo para crear gráficos básicos rápidamente. Este atajo está disponible a través del subcomando `plot` de `rush`. Esto le permite crear gráficos básicos sin necesidad de aprender R y la gramática de los gráficos.

Por debajo, `rush plot` usa la función `qplot` del paquete `ggplot2`. Aquí está la primera parte de su documentación:

```
R -q -e '?ggplot2::qplot' | trim 14
```

Como se mostró anteriormente, `rush plot` puede crear tanto visualizaciones gráficas (que constan de píxeles) como visualizaciones textuales (que constan de caracteres ASCII y secuencias de escape ANSI) con la misma sintaxis. Cuando `rush` detecta que su salida se canaliza a otro comando (como `display` o se redirige a un archivo como *tips.png*, producirá una visualización gráfica; de lo contrario, producirá una visualización textual.

Las opciones de trazado y guardado de `rush plot` son:

```
rush plot --help
```

Las opciones más importantes son las opciones de trazado que toman un *<nombre>* como argumento. Por ejemplo, la opción `--x` permite especificar qué columna se debe usar para determinar dónde se deben colocar las *cosas* a lo largo del eje x. Lo mismo ocurre con la opción `--y`. Las

opciones `--color` y `--fill` se utilizan para especificar qué columna desea utilizar para colorear. Probablemente puedas adivinar de qué se tratan las opciones `--size` y `--alpha`. Otras opciones comunes se explican a lo largo de las secciones a medida que se crean varias visualizaciones.

6.3.3. Creando Gráficos de Barras

Los gráficos de barras son especialmente útiles para mostrar el recuento de valores de una característica categórica. Aquí hay una visualización textual de la característica *time* en el conjunto de datos tips:

```
rush plot --x time tips.csv
```

Para obtener la visualización gráfica, ejecutamos:

```
rush plot --x time tips.csv > plot-bar.png
```

La conclusión que podemos sacar de este gráfico de barras es sencilla: hay más del doble de puntos de datos para la cena que para el almuerzo.

6.3.4. Creando Histogramas

Los recuentos de una variable continua se pueden visualizar con un histograma. A continuación, se ha utilizado el atributo *time* para establecer el color de relleno. Como resultado, `rush plot` crea convenientemente un histograma apilado.

```
rush plot --x tip --fill time tips.csv
```

> Existen dos atajos de sintaxis que pueden resultar útiles. Los dos signos de exclamación (`!!`) se reemplazan con el comando anterior. El signo de exclamación y el signo de dólar (`!$`) se reemplazan por la última parte del comando anterior.

Con estos atajos, podemos redirigir la salida anterior a un archivo mediante:

```
!! > plot-histogram.png
```

Este histograma revela que la mayoría de las propinas rondan los 2.5 USD. Debido a que los dos grupos, cena y almuerzo, están apilados uno encima del otro y muestran recuentos absolutos, es difícil compararlos. Quizás un gráfico de densidad pueda ayudar con esto.

6.3.5. Creando Gráficos de Densidad

Un gráfico de densidad es útil para visualizar la distribución de una variable continua. `rush plot` utiliza heurísticas para determinar la geometría adecuada, pero puede anular esto con la opción `geom`:

```
rush plot --x tip --fill time --geom density tips.csv
```

En este caso, la representación textual realmente muestra sus limitaciones en comparación con la representación visual:

```
rush plot --x tip --fill time --geom density tips.csv > plot-density.png
```

6.3.6. Gráficas erróneas

Se han mostrado tres tipos de visualizaciones. En `ggplot2`, estos corresponden a las funciones `geom_bar`, `geom_histogram` y `geom_density`. *geom* es la abreviatura de geometría y dicta lo que realmente se está trazando. Esta hoja de trucos para `ggplot2` proporciona una buena descripción general de los tipos de geometría disponibles. Los tipos de geometría que puede utilizar dependen de las columnas (y sus tipos) que especifique. No todas las combinaciones tienen sentido. Tomemos como ejemplo este gráfico de líneas:

```
rush plot --x tip --y bill --color size --size day --geom path tips.csv
```

En la representación visual:

```
rush plot --x tip --y bill --color size --size day --geom path tips.csv > plot-accident.png
```

Las filas en *tips.csv* son observaciones independientes, mientras que dibujar una línea entre los puntos de datos asume que están conectados. Es mejor visualizar la relación entre la *tip* y la *bill* con un diagrama de dispersión.

6.3.7. Creando Diagramas de Dispersión

Un diagrama de dispersión, donde la geometría es un punto, resulta ser el valor predeterminado al especificar dos características continuas:

```
rush plot --x bill --y tip --color time tips.csv
```

Notar que el color de cada punto se especifica con la opción `--color` (y no con la opción `--fill`). La representación visual:

```
rush plot --x bill --y tip --color time tips.csv > plot-scatter.png
```

De este diagrama de dispersión podemos concluir que existe una relación entre el importe de la factura y la propina. Quizás sea útil examinar estos datos desde un nivel superior creando líneas de tendencia.

6.3.8. Creando Líneas de Tendencia

Si se anula la geometría predeterminada con ***`smooth`***, se pueden visualiza líneas de tendencia. Estos son útiles para ver el panorama general:

```
rush plot --x bill --y tip --color time --geom smooth tips.csv
```

`rush plot` no puede manejar la transparencia, por lo que una representación visual es mucho mejor en este caso:

```
rush plot --x bill --y tip --color time --geom smooth tips.csv > plot-trend.png
```

Si se desea visualizar los puntos originales junto con las líneas de tendencia, se debe recurrir a escribir código `ggplot2` con `rush run`:

```
rush run --library ggplot2 'ggplot(df, aes(x = bill, y = tip, color = time)) + geom_point()
rush run --library ggplot2 'ggplot(df, aes(x = bill, y = tip, color = time)) + geom_point()
```

6.3.9. Creando Box-Plot

Un BOx-Plotvisualiza, para una o más características, un resumen de cinco números: el mínimo, el máximo, la mediana de la muestra y el primer y tercer cuartil. En este caso, necesitamos convertir la característica *size* a una categórica usando la función `factor()`, de lo contrario, todos los valores de la característica *bill* se agruparán.

```
rush plot --x 'factor(size)' --y bill --geom boxplot tips.csv
rush plot --x 'factor(size)' --y bill --geom boxplot tips.csv > plot-boxplot.png
```

Como era de esperar, este diagrama de caja muestra que, en promedio, un tamaño de grupo más grande genera una factura más alta.

6.3.10. Añadiendo Etiquetas

Las etiquetas predeterminadas se basan en los nombres de las columnas (o especificación). En la imagen anterior, la etiqueta ***factor(size)*** debería mejorarse. Usando las opciones `--xlab` y `--ylab` puede anular las etiquetas de los ejes x e y. Se puede agregar un título con la opción `--title`.

Aquí hay un diagrama de violín (que es una combinación de un diagrama de caja y un diagrama de densidad) que demuestra esto:

```
rush plot --x 'factor(size)' --y bill --geom violin --title 'Distribución del importe de la

rush plot --x 'factor(size)' --y bill --geom violin --title 'Distribución del importe de la
```

Agregar a una visualización las etiquetas adecuadas y un título es especialmente útil si se desea compartir con otros para que sea más fácil comprender lo que se muestra.

6.3.11. Yendo Más Allá de los Gráficos Básicos

Aunque `rush plot` es adecuado para crear gráficos básicos cuando se está explorando datos, ciertamente tiene sus limitaciones. A veces se necesita más flexibilidad y opciones sofisticadas como múltiples geometrías, transformaciones de coordenadas, entre otros. En ese caso, podría valer la pena aprender más sobre el paquete subyacente del cual `rush plot` extrae sus capacidades, es decir, el paquete `ggplot2` para R. En Python , existe el paquete `plotnine`, que es una reimplementación de `ggplot2` para Python.

6.4. Visualización de imágenes en la terminal

Hay varias herramientas disponibles para visualizar imágenes en la terminal. Una de ellas es `timg` que se puede instalar de la siguiente manera (ejecutar fuera del contenedor docker):

```
cd
mkdir -p software
cd ~/software && git clone https://github.com/hzeller/timg.git && cd timg
mkdir build && cd build
cmake ../ -DWITH_OPENSLIDE_SUPPORT=On && make && sudo make install
```

Para ejecutar lo anterior, es necesario tener instalado los siguientes paquetes (ya instalados en el servidor):

```
sudo apt install -y cmake git g++ pkg-config libgraphicsmagick++-dev libturbojpeg-dev libex
```

Para utilizar `timg`, se puede ejecutar el siguiente comando:

```
timg ruta_a_imagen
```

Por ejemplo:

```
cd ~/DSITCL #o como se haya nombrado el directorio
cd ch07
timg tips.png
```

6.5. Copia de archivos hacia el servidor

Para copiar archivos hacia un servidor remoto via ssh se puede utilizar el comando `scp` (*secure copy*).

La sintaxis básica es:

```
scp archivo_local usuario@servidor:/directorio_destino
```

Por ejemplo, para copiar un archivo llamado *datos.csv* al directorio */home/user* del servidor *mi.servidor.com* usando el usuario *juan*:

```
scp datos.csv juan@mi.servidor.com:/home/juan/
```

También se pueden copiar directorios completos usando la opción `-r`:

```
scp -r directorio_local usuario@servidor:/directorio_destino
```

Es posible especificar un puerto diferente al 22 (por defecto) usando la opción `-P`:

```
scp -P 2222 archivo.txt usuario@servidor:/directorio_destino
```

Para mostrar el progreso de la transferencia, usar la opción `-v`:

```
scp -v archivo.txt usuario@servidor:/directorio_destino
```

Cuando se requiere copiar múltiples archivos, se pueden especificar uno tras otro:

```
scp archivo1.txt archivo2.csv usuario@servidor:/directorio_destino
```

El comando solicitará la contraseña del usuario en el servidor remoto para completar la transferencia.

6.6. Para Profundizar

- Un tutorial adecuado de `ggplot2` está desafortunadamente fuera del alcance de este curso. Si se quieres mejora en la visualización de tus datos, se recomienda profundizar en el uso de `ggplot2`. Los capítulos 3 y 28 del libro *R for Data Science* de Hadley Wickham y Garrett Grolemund son un excelente recurso.
- Estos capítulos, se tradujeron a Python usando Plotnine y Pandas.

7. Paralelización

En los capítulos anteriores, se han mostrado comandos y *pipelines* que se ocupan de una tarea completa a la vez. En la práctica, sin embargo, se puede encontrar con una tarea que requiere que el mismo comando o *pipeline* se ejecute varias veces. Por ejemplo, es posible que se deba:

- Scrapear cientos de páginas web,
- Hacer docenas de llamadas a una API y transformar su salida,
- Entrenar un clasificador para un rango de valores de parámetros,
- Generar diagramas de dispersión para cada par de características en un conjunto de datos.

En cualquiera de los ejemplos anteriores, hay una cierta forma de repetición involucrada. Posiblemente nos encarguemos de esto con un ciclo for o un ciclo while. En la línea de comandos, lo primero que puede hacerse es presionar **`Arriba`** para recuperar el comando anterior, modificarlo si es necesario y presionar **`Enter`** para ejecutar el comando nuevamente. Esto está bien para dos o tres veces, pero evidentemente no es eficiente para repetirlo cientos de veces. Tal enfoque rápidamente se vuelve engorroso, ineficiente y propenso a errores. Sin embargo, también se pueden escribir tales ciclos en la línea de comandos.

A veces, es suficiente repetir un comando rápido uno tras otro (de manera *serial*). Cuando se tienen varios núcleos (y quizás incluso varias máquinas), es eficiente poder hacer uso de ellos, especialmente cuando se enfrenta a una tarea intensiva en datos. Cuando se utilizan varios núcleos o máquinas, el tiempo total de ejecución puede reducirse significativamente. En este capítulo se presentará una herramienta muy poderosa llamada `parallel` que puede encargarse exactamente de esto. Permite aplicar un comando o *pipeline* para un rango de argumentos como números, líneas y archivos. Además, como su nombre lo indica, permite ejecutar sus comandos en *paralelo*.

Esta herramienta se puede combinar con cualquier otra herramienta discutida en este curso, lo cual aumenta la eficiencia en la forma en que se usa la línea de comandos para la ciencia de datos.

Este capítulo comienza con los siguientes archivos:

```
cd /data/ch08
ls
```

Cualquier otro archivo se descarga o se genera utilizando herramientas de línea de comandos.

7.1. Procesamiento en serie

Antes de profundizar en la paralelización, se discutirá brevemente la ejecución de ciclos en serie. Esta funcionalidad siempre está disponible, y la sintaxis se parece mucho a la ejecución de ciclos en otros lenguajes de programación.

De los ejemplos proporcionados en la introducción de este capítulo, podemos destilar tres tipos de elementos para recorrer: números, líneas y archivos. Estos tres tipos de elementos se discutirán en las siguientes tres subsecciones, respectivamente.

7.1.1. Iterando sobre números

Supongamos que se necesita calcular el cuadrado de cada entero par entre 0 y 100. Hay una herramienta llamada `bc`, que es una *calculadora básica* donde puede enviar una ecuación. El comando para calcular el cuadrado de 4 se ve así:

```
echo "4^2" | bc
echo "16^2 + 7" | bc
```

Para un cálculo único, esto servirá. Sin embargo, como se mencionó en la introducción, no es muy eficiente presionar **Up**, cambiar el número y presionar **Enter** 50 veces. En este caso, es mejor dejar que el shell haga el trabajo con un ciclo for:

```
for i in {0..100..2}
do
echo "$i^2" | bc
done | trim
```

- El shell *zsh* tiene una característica llamada expansión de llaves, que transforma *{0..100..2}* en una lista separada por espacios: *0 2 4 ... 98 100*. A la variable *i* se le asigna el valor 0 en la primera iteración, 1 en la segunda iteración, y así sucesivamente.
- El valor de esta variable se puede usar anteponiéndole un signo oeso (*$*). El shell reemplazará *$i* con su valor antes de que se ejecute `echo`. Notar que puede haber más de un comando entre *do* y *done*.

7.1.2. Iterando Sobre Líneas

El segundo tipo de elementos sobre los que se puede iterar son líneas. Estas líneas pueden provenir de un archivo o de la entrada estándar (*stdin*). Este es un enfoque muy genérico porque las líneas pueden contener cualquier cosa, incluyendo: números, fechas y direcciones de correo electrónico.

Supongamos que se requiere enviar un correo electrónico a todos sus contactos. Primero generamos algunos usuarios falsos usando la API gratuita de Random User Generator:

```
curl -s "https://randomuser.me/api/1.2/?results=5&seed=dsatcl2e" > users.json
cat users.json | jq -r '.results[].email' > emails
bat emails
```

Se puedes iterar sobre las líneas de *emails* con un ciclo while:

```
while read line
do
echo "Enviando invitación a ${line}."
done < emails
```

- En este caso se necesitas usar un ciclo while porque la shell no sabe de antemano cuántas líneas componen la entrada.

- Aunque las llaves alrededor de la variable *line* no son necesarias en este caso (ya que los nombres de las variables no pueden contener puntos), sigue siendo una buena práctica.
- La redirección (`< emails`) también se puede colocar antes de `while`.

También se puedes proporcionar entrada a un ciclo while de forma interactiva especificando el archivo especial de entrada estándar */dev/stdin*. Presiona **Ctrl-D** cuando al terminar:

```
while read line; do echo "Escribiste: ${line}."; done < /dev/stdin
```

Este método, sin embargo, tiene la desventaja de que, una vez que se presiona **Enter**, los comandos entre ***do*** y ***done*** se ejecutan inmediatamente para esa línea de entrada.

7.1.3. Iterando sobre Archivos

Para manejar caracteres especiales, se usa el llamado *globbing* (es decir, expansión de nombre de ruta) en lugar de `ls`:

```
for chapter in /data/*
do
echo "Processing Chapter ${chapter}."
done
```

Al igual que con la expansión de llaves, la expresión ***/data/**** primero es expandida en una lista por *zsh* antes de ser procesada por el ciclo for.

Una alternativa más elaborada a listar archivos es `find`, que:

- Puede buscar recursivamente dentro de los directorios.
- Permite búsquedas elaboradas en propiedades como tamaño, tiempo de acceso y permisos.
- Maneja caracteres especiales como espacios y nuevas líneas.

El siguiente ejemplo lista todos los archivos ubicados en el directorio */data* que tienen *csv* como extensión y son más pequeños que 2 kilobytes:

```
find /data -type f -name '*.csv' -size -2k
```

7.2. Procesamiento Paralelo

Supongamos que se tiene una herramienta de ejecución muy extensa, como esta:

```
bat slow.sh
```

- `ts` agrega una marca de tiempo.
- La variable ***RANDOM*** llama a una función interna de Bash que devuelve un entero pseudoaleatorio entre 0 y 32767. Tomar el resto de la división de ese entero por 5 y sumar 1 asegura que *duración* esté entre 1 y 5.
- `sleep` pausa la ejecución durante un número determinado de segundos.

Este proceso probablemente no ocupe todos los recursos disponibles, pero puede que se necesite ejecutar este comando muchas veces. Por ejemplo, si se necesita descargar una secuencia completa de archivos.

Una forma ingenua de paralelizar es ejecutar los comandos en segundo plano. Ejecutamos `slow.sh` tres veces:

```
for i in {A..C}; do
./slow.sh $i &
done
```

- El ampersand (`&`) envía el comando a segundo plano, lo que permite que el ciclo for continúe inmediatamente con la siguiente iteración.

  Notar que que no todo se puede paralelizar. Las llamadas a la API pueden estar limitadas a un cierto número, o algunos comandos solo pueden tener una instancia.

La Figura 13 ilustra, a nivel conceptual, la diferencia entre el procesamiento en serie, el procesamiento paralelo ingenuo y el procesamiento paralelo con GNU Parallel en términos del número de procesos concurrentes y la cantidad total de tiempo que se tarda en ejecutar todo.

Hay dos problemas con este enfoque ingenuo. Primero, no hay forma de controlar cuántos procesos se están ejecutando simultáneamente. Si se inician demasiados trabajos a la vez, podrían estar compitiendo por los mismos recursos, como CPU, memoria, acceso a disco y ancho de banda de red. Esto podría llevar a un tiempo más largo para ejecutar todo. En segundo lugar, es difícil saber qué salida pertenece a qué entrada.

7.2.1. Introduciendo GNU Parallel

`parallel` es una herramienta de línea de comandos que permite paralelizar y distribuir comandos y *pipelines*. La interesante de esta herramienta es que las herramientas existentes se pueden utilizar tal como están; no necesitan ser modificadas.

> Notar que hay dos herramientas de línea de comandos con el nombre `parallel`. Si se está utilizando la imagen de Docker, entonces está la correcta instalada. De lo contrario, se puede comprobar ejecutando `parallel --version`. Debería decir GNU parallel.

A continuación un ejemplo de lo fácil que es reemplazar el ciclo for de antes:

```
seq 0 2 100 | parallel "echo {}^2 | bc" | trim
```

Este es `parallel` en su forma más simple: los elementos para iterar se pasan a través de la entrada estándar (*stdin*) y no hay ningún argumento aparte del comando que `parallel` necesita ejecutar.

En la Figura 14 se muestra una ilustración de cómo `parallel` distribuye concurrentemente la entrada entre los procesos y recopila sus salidas.

Como se puede observar, básicamente actúa como un ciclo for.

A continuación, otro ejemplo que reemplaza el ciclo for de la sección anterior.

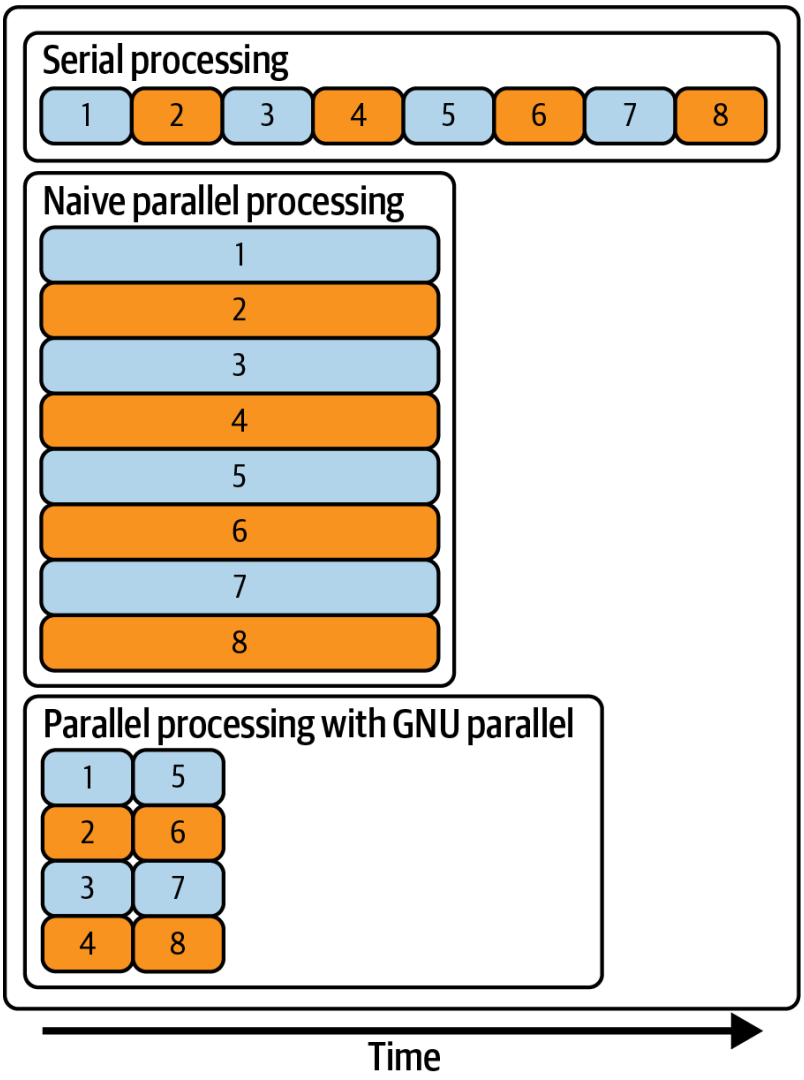


Figura 13: Diagrama de procesamiento en paralelo.

```
parallel --jobs 2 ./slow.sh ::: {A..E}
```

La opción `--jobs`, especifico que `parallel` puede ejecutar como máximo dos trabajos simultáneamente. Los argumentos para `slow.sh` se especifican como un argumento en lugar de a través de la entrada estándar.

> `parallel` tiene muchas opciones para su uso, que pueden verse es su página del manual: `man parallel`.

7.2.2. Especificando la Entrada

El argumento más importante para `parallel`, es el comando o la *pipeline* que se debe ejecutar para cada entrada. La pregunta es: £dónde se debe insertar el elemento de entrada en la línea de

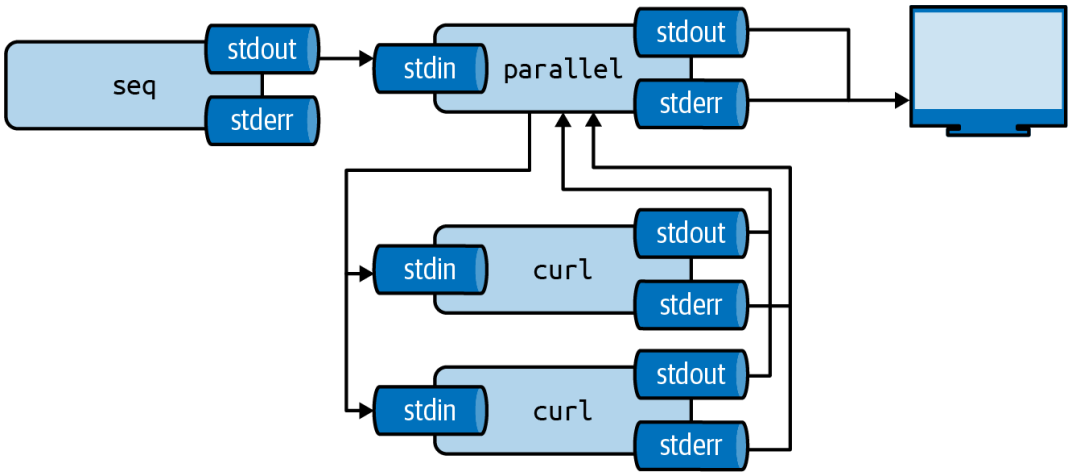


Figura 14: Diagrama de salida en paralelo.

comandos? Si no se especifica nada, entonces el elemento de entrada se añadirá al final:

```
seq 3 | parallel cowsay
parallel --jobs 1 --keep-order cowsay ::: 1 2 3
```

Lo anterior es lo mismo que ejecutar:

```
cowsay 1
cowsay 2
cowsay 3
```

Aunque esto a menudo funciona, es aconsejable ser explícito acerca de dónde se debe insertar el elemento de entrada en el comando utilizando marcadores de posición. En este caso, debido a que se quiere utilizar toda la línea de entrada (un número) a la vez, solo se necesita un marcador de posición. Se especifica el marcador de posición, es decir, dónde poner el elemento de entrada, con un par de llaves (`{}`):

```
seq 3 | parallel cowsay {}
```

> Hay otras maneras de proporcionar entrada a `parallel` que implican una sintaxis poco común. Para mas detalles, leer la página del manual de `parallel`.

Cuando los elementos de entrada son nombres de archivo, hay un par de modificadores que permiten utilizar solo partes del nombre de archivo. Por ejemplo, con `{/}`, solo se utilizará el nombre base del archivo:

```
find /data/ch03 -type f | parallel echo '{#}\) \"{}\" has basename \"{/}\"'
```

> Los caracteres como los paréntesis (`()`) y las comillas (`"`) tienen un significado especial en el shell. Para usarlos literalmente, se utiliza un *backslash* `\` delante de ellos. Esto se le llama *escapar* el carácter.

Si la línea de entrada tiene varias partes separadas por un delimitador, se puede añadir números a los marcadores de posición. Por ejemplo:

```
touch input.csv
cat input.csv | parallel --colsep , "mv {2} {1}" > /dev/null
```

Aquí, se pueden aplicar los mismos modificadores de marcador de posición. También es posible reutilizar el mismo elemento de entrada. Si la entrada a `parallel` es un archivo CSV con una cabecera (*header*), entonces se puede utilizar los nombres de las columnas como marcadores de posición:

```
cat input.csv | parallel -C, --header : "invite {name} {email}"
```

> Para testear si los marcadores de posición están configurados correctamente, se puede utilizar la opción `--dryrun`. En lugar de ejecutar realmente el comando, `parallel` imprimirá todos los comandos exactamente como si se hubieran ejecutado.

7.2.3. Controlando el Número de Trabajos Concurrentes

Por defecto, `parallel` ejecuta un trabajo por núcleo de CPU. Se puede controlar el número de trabajos que se ejecutarán concurrentemente con la opción `--jobs` o `-j`. Especificar un número significa que muchos trabajos se ejecutarán concurrentemente. Si se coloca un signo más delante del número, entonces `parallel` ejecutará N trabajos más el número de núcleos de CPU. Si se coloca un signo menos delante del número, entonces `parallel` ejecutará N-M trabajos. Donde N es el número de núcleos de CPU.

También se puede especificar un porcentaje, donde el valor por defecto es el 100 % del número de núcleos de CPU. El número óptimo de trabajos para ejecutar concurrentemente depende de los comandos reales que se estén ejecutando.

```
seq 5 | parallel -j0 "echo Hi {}"
```

```
seq 5 | parallel -j80% "echo Hi {}"
```

Si se especifica `-j1`, entonces los comandos se ejecutarán en serie. Esto tiene sus usos, por ejemplo, cuando se necesita acceder a una API que sólo permite una conexión a la vez. Si se especifica `-j0`, entonces `parallel` ejecutará tantos trabajos en paralelo como sea posible. Es equivalente a ejecutar un ciclo con el ampersand (`&`) y no es aconsejable.

7.2.4. Log y Salida

Para guardar el resultado de cada comando, puede se puede realizar en primera instancia lo siguiente:

```
seq 5 | parallel "echo \"Hi {}\" > hi-{}.txt"
ls hi*
```

Esto guardará el resultado en archivos individuales. O, si se quiere guardar todo en un solo archivo grande, se puede realizar lo siguiente:

```
seq 5 | parallel "echo Hi {}" >> one-big-file.txt
bat one-big-file.txt
```

Sin embargo, `parallel` ofrece la opción `--results`, que almacena el resultado en archivos separados. Para cada trabajo, `parallel` crea tres archivos: *seq*, que contiene el número de trabajo, *stdout* que contiene el resultado producido por el trabajo, y *stderr* que contiene cualquier error producido por el trabajo. Estos tres archivos se colocan en subdirectorios basados en los valores de entrada.

`parallel` sigue imprimiendo todo el resultado, lo cual es redundante en este caso. Se puede redirigir tanto la entrada estándar como la salida estándar a */dev/null* de la siguiente manera:

```
seq 10 |
parallel --results outdir "curl 'https://anapioficeandfire.com/api/characters/{}' |
jq -r '.aliases[0]'" 2>/dev/null 1>&2
l
tree outdir | trim
```

La Figura 15 muestra una descripción general de cómo funciona la opción `--results`.

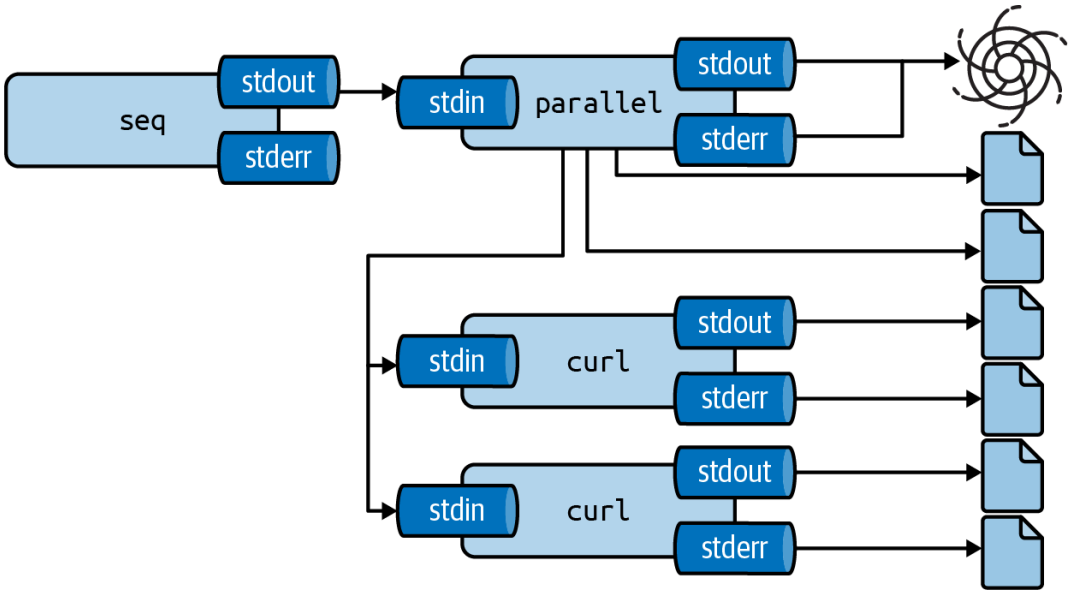


Figura 15: Diagrama de resultados en paralelo.

Cuando se ejecutan múltiples trabajos en paralelo, el orden en que se ejecutan los trabajos puede no corresponder al orden de la entrada. Por lo tanto, el resultado de los trabajos también se mezcla. Para mantener el mismo orden, especifique la opción `--keep-order` o la opción `-k`.

A veces es útil registrar qué entrada generó qué resultado. `parallel` permite *etiquetar* el resultado con la opción `--tag`, que antepone cada línea con el elemento de entrada.

```
seq 5 | parallel --tag "echo 'sqrt({})' | bc -l"
parallel --tag --keep-order "echo '{1}*{2}' | bc -l" ::: 3 4 ::: 5 6 7
```

7.2.5. Creando Herramientas Paralelas

La herramienta `bc`, que se utilizó anteriormente, no es paralela en sí misma. Sin embargo, se puede paralelizar usando `parallel`. La imagen de Docker contiene una herramienta llamada `pbc`:

```
bat $(which pbc)
```

Esta herramienta permite simplificar el código usado al principio del capítulo. Y puede procesar valores separados por comas simultáneamente:

```
seq 100 | pbc '{1}^2' | trim
paste -d, <(seq 4) <(seq 4) <(seq 4) | pbc 'sqrt({1}+{2})^{3}'
```

7.3. Procesamiento Distribuido

A veces necesitas más potencia que la de la que máquina local. Afortunadamente, `parallel` también puede aprovechar la potencia de las máquinas remotas, lo que realmente permite acelerar la ejecución. Para este efecto, `parallel` no tiene que estar instalado en la máquina remota. Todo lo que se requiere es que se pueda conectar a la máquina remota con el protocolo *Secure Shell* (o SSH), que es también lo que `parallel` usa para distribuir los comandos.

Sin embargo, este apartado de computación distribuida escapa al alcance del curso.

7.4. Para Profundizar en el Tema

Una vez se que tenga una comprensión básica de `parallel` y sus opciones más importantes, se recomienda consultar su tutorial en línea. Donde se explica, entre otras cosas, diferentes formas de especificar la entrada, mantener un registro de todos los trabajos y cómo establecer tiempos de espera, reanudar y reintentar trabajos.

8. Modelación de Datos

En este capítulo trabajaremos en el cuarto paso del modelo OSEMN: modelación de datos. En términos generales, un modelo es una descripción abstracta o de nivel superior de los datos. Modelar es un poco como crear visualizaciones en el sentido de que estamos dando un paso atrás de los puntos de datos individuales para ver el panorama general.

Las visualizaciones se caracterizan por formas, posiciones y colores: se pueden interpretar. Los modelos, por otro lado, se caracterizan internamente por números, lo que significa que los computadores pueden usarlos para hacer cosas como predicciones sobre nuevos puntos de datos.

En este capítulo consideraremos tres tipos de algoritmos comúnmente utilizados para modelar datos:

- Reducción de dimensionalidad.
- Regresión.
- Clasificación.

Supongamos que tenemos un archivo CSV, también conocido como un *conjunto de datos* o *dataset*. Cada fila, excepto el encabezado, se considera un *punto de datos* o una *observación*. Cada punto de datos tiene una o más *características*, *atributos*, *variables*, o *propiedades* que se han medido. A veces, un punto de datos también tiene una *etiqueta*, que corresponde, en términos generales, un resultado o clasificación.

El primer tipo de algoritmo (reducción de dimensionalidad) suele ser no supervisado, lo que significa que crea un modelo basado únicamente en las características del conjunto de datos. Los dos últimos tipos de algoritmos (regresión y clasificación) son por definición algoritmos supervisados, lo que significa que también incorporan las etiquetas en el modelo.

Este capítulo comienza con los siguientes archivos:

```
cd /data/ch09
ls
```

8.1. Vino

A lo largo de este capítulo, usaremos un conjunto de datos de notas de catadores de vino sobre variedades tintas y blancas de un vino portugués llamado *vinho verde*.

Cada punto de datos representa un vino. Cada vino se clasifica en 11 propiedades fisicoquímicas:

- acidez fija,
- acidez volátil,
- ácido cítrico,
- azúcar residual,
- cloruros,
- dióxido de azufre libre,
- dióxido de azufre total,

- densidad,
- pH,
- sulfatos y
- alcohol.

También hay una puntuación de calidad general entre 0 (muy mala) y 10 (excelente), que es la mediana de al menos tres evaluaciones de expertos en vinos. Se puede encontrar más información sobre este conjunto de datos en el Repositorio de aprendizaje automático de UCI.

El conjunto de datos se divide en dos archivos: uno para vino blanco y otro para vino tinto. El primer paso es obtener los dos archivos usando `curl` y `parallel`, ejecutando el siguiente comando:

```
parallel "curl -sL http://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/wi
::: red white
```

Los dos puntos triples son solo otra forma de pasar datos a `parallel`.

Si esto falla, ejecutar:

```
cp /data/.cache/wine-*.csv .
```

Inspeccionemos ambos archivos y contamos el número de líneas:

```
cat wine-red.csv | nl |
fold |
trim

cat wine-white.csv | nl | fold | trim
wc -l wine-{red,white}.csv
```

- Para mayor claridad,Se utiliza `nl` para agregar números de línea, para una mayor claridad.
- `fold` se utiliza para ver el encabezado completo.

A primera vista, estos datos parecen bastante limpios. Sin embargo, se procesarán para que se ajusten más a lo que esperan la mayoría de las herramientas de línea de comandos.

Específicamente:

- Convertir el encabezado a minúsculas.
- Reemplazar los punto y coma con comas.
- Reemplazar espacios con guiones bajos.
- Eliminar comillas innecesarias.

La herramienta `tr` puede encargarse de todos estos procedimientos:

```
for COLOR in red white; do
< wine-$COLOR.csv tr '[A-Z]; ' '[a-z],_' | tr -d \" > wine-${COLOR}-clean.csv
done
```

Vamos a crear también un único conjunto de datos combinando los dos archivos. Usaremos `csvstack` para agregar una columna llamada *type*, que será red para las filas del primer archivo y white para las filas del segundo archivo:

```
csvstack -g red,white -n type wine-{red,white}-clean.csv |
xsv select 2-,1 > wine.csv
```

- `csvstack` coloca la nueva columna *type* al principio.
- Algunos algoritmos asumen que la etiqueta es la última columna, así que usamos `xsv` para mover la columna *type* al final.

Es una buena práctica comprobar si faltan valores en este conjunto de datos, porque la mayoría de los algoritmos de aprendizaje automático no pueden manejarlos:

```
csvstat wine.csv --nulls
```

No hay valores faltantes. Si faltaran valores, se podrían llenar con, por ejemplo, el valor promedio o más común de esa característica. Un enfoque alternativo y menos sutil es eliminar cualquier punto de datos que tenga al menos un valor faltante. Veamos cómo se ve la distribución de la calidad para los vinos tintos y blancos.

```
rush run -t \
'ggplot(df, aes(x = quality, fill = type)) + geom_density(adjust = 3, alpha = 0.5)' \
wine.csv > wine-quality.png
```

En el gráfico de densidad se puede observar que la calidad del vino blanco se distribuye más hacia valores más altos. £Significa esto que los vinos blancos son en general mejores que los tintos, o que los expertos en vinos blancos dan más fácilmente puntuaciones más altas que los expertos en vinos tintos? Eso es algo que los datos no nos dicen. £O quizás existe una relación entre el alcohol y la calidad? Usemos `rush` para averiguarlo:

```
rush plot --x alcohol --y quality --color type \
--geom smooth wine.csv > wine-alcohol-vs-quality.png
```

8.2. Reducción de Dimensionalidad con Tapkee

El objetivo de la reducción de dimensionalidad es mapear puntos de datos de alta dimensionalidad en un espacio de menor dimensionalidad. El reto es mantener los puntos de datos similares juntos en el espacio de menor dimensionalidad. Como hemos visto en la sección anterior, nuestro conjunto de datos de vinos contiene 13 características. Utilizaremos solo dos dimensiones porque es mas fácil de visualizar.

La reducción de dimensionalidad a menudo se considera parte de la exploración. Es útil cuando hay demasiadas características para trazar. Se podría hacer una matriz de diagramas de dispersión, pero eso sólo muestra dos características a la vez. También es útil como paso de preprocesamiento para otros algoritmos de aprendizaje automático.

La mayoría de los algoritmos de reducción de dimensionalidad no están supervisados. Esto significa que no emplean las etiquetas de los puntos de datos para construir el mapeo de menor dimensionalidad.

En esta sección se presentarán dos técnicas: *PCA*, que significa Principal Component Analysis y t-SNE, que significa t-distributed Stochastic Neighbor Embedding.

8.2.1. Introducción a Tapkee

Tapkee es una biblioteca de plantillas `C++` para la reducción de dimensionalidad. La biblioteca contiene implementaciones de muchos algoritmos de reducción de dimensionalidad, incluyendo:

- Locally Linear Embedding
- Isomap
- Multidimensional Scaling
- PCA
- t-SNE

Se puede encontrar más información sobre estos algoritmos en el sitio web de Tapkee. Aunque Tapkee es principalmente una biblioteca que puede incluirse en otras aplicaciones, también ofrece una herramienta de línea de comandos `tapkee`, que utilizaremos para realizar la reducción de dimensionalidad en el conjunto de datos de vinos.

8.2.2. Mapeos Lineales y No Lineales

En primer lugar, se escalaran los atributos utilizando la estandarización de tal manera que cada característica sea igualmente importante. Esto generalmente conduce a mejores resultados al aplicar algoritmos de aprendizaje automático.

Para escalar se utiliza `rush` y el paquete `tidyverse`.

```
rush run --tidyverse --output wine-scaled.csv \
'select(df, -type) %>%
scale() %>%
as_tibble() %>%
mutate(type = df$type)' wine.csv
csvlook wine-scaled.csv | trim
```

1. Eliminar temporalmente la columna ***type*** porque `scale()` sólo funciona en columnas numéricas.
2. La función `scale()` acepta un data frame, pero devuelve una matriz.
3. La función `as_tibble()` convierte la matriz de nuevo en un data frame.
4. Finalmente, se vuelve a añadir la columna ***type***.

Ahora aplicamos ambas técnicas de reducción de dimensionalidad y visualizamos el mapeo utilizando `Rio-scatter`:

```
xsv select '!type' wine-scaled.csv |
header -d |
tapkee --method pca |
tee wine-pca.txt | trim
```

Aquí:

1. Deseleccionamos la columna *type*.
2. Eliminamos la cabecera.
3. Aplicamos PCA.

```
cat wine-pca.txt | header -a pc1,pc2 |
paste -d, - <(xsv select type wine-scaled.csv) |
tee wine-pca.csv | csvlook | trim
```

Aquí:

1. Añadimos la cabecera con las columnas *pc1* y *pc2*.
2. Añadimos la columna *type*.

Ahora podemos crear un diagrama de dispersión:

```
rush plot --x pc1 --y pc2 --color type --shape type wine-pca.csv > wine-pca.png
```

Vamos a realizar t-SNE con el mismo enfoque:

```
xsv select '!type' wine-scaled.csv |
header -d |
tapkee --method t-sne |
header -a x,y |
paste -d, - <(xsv select type wine-scaled.csv) |
rush plot --x x --y y --color type --shape type > wine-tsne.png
```

Aquí:

1. Deseleccionamos la columna *type*.
2. Eliminamos la cabecera.
3. Aplicamos t-SNE.
4. Añadimos la cabecera con las columnas *x* e *y*.
5. Añadimos la columna *type*.
6. Creamos un diagrama de dispersión.

Se puede observar que t-SNE hace un mejor trabajo que PCA en la separación de los vinos tintos y blancos en función de sus propiedades fisicoquímicas. Estos diagramas de dispersión verifican que el conjunto de datos tiene una cierta estructura; hay una relación entre las características y las etiquetas.

8.3. Regresión con Vowpal Wabbit

En esta sección, crearemos un modelo que prediga la calidad del vino blanco, basado en sus propiedades fisicoquímicas. Debido a que la calidad es un número entre 0 y 10, podemos considerar esto como una tarea de regresión.

Para esto utilizaremos Vowpal Wabbit, o `vw`.

8.3.1. Preparando los Datos

En lugar de trabajar con CSV, `vw` tiene su propio formato de datos. La herramienta `csv2vw` puede, como su nombre lo indica, convertir CSV a este formato. La opción `--label` se utiliza para indicar qué columna contiene las etiquetas. Examinemos el resultado:

```
csv2vw wine-white-clean.csv --label quality | trim
```

En este formato, cada línea es una observación. La línea comienza con la etiqueta, seguida de un símbolo de *pipe* (|) y luego pares nombre/valor de característica separados por espacios. Si bien este formato puede parecer demasiado complicado en comparación con el formato CSV, ofrece más flexibilidad como pesos, etiquetas, espacios de nombres y una representación de característica dispersa. Con el conjunto de datos de vino no se necesita esta flexibilidad, pero podría ser útil al aplicar `vw` a problemas más complicados. Este artículo explica el formato `vw` con más detalle.

Una vez que creado, o *entrenado* un modelo de regresión, se puede utilizar para hacer predicciones sobre nuevos puntos de datos no vistos. En otras palabras, si le damos al modelo un vino que no ha visto antes, puede predecir, o *probar*, su calidad. Para evaluar adecuadamente la precisión de estas predicciones, necesitamos reservar algunos datos que no se utilizarán para el entrenamiento. Es común utilizar el 80 % del conjunto de datos completo para el entrenamiento y el 20 % restante para las pruebas.

Se puede hacer esto dividiendo primero el conjunto de datos completo en cinco partes iguales usando `split`. Verificamos la cantidad de observaciones en cada parte usando `wc`.

```
csv2vw wine-white-clean.csv --label quality |
shuf |
split -d -n r/5 - wine-part-
wc -l wine-part-*
```

- La herramienta `shuf` aleatoriza el conjunto de datos para garantizar que tanto el entrenamiento como la prueba tengan una distribución de calidad similar.

Ahora se puede usar la primera parte (por lo tanto, el 20 %) para el conjunto de pruebas *wine-test.vw* y combinar las cuatro partes restantes (por lo tanto, el 80 %) en el conjunto de entrenamiento *wine-train.vw*:

```
mv wine-part-00 wine-test.vw
cat wine-part-* > wine-train.vw
```

```
rm wine-part-*
wc -l wine-*.vw
```

8.3.2. Entrenando el Modelo

La herramienta `vw` acepta muchas opciones diferentes (¸casi 400!). Afortunadamente, no se necesitan todas para ser efectivo.

Se coloca cada opción en una linea separada para explicarlo posteriormente:

```
vw \
--data wine-train.vw \
--final_regressor wine.model \
--passes 10 \
--cache_file wine.cache \
--nn 3 \
--quadratic :: \
--l2 0.000005 \
--bit_precision 25
```

1. El archivo *wine-train.vw* se usa para entrenar el modelo.
2. El modelo, o *regresor*, se almacenará en el archivo *wine.model*.
3. Número de pases de entrenamiento.
4. Se necesita almacenamiento en caché cuando se realizan múltiples pases.
5. Se utiliza una red neuronal con 3 unidades ocultas.
6. Se crea y usa características cuadráticas, basadas en todas las características de entrada. Cualquier duplicado será eliminado por `vw`.
7. Usa regularización l2.
8. Usa 25 bits para almacenar las características.

8.3.3. Probando el Modelo

El modelo se almacena en el archivo *wine.model*. Para usar ese modelo para hacer predicciones, se ejecuta `vw` nuevamente, pero ahora con un conjunto diferente de opciones:

```
vw \
--data wine-test.vw \
--initial_regressor wine.model \
--testonly \
--predictions predictions \
--quiet
bat predictions | trim
```

1. El archivo *wine-test.vw* se usa para probar el modelo.
2. Use el modelo almacenado en el archivo *wine.model*.
3. Ignorar la información de etiqueta y solo probar.

4. Las predicciones se almacenan en un archivo llamado *predictions*.
5. No emita diagnósticos ni actualizaciones de progreso.

Usaremos `paste` para combinar las predicciones en el archivo *predictions* con los valores verdaderos, u *observados*, que están en el archivo *wine-test.vw*. Usando `awk`, se puede comparar los valores predichos con los valores observados y calcular el error absoluto medio (MAE, por sus siglas en ingles). El MAE nos dice qué tan lejos está `vw` en promedio, cuando se trata de predecir la calidad de un vino blanco.

```
paste -d, predictions <(cut -d '|' -f 1 wine-test.vw) |
tee results.csv |
awk -F, '{E+=sqrt(($1-$2)^2)} END {print "MAE: " E/NR}' |
cowsay
```

Entonces, las predicciones están en promedio alrededor de 0.6 puntos de diferencia. Visualizamos la relación entre los valores observados y los valores predichos usando `rush plot`:

```
cat results.csv | header -a "predicted,observed" |
rush plot --x observed --y predicted --geom jitter

cat results.csv | header -a "predicted,observed" |
rush plot --x observed --y predicted --geom jitter > wine-regression.png
```

Veamos cómo funciona `vw` cuando se usan todos los valores predeterminados:

```
vw -d wine-train.vw -f wine2.model --quiet
vw -data wine-test.vw -i wine2.model -t -p predictions --quiet
paste -d, predictions <(cut -d '|' -f 1 wine-test.vw) |
awk -F, '{E+=sqrt(($1-$2)^2)} END {print "MAE: " E/NR}'
```

1. Entrenar un modelo de regresión
2. Probar el modelo de regresión
3. Calcular el error absoluto medio

Aparentemente, con los valores predeterminados, el MAE es 0.04 más alto, lo que significa que las predicciones son ligeramente peores.

En esta sección, solo se ha podido revelar la superficie de lo que `vw` puede hacer. Hay una razón por la que acepta tantas opciones. Además de la regresión, también admite, entre otras cosas, clasificación binaria, clasificación multiclase, aprendizaje por refuerzo y asignación de Dirichlet latente. Su sitio web contiene muchos tutoriales y artículos para aprender más.

8.4. Clasificación con SciKit-Learn Laboratory

En esta sección se entrenará un modelo de clasificación, o *clasificador*, que predice si un vino es tinto o blanco. Si bien se puede utilizar `vw` para esto, demostraremos otra herramienta: SciKit-Learn Laboratory (SKLL).

Como su nombre lo indica, está construido sobre SciKit-Learn, un paquete popular de aprendizaje automático para Python. SKLL, que también es un paquete de Python, proporciona la herramienta `run_experiment`, que permite usar SciKit-Learn desde la línea de comandos. En lugar de `run_experiment`, se utiliza el alias `skll` que es más fácil de recordar, ya que corresponde al nombre del paquete:

```
alias skll=run_experiment
skll
```

8.4.1. Preparando los Datos

`skll` espera que el conjunto de datos de entrenamiento y prueba tengan los mismos nombres de archivo, ubicados en directorios separados. Debido a que sus predicciones no están necesariamente en el mismo orden que el conjunto de datos original, agregamos una columna, *id*, que contiene un identificador único para que pueda hacer coincidir las predicciones con las observaciones correctas.

Creamos un conjunto de datos equilibrado:

```
NUM_RED="$(< wine-red-clean.csv wc -l)"
csvstack -n type -g red,white \
wine-red-clean.csv \
<(< wine-white-clean.csv body shuf | head -n $NUM_RED) |
body shuf |
nl -s, -w1 -v0 |
sed '1s/0,/id,/' |
tee wine-balanced.csv | csvlook
```

1. Almacenar el número de vinos tintos en la variable *NUM_RED*.
2. Combinar todos los vinos tintos con una muestra aleatoria de vinos blancos.
3. Agregar números de línea usando `nl` delante de cada línea.
4. Reemplazar el 0 en la primera línea con id para que sea un nombre de columna adecuado.

Dividimos este conjunto de datos equilibrado en un conjunto de entrenamiento y un conjunto de prueba:

```
mkdir -p {train,test}
HEADER="$(< wine-balanced.csv header)"
< wine-balanced.csv header -d | shuf | split -d -n r/5 - wine-part-
wc -l wine-part-*
cat wine-part-00 | header -a $HEADER > test/features.csv && rm wine-part-00
cat wine-part-* | header -a $HEADER > train/features.csv && rm wine-part-*
wc -l t*/features.csv
```

8.4.2. Ejecutando el Experimento

Entrenar un clasificador en `skll` se hace definiendo un experimento en un archivo de configuración. Consiste en varias secciones que especifican, por ejemplo, dónde buscar los conjuntos de datos, qué clasificadores, etc.

Aquí está el archivo de configuración *classify.cfg* que utilizaremos:

```
bat classify.cfg
```

Ejecutamos el experimento usando `skll`:

```
skll -l classify.cfg 2>/dev/null
```

La opción `-l` especifica que se ejecute en modo local. `skll` también ofrece la posibilidad de ejecutar experimentos en clústeres. El tiempo que tarda en ejecutarse un experimento depende de la complejidad de los algoritmos elegidos y del tamaño de los datos.

8.4.3. Analizando los Resultados

Una vez que todos los clasificadores han sido entrenados y probados, los resultados se pueden encontrar en el directorio *output*:

```
ls -1 output
```

`skll` genera cuatro archivos para cada clasificador: un registro, dos con resultados y uno con predicciones. Extraemos los nombres de los algoritmos y los ordenamos por su precisión usando la siguiente consulta SQL:

```
cat output/wine_summary.tsv |
csvsql --query "SELECT learner_name, accuracy FROM stdin ORDER BY accuracy DESC" |
csvlook -I
```

La columna relevante aquí es ***accuracy***, que indica el porcentaje de puntos de datos que se clasifican correctamente. De esto vemos que en realidad todos los algoritmos están funcionando muy bien. El *RandomForestClassifier* es el algoritmo con mejor rendimiento, seguido de cerca por el *KNeighborsClassifier*.

Cada archivo JSON contiene una matriz de confusión, que le da una visión adicional del rendimiento de cada clasificador. Una matriz de confusión es una tabla donde las columnas se refieren a las etiquetas verdaderas (tinto y blanco) y las filas se refieren a las etiquetas predichas. Los números más altos en la diagonal significan más predicciones correctas. Con `jq` podemos imprimir el nombre de cada clasificador y extraer la matriz de confusión asociada:

```
jq -r '.[] | "\(.learner_name):\n\(.result_table)\n"' output/*.json
```

Una matriz de confusión es especialmente útil cuando se tienen más de dos clases, para poder visualizarqué tipo de clasificaciones erróneas ocurren, y cuándo el costo de una clasificación incorrecta no es el mismo para cada clase.

Desde una perspectiva de uso, es interesante considerar que `vw` y `skll` toman dos enfoques diferentes. `vw` usa opciones de línea de comandos, mientras que `skll` requiere un archivo separado. Ambos enfoques tienen sus ventajas y desventajas. Si bien las opciones de línea de comandos permiten un uso más ad-hoc, un archivo de configuración es quizás más fácil de reproducir. Por otro lado, como hemos visto, invocar `vw` con cualquier número de opciones se puede colocar fácilmente en un script. Lo contrario, hacer que `skll` acepte opciones de modo que no necesite un archivo de configuración, es menos sencillo.

8.5. Para Mayor Exploración

- El libro *Python Machine Learning* de Sebastian Raschka y Vahid Mirjalili ofrece una visión general completa del aprendizaje automático y cómo aplicarlo usando Python.
- Los últimos capítulos de *R for Everyone* de Jared Lander explican cómo realizar diversas tareas de aprendizaje automático usando R.
- Si se quiere obtener una comprensión más profunda del aprendizaje automático, se recomienda *Pattern Recognition and Machine Learning* de Christopher Bishop y *Information Theory, Inference, and Learning Algorithms* de David MacKay.
- Si se está interesado en aprender más sobre el algoritmo t-SNE, se recomienda el artículo original sobre él: *Visualizing Data Using T-SNE* de Laurens van der Maaten y Geoffrey Hinton.

  Cabe destacar que se pueden crear los propios scripts R o Python para crear modelos de clasificación o regresión según la necesidad de cada caso, y llamarlos directamente desde la terminal.

9. Nushell

Nushell es un intérprete de línea de comandos contemporáneo que reinventa la experiencia tradicional de shell al representar todos los datos en un formato estructurado. A diferencia de los shells convencionales que procesan secuencias o cadenas de texto, Nushell aprovecha este enfoque de datos estructurados para facilitar la manipulación y el procesamiento intuitivos de datos a través de *pipes*. Esto permite a los usuarios encadenar comandos sin problemas, pasando datos estructurados entre ellos utilizando el símbolo *pipe* (|), lo que hace que las operaciones complejas sean más eficientes y legibles. Diseñado pensando en los desarrolladores modernos, Nushell ofrece un lenguaje de scripting robusto, se integra a la perfección con varios formatos de datos y prioriza la experiencia del usuario con una rica interactividad y mensajes de error informativos.

En este capitulo, trabajaremos fuera del contenedor docker.

9.1. Instalación de nushell

A continuación el conjunto de instrucciones necesarias para instalar nushell:

```
#dependencias
sudo apt install -y pkg-config libssl-dev build-essential

#instalacion de rust
cd
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source .bashrc

#limitar uso de memoria para cargo
mem_gb=$(awk '/^MemTotal:/ {print int($2/1024/1024)}' /proc/meminfo)
cargo_jobs=$((mem_gb / 2))
[ "$cargo_jobs" -lt 1 ] && cargo_jobs=1
[ "$cargo_jobs" -gt "$(nproc)" ] && cargo_jobs="$(nproc)"
export CARGO_BUILD_JOBS=$cargo_jobs

#instalacion de nu y plugins
cargo install --locked nu

cargo install nu_plugin_inc nu_plugin_gstat nu_plugin_query nu_plugin_formats
cargo install nu_plugin_polars

#ejecutar nu, decir si a todo
nu
```

Una vez en nushell, ejecutar lo siguiente para poder utilizar los plugins:

```
plugin add ~/.cargo/bin/nu_plugin_inc
```

```
plugin add ~/.cargo/bin/nu_plugin_gstat
plugin add ~/.cargo/bin/nu_plugin_query
plugin add ~/.cargo/bin/nu_plugin_formats
plugin add ~/.cargo/bin/nu_plugin_polars

plugin use ~/.cargo/bin/nu_plugin_inc
plugin use ~/.cargo/bin/nu_plugin_gstat
plugin use ~/.cargo/bin/nu_plugin_query
plugin use ~/.cargo/bin/nu_plugin_formats
plugin use ~/.cargo/bin/nu_plugin_polars
```

Los plugins integran funcionalidades adicionales a nushell.

Por otro lado, cambiaremos el editor de texto utilizado hasta el momento, `nano`, por uno que permite entre otras funcionalidades, resaltado de sintaxis:

```
cargo install --git https://github.com/curlpipe/ox ox
```

9.2. Elementos básicos de nushell

En esta sección se revisarán los primeros elementos de nushell.

Nushell funciona bajo la base de que todo son datos estructurados, y por lo tanto se pueden realizar consultas, similares a SQL, directamente en la salida de cada comando. Comenzamos en el siguiente directorio:

```
# cd path/to/DSITCL/
cd ch09
ls
```

Observar como `ls` retorna en lugar de que vuelva un bloque de texto plano, una tabla estructurada. La tabla hace más que mostrar el directorio de una manera diferente. Al igual que las tablas en una hoja de cálculo, esta tabla permite trabajar con los datos de forma más interactiva.

Para ordenar la tabla por tamaño, tomamos la salida de `ls` y la introduciremos en un comando que puede ordenar tablas en función del contenido de una columna.

```
ls | sort-by size | reverse
ls | sort-by size -r
ls | sort-by modified
```

Se puede observar que para que esto funcione no se utilizan argumentos de línea de comandos de `ls`. En su lugar, se utiliza el comando `sort-by` que Nu proporciona para hacer la clasificación de la salida del comando `ls`. Para ver los archivos más grandes en la parte superior, también se utiliza `reverse` o la opcion `-r` de `sort-by`.

Nu proporciona muchos comandos que pueden funcionar en tablas. Por ejemplo, se puede utilizar `where` para filtrar el contenido de la tabla `ls` para que sólo muestre los archivos de más de 5

kilobytes

```
ls | where size > 100kb
```

Al igual que en la filosofía Unix, ser capaz de hacer que los comandos se comuniquen entre sí entrega formas de mezclar y combinar en muchas combinaciones diferentes. Veamos un comando diferente:

```
ps
```

Este comando entrega una lista de todos los procesos actuales que el sistema está ejecutando, cuál es su estado y cuál es su nombre. También podemos ver la carga de CPU de los procesos.

Si se quisiera mostrar los procesos que están utilizando activamente la CPU, se puede proceder como sigue:

```
ps | where cpu > 0
```

Nu no solo permite tablas estructuradas de forma simple, si no que también permite anidaciones arbitrarias. Por ejemplo, consideremos el comando `help commands`. Este comando proporciona información para todos los comandos de Nushell como una tabla.

```
help commands
```

La salida es bastante extensa, por lo que se obtendra la fila para el comando `each`.

```
help commands | where name == each
help commands | where name == each | first
```

Al ecuperar una sola fila de una tabla se obtiene un *registro* (*record*), que es un conjunto de pares clave-valor. Observar como la columnas signatures contiene un registro en lugar de valores simples. Para ver sólo una de estas claves o columnas, se puede utilizar el comando `get`:

```
help commands | where name == each | first | get description
help commands | where name == each | first | get params
```

El comando `get` nos permite saltar al contenido de los datos estructurados (una tabla, registro o lista). Incluso se puede pasar columnas anidadas para acceder a los datos a cualquier profundidad.

```
help commands | where name == each | first | get category
help commands | where name == each | first | get command_type
help commands | where name == each | first | get description
help commands | where name == each | first
```

Estas columnas anidadas se llaman *rutas de celda* (*cellpaths*). Nu tomará la ruta de la celda e irá al bit de datos correspondiente en una tabla, registro o lista. Las rutas de celda también admiten números de fila, por lo que los comandos anterior también se puede escribir como:

```
help commands | where name == each | first | get params
help commands | where name == each | first | get params.0
help commands | where name == each | first | get params.3.description
```

Si se quiere ver la estructura anidada, se puede utilizar `table -e`:

```
help commands | where name == each | first | table -e
```

Notar que se pueden acceder a todas las herramientas estandar de la terminal utilizando el símbolo ^ si existe una versión nushell. Si no existe, no es necesario utilizarlo:

```
^ls
cat wine.csv
^ps
^ps -X
```

Todo comando nushell que retorna datos lo hará en una tabla, por ejemplo:

```
#muestra los procesos en ejecucion
ps

#ordenados por mayor utilizacion de cpu
ps | where cpu > 0 | sort-by cpu -r
```

9.2.1. Obtener Ayuda

Se puede leer texto de ayuda para cualquiera de los comandos incorporados de Nu usando el comando `help` o pasando la opción `--help` a un comando. También se puede buscar un tema utilizando `help -f <topic>`.

```
help explore
help http get
help -f open
```

9.3. Trabajando con tablas

Una de las maneras comunes de ver datos en Nu es a través de una tabla. Nu viene con una serie de comandos para trabajar con tablas para que sea conveniente encontrar lo que se está buscando, y para reducir los datos a solo lo que se necesita.

Para comenzar, obtenemos una tabla que podamos usar:

```
ls
```

9.3.1. Ordenando los Datos

Se puede ordenar una tabla llamando al comando `sort-by` y estableciendo qué columnas utilizar para ordenar. Si se quisiese ordenar la tabla anterior por el tamaño del archivo:

```
ls | sort-by size
```

Se puede ordenar una tabla por cualquier columna que se pueda comparar. Por ejemplo, se podría también haber ordenado la anterior usando las columnas name, accessed, o modified.

9.3.2. Seleccionando Datos

Se puede seleccionar datos de una tabla eligiendo seleccionar columnas específicas o filas específicas, con el comando `select`:

```
ls | select name size
```

Esto ayuda a crear una tabla que está más enfocada en lo que se necesitan. Si solo quisiéramos ver los 5 archivos más pequeños en este directorio, ejecutamos:

```
ls | sort-by size | first 5
```

Notar que primero se ordenam la tabla por tamaño para llegar al archivo más pequeño, y luego se utiliza `first 5` para devolver las primeras 5 filas de la tabla.

También se pueden saltar filas que no se necesitan:

```
ls | sort-by size | first 5 | skip 2
```

También se pueden seleccionar filas directamente:

```
ls | sort-by name | select 5
```

9.3.3. Obteniendo Datos de una Tabla

Hasta ahora, se ha trabajado con tablas recortando la tabla a solo lo que se necesita. A veces, se puede necesitar ir un paso más allá y solo mirar los valores en las celdas en sí mismas en lugar de tomar una columna completa. Por ejemplo, supongamos que queremos obtener solo una lista de los nombres de los archivos:

```
ls | get name
```

Esto podría verse como el comando `select` que se vió anteriormente:

```
ls | select name
```

Se ven similares, pero hay diferencias:

- `select`: crea una nueva tabla que incluye solo las columnas especificadas.
- `get`: devuelve los valores dentro de la columna especificada como una lista.

9.3.4. Cambiando Datos en una Tabla

En adición a seleccionar datos de una tabla, también se pueden actualizar lo que la tabla tiene. Se puede necesitar combinar tablas, añadir nuevas columnas, o editar el contenido de una celda. En Nu, en lugar de editar en el lugar, cada uno de los comandos en la sección devolverá una nueva tabla.

9.3.4.1. Concatenando Tablas

Se pueden concatenar tablas usando `append`:

```
let first = [[a b]; [1 2]]
let second = [[a b]; [3 4]]

$first
$second

$first | append $second
```

Si los nombres de columna no son idénticos, entonces, adicionalmente, se crearán columnas y valores según sea necesario:

```
let first = [[a b]; [1 2]]
let second = [[a b]; [3 4]]
let third = [[a c]; [3 4]]

$first | append $second | append $third
```

También puede usar el operador `++` como un reemplazo en línea para `append`:

```
$first ++ $second ++ $third
```

9.3.4.2. Combinando Tablas Se puede utilizar el comando `merge` para combinar dos (o más) tablas:

```
let first = [[a b]; [1 2]]
let second = [[c d]; [3 4]]

$first | merge $second
```

Con una tercera tabla:

```
let third = [[e f]; [5 6]]
```

Se pueden unir las tres tablas de esta manera:

```
$first | merge $second  | merge $third
```

O se podría usar el comando `reduce` para combinar dinámicamente todas las tablas:

```
[$first $second $third] | reduce {|elt, acc| $acc | merge $elt }
```

9.3.5. Añadiendo una columna

Se puede utilizar el comando `insert` para agregar una nueva columna a la tabla. Por ejemplo:

```
{edition: 2024} | save -f edition.json
open edition.json
```

Agregamos una columna llamada next_edition con el valor 2025:

```
open edition.json | insert next_edition 2025
```

Notar que esta visualización puede ser ligeramente confusa, porque parece que lo que se acaba de hacer es añadir una fila. En este caso, se debe recordar que las filas tienen números, las columnas tienen nombres. Si todavía es confuso, se puede añadir una fila mas, lo quehará que la tabla se renderice como se espera:

```
open edition.json | insert next_edition 2025 | append {edition: 2021 next_edition: 2023}
```

Notar que si se abre el archivo original, el contenido se ha mantenido igual:

```
open edition.json
```

Los cambios en Nu son cambios funcionales, lo que significa que trabajan en los valores en sí mismos en lugar de intentar causar un cambio permanente. Esto permite hacer muchos tipos diferentes de trabajo en la *pipeline* hasta que se está listo para escribir el resultado con los cambios elegidos. Se puede escribir el resultado usando el comando `save`:

```
open edition.json
| insert next_edition 2025
| append {edition: 2021 next_edition: 2023}
| save -f edition2.json

open edition.json
open edition2.json
```

9.3.6. Actualizando una Columna

De forma similar al comando `insert`, el comando [`update` sirve para cambiar el contenido de una columna a un nuevo valor.

```
open edition.json
```

Y ahora, actualizamos la edición:

```
open edition.json | update edition 2025
```

También se puede utilizar el comando `upsert`para insertar o actualizar dependiendo de si la columna ya existe.

9.3.7. Moviendo Columnas

Se puede usar `move` para mover columnas en una tabla. Por ejemplo, si se quisiera mover la columna name de `ls` después de la columna size, se puede realizar:

```
ls

ls | move name --after size
```

9.3.8. Renombrando Columnas

También se puede columnas en una tabla pasándola por el comando `rename`:

```
ls | rename filename filetype filesize filedate
```

9.3.9. Rechazando/Eliminando Columnas

También pueden rechazar/eliminar columnas en una tabla pasándola por el comando de `reject`:

```
ls -l
```

```
ls -l | reject readonly num_links inode created accessed modified
```

9.4. Moviéndose por el Sistema

Una característica definitoria de un shell es la capacidad de navegar e interactuar con el sistema de archivos. Nushell es, por supuesto, ninguna excepción. Aquí hay algunos comandos comunes que puede usar al interactuar con el sistema de archivos:

9.4.1. Viendo el Contenido de un Directorio

```
ls
```

Como se ve indicó anteriormente, el comando `ls` devuelve el contenido de un directorio. El `ls` de Nushell devolverá el contenido como una tabla.

El comando `ls` también toma un argumento opcional para cambiar lo que se quiere ver. Por ejemplo, se puede listar los archivos que terminan en .csv

```
ls *.csv
```

9.4.2. Patrones Glob (comodines)

Al asterisco (`*`) en el argumento opcional anterior `*.csv` a veces se llama comodín o glob. Permite hacer coincidir cualquier cosa. Se puede leer este glob `*.csv` como *coincidir con cualquier nombre de archivo, siempre que termine con .csv.*

El glob más general es `*`, que coincidirá con todas las rutas. Con mayor frecuencia, se verá este patrón utilizado como parte de otro patrón, por ejemplo, `*.bak` y `temp*`.

Nushell también admite un `*` doble que atravesará rutas que están anidadas dentro de otros directorios. Por ejemplo, `ls **/*` enumerará todas las rutas no ocultas anidadas en el directorio actual, recursivamente.

```
ls **/*.csv
```

Aquí, buscanmos cualquier archivo que termine con .csv. Los dobles asteriscos especifican adicionalmente *en cualquier directorio a partir de aquí.*

La sintaxis globbing de Nushell no solo admite `*`, sino también la coincidencia de caracteres individuales con `?` y grupos de caracteres con `[...]`.

Escapar los patrones `*`, `?` y `[]` funciona delimitándolos entre comillas simples, comillas dobles o cadena raw. Por ejemplo, para mostrar el contenido de un directorio llamado `[slug]`, se utiliza use `ls "[slug]"` o `ls '[slug]'`.

Sin embargo, las cadenas delimitados con *comillas inversas* (o backtips) no escapan los glob. Por ejemplo, compare los siguientes escenarios:

1. Sin comillas: Patrón glob

   Una *cadena de palabra simple* sin comillas con caracteres glob se interpreta como un patrón glob, por lo que lo siguiente eliminará todos los archivos en el directorio actual que contengan `myfile` como cualquier parte del nombre del archivo:

   ```
   rm *myfile*
   ```

2. Entre comillas: Literal de cadena con asteriscos

   Cuando se cita con comillas simples o dobles, o se usa una *cadena raw*, se pasa al comando una *cadena* con los asteriscos literales escapados (u otros caracteres glob). El resultado no es un glob. El siguiente comando solo eliminará un archivo llamado literalmente `*myfile*` (incluidos los asteriscos). Otros archivos con `myfile` en el nombre no se ven afectados:

   ```
   rm "*myfile*"
   ```

3. Entre comillas invertidas: Patrón glob

   Los asteriscos (y otros patrones glob) dentro de una *cadena entre comillas invertidas* se interpretan como un patrón glob. Observe que este es el mismo comportamiento que el del ejemplo de cadena de palabra simple en el n.ř 1 anterior.

   Lo siguiente, como en el primer ejemplo, elimina todos los archivos en el directorio actual que contienen `myfile` como parte del nombre del archivo

   ```
   rm `*myfile*`
   ```

   Nushell también incluye un comando `glob` dedicado con soporte para escenarios globbing más complejos.

9.4.2.1. Conversión de Cadenas a Globs Las técnicas de comillas mencionadas anteriormente son útiles al construir globos literales, pero es posible que se deban construir globs mediante programación. Hay varias técnicas disponibles para este propósito:

1. `into glob`

   El comando `into glob` se puede usar para convertir una cadena (y otros tipos) en un glob. Por ejemplo:

```
# Encuentra archivos cuyo nombre incluya el mes actual en el formato YYYY-mm
let current_month = (date now | format date '%Y-%m')
let glob_pattern = ($"*($current_month)*" | into glob)
ls $glob_pattern
```

2. El operador de propagación (*spread operator*) combinado con el comando `glob`:

   El comando `glob` (nota: no es lo mismo que `into glob`) produce una `list` de nombres de archivo que coinciden con el patrón de globo. Esta lista se puede expandir y pasar a los comandos del sistema de archivos utilizando el operador de propagación `...`:

   ```
   # Encuentra archivos cuyo nombre incluya el mes actual en el formato YYYY-mm
   let current_month = (date now | format date '%Y-%m')
   ls ...(glob $"*($current_month)*")
   ```

3. Forzar el tipo `glob` mediante anotación:

   ```
   # Encuentra archivos cuyo nombre incluya el mes actual en el formato YYYY-mm
   let current_month = (date now | format date '%Y-%m')
   let glob_pattern: glob = ($"*($current_month)*")
   ls $glob_pattern
   ```

9.5. Como entender nu

9.5.1. Nushell no es bash

Nushell es tanto un lenguaje de programación como un shell. Debido a esto, tiene su propia forma de trabajar con archivos, directorios, sitios web y más. Se ha modelado para que funcione en estrecha colaboración con lo que se puede estar familiarizado con otros shells. Las *pipelines* funcionan uniendo dos comandos:

```
ls | length
```

Nushell, por ejemplo, también tiene soporte para otras capacidades comunes como obtener el código de salida de comandos ejecutados previamente.

Si bien tiene estas comodidades, Nushell no es bash. La forma de trabajar de bash, y el estilo POSIX en general, no es algo que Nushell soporte. Por ejemplo, en bash, se podrías usar:

```
echo "hello" > output.txt
```

En Nushell, se usa `>` como el operador mayor que. Esto encaja mejor con el aspecto del lenguaje de Nushell. En cambio, se canalizas a un comando que tiene la tarea de guardar contenido:

```
"hello" | save output.txt
```

La forma en que Nushell ve los datos es que los datos fluyen a través de la *pipeline* hasta que llegan al usuario o son manejados por un comando final. Se puede simplemente escribir datos, desde cadenas hasta listas y registros de estilo JSON, y seguirlos con `|` para enviarlos a través

de la *pipeline*. Nushell usa comandos para hacer el trabajo y producir más datos. Aprender estos comandos y cuándo usarlos ayuda a componer muchos tipos de *pipelines*.

9.5.2. Nushell como un lenguaje compilado

Una parte importante del diseño de Nushell y específicamente donde difiere de muchos lenguajes dinámicos es que Nushell convierte el código fuente en algo para ejecutar, y luego ejecuta el resultado. No tiene una función `eval` que permita continuar introduciendo nuevo código fuente durante el tiempo de ejecución. Esto significa que las tareas como incluir archivos para que formen parte de un proyecto deben ser rutas conocidas, al igual que las inclusiones en lenguajes compilados como C++ o Rust.

Por ejemplo, lo siguiente no tiene sentido en Nushell, y fallará al ejecutarse si se ejecuta como un script:

```
"def abc [] { 1 + 2 }" | save output.nu
source "output.nu"
abc
```

El comando `source` aumentará el código fuente que se compila, pero el `save` de la línea anterior no habrá tenido la oportunidad de ejecutarse. Nushell ejecuta todo el bloque como si fuera un solo archivo, en lugar de ejecutar una línea a la vez. En el ejemplo, dado que el archivo `output.nu` no se crea hasta después del paso de compilación, el comando `source` no puede leer las definiciones del mismo durante el tiempo de análisis.

Otro problema común es intentar crear dinámicamente el nombre de archivo desde el que obtener el código fuente:

```
source $"($my_path)/common.nu"
```

Esto no funciona si `my_path` es una variable de tiempo de ejecución regular declarada con `let`. Esto requeriría que el evaluador se ejecutara y evaluara la cadena, pero desafortunadamente Nushell necesita esta información en tiempo de compilación.

Sin embargo, si `my_path` es una constante, entonces esto funcionaría, ya que la cadena se puede evaluar en tiempo de compilación:

```
const my_path = ([$nu.home-path nushell] | path join)
source $"($my_path)/common.nu" # sources /home/user/nushell/common.nu
```

Nushell está diseñado para usar un solo paso de compilación para todo el código fuente, y esto es independiente de la evaluación. Esto permitirá un fuerte soporte IDE, mensajes de error precisos, un lenguaje más fácil de usar para herramientas de terceros y, en el futuro, incluso una salida más sofisticada, como la posibilidad de compilar Nushell directamente a un archivo binario.

9.5.3. Las variables son inmutables por defecto

Otra sorpresa común para las personas que vienen de otros lenguajes es que las variables de Nushell son inmutables por defecto. Al llegar a Nushell, se querrá dedicar algún tiempo a familiarizarte con el trabajo en un estilo más funcional, ya que esto tiende a ayudar a escribir código que funciona mejor con variables inmutables.

Si se está acostumbrado a usar variables mutables para diferentes tareas, llevará algún tiempo aprender a hacer cada tarea en un estilo más funcional. Nushell tiene un conjunto de capacidades integradas para ayudar con muchos de estos patrones, y aprenderlos ayudará a escribir código en un estilo más Nushell. El beneficio adicional de acelerar scripts ejecutando partes del código en paralelo es una buena ventaja.

9.6. Formateando salida de comandos (parsing)

Nushell ofrece la posibilidad de transformar la salida de comandos nushell, con diferentes formas de lograr el mismo objetivo.

Las funciones integradas que se pueden utilizar incluyen:

- `lines`
- `detect columns`
- `parse`
- `str ...`
- `from ssv`

A continuación, se muestran algunos ejemplos ilustrativos de salida tabular.

9.6.1. `detect columns` (bastante automático)

Esta herramienta intenta dividir automáticamente el texto en varias columnas.

```
df -h
```

```
df -h | str replace "Mounted on" Mounted_On | detect columns
```

Para una salida como la de `df` esta es probablemente la forma más compacta de lograr una salida tabular. `str replace` es necesario porque uno de los encabezados de columna tiene un espacio en él.

9.6.2. Usando `from ssv`

También se puede usar el analizador de datos `from` incorporado para `ssv` (*s*pace *s*eparated *v*alues):

```
df -h | str replace "Mounted on" Mounted_On | from ssv --aligned-columns --minimum-spaces 1
```

La salida es idéntica al ejemplo anterior. `from ssv` admite varias opciones para ajustar su comportamiento. Notar que de todas formas se debe corregir los encabezados de las columnas si contienen espacios inesperados.

9.6.3. Usando `parse`

Permite analizar un patrón arbitrario de una cadena de texto en una tabla de varias columnas.

```
cargo search shells --limit 10

cargo search shells --limit 10 | lines | parse "{crate_name} = {version} #{description}" |
```

9.7. Descargando datos

Nushell tiene su propia herramienta de descarga, similar a `curl`, llamada `http get`.

9.7.1. Descargando un archivo json

Por ejemplo, para descargar un archivo desde internet, utilizamos:

```
curl https://jsonplaceholder.typicode.com/posts
http get https://jsonplaceholder.typicode.com/posts
```

Podemos almacenar el resultado en una variable:

```
let response = http get https://jsonplaceholder.typicode.com/posts
$response
```

Y explorar el resultado con `explore`:

```
$response | explore
```

O podemos ver directamente los primeros elementos:

```
$response | first 10
```

Tambien podemos guardar el resultado en un archivo:

```
$response | save -f posts.json

ls
open posts.json
```

9.7.2. Descargando desde multiples urls

Supongamos que se están consultando varios *endpoints*, tal vez con diferentes parámetros de consulta y se desea ver todas las respuestas como un único conjunto de datos. Se puede usar `each` para ejecutar comandos nu en cada fila de datos.

Por ejemplo:

```
{
  "urls": [
    "https://jsonplaceholder.typicode.com/posts/1",
    "https://jsonplaceholder.typicode.com/posts/2",
    "https://jsonplaceholder.typicode.com/posts/3"
  ]
} | save -f urls.json

open urls.json
open urls.json | get urls
```

Luego, ejecutamos:

```
open urls.json | get urls | each { |u| http get $u }
```

Si se especifica la opcion `--raw`, el archivo json resultante no se transformará en una tabla:

```
open urls.json | get urls | each { |u| http get $u -r }
```

Para combinar estas respuestas en un formato JSON válido, se puede convertir la tabla a json:

```
open urls.json | get urls | each { |u| http get $u } | to json
```

Notar que el archivo `urls.json` es opcional, pues se puede realizar directamente:

```
{
  "urls": [
    "https://jsonplaceholder.typicode.com/posts/1",
    "https://jsonplaceholder.typicode.com/posts/2",
    "https://jsonplaceholder.typicode.com/posts/3"
  ]
} | get urls | each { |u| http get $u }
```

Tambien se puede realizar consultas `post` con el comando `http post`:

```
{
  "my_payload": {
    "title": "foo",
    "body": "bar",
    "userId": 1
  }
} | get my_payload | to json | http post https://jsonplaceholder.typicode.com/posts $in
```

`$in` indica la salida del comando anterior.

Se puede colocar todo esto en una *pipeline* donde se leen datos, se manipulan y luego se envian de vuelta a la API. En el siguiente ejemplo se descarga una publicacióm, se incrementa el `id`, y

se sube de vuelta al *endpoint*. En este ejemplo en particular, el *endpoint* de prueba devuelve una respuesta arbitraria:

```
open urls.json | get urls | first | http get $in | upsert id {|item| $item.id | inc}
| to json | http post https://jsonplaceholder.typicode.com/posts $in
```

En nushell, también se pueden escribir comandos en varias lineas. El ejemplo anterior queda:

```
open urls.json
| get urls
| first
| http get $in
| upsert id {|item|
   $item.id | inc
  }
| to json
| http post https://jsonplaceholder.typicode.com/posts $in
```

9.8. Trabajando con listas

Las listas son equivalentes a las columnas individuales de las tablas. Una lista es esencialmente una tabla de una columna (sin nombre de columna). Por lo tanto, cualquier comando que opere en una columna *también* opera en una lista. Por ejemplo, `where` se puede usar con listas:

```
[bell book candle] | where ($it =~ 'b')
```

9.8.1. Creando listas

Una lista es una colección ordenada de valores. Una lista se crea usando corchetes que rodean los valores separados por espacios, saltos de línea y/o comas. Por ejemplo, `[foo bar baz]` o `[foo, bar, baz]`.

> Las listas de Nushell son similares a las matrices JSON. El mismo `[ Ïtem1", Ïtem2", Ïtem3" ]` que representa una matriz JSON también se puede usar para crear una lista de Nushell.

9.8.2. Actualizando listas

Se usa `insert` para insertar valores en las listas a medida que fluyen a través de la *pipeline*, por ejemplo, insertamos el valor `10` en el medio de una lista:

```
[1, 2, 3, 4] | insert 2 10
```

También se puede usar `update` para reemplazar el segundo elemento con el valor `10`.

```
[1, 2, 3, 4] | update 1 10
```

9.8.3. Eliminando o Agregando Elementos de la Lista

Además de `insert` y `update`, también se tiene `prepend` y `append`. Estos permiten insertar al principio de una lista o al final de la lista, respectivamente.

Por ejemplo:

```
let colors = [yellow green]
let colors = ($colors | prepend red)
let colors = ($colors | append purple)
let colors = ($colors ++ ["blue"])
let colors = (["black"] ++ $colors)
$colors # [black red yellow green purple blue]
```

En caso de que se requiera eliminar elementos de la lista, hay muchas maneras. `skip` permite omitir las primeras filas de la entrada, mientras que `drop` permite omitir filas numeradas específicas del final de la lista.

```
let colors = [red yellow green purple]
let colors = ($colors | skip 1)
let colors = ($colors | drop 2)
$colors # [yellow]
```

También está `last` y `first` que permiten tomar valores desde el final o el principio de la lista, respectivamente.

```
let colors = [red yellow green purple black magenta]
let colors = ($colors | last 3)
$colors # [purple black magenta]
```

Y desde el principio de una lista,

```
let colors = [yellow green purple]
let colors = ($colors | first 2)
$colors # [yellow green]
```

9.8.3.1. Usando el Operador de Propagación

Para agregar una o más listas, opcionalmente con valores intercalados, se puede usar el operador de propagación (`...`)

```
let x = [1 2]

[
   ...$x
   3
   ...(4..7 | take 2)
]
```

9.8.4. Iterando sobre Listas

Para iterar sobre los elementos de una lista, se utiliza el comando `each` con un bloque de código Nu que especifica qué hacer con cada elemento. El parámetro de bloque (por ejemplo, `|elt|` en `{ |elt| print $elt }`) es el elemento de lista actual, pero el filtro `enumerate` se puede usar para proporcionar valores de `index` y `item` si es necesario. Por ejemplo:

```
let names = [Mark Tami Amanda Jeremy]

$names | each { |elt| $"Hello, ($elt)!" }

$names | enumerate | each { |elt| $"($elt.index + 1) - ($elt.item)" }
```

El comando `where` se puede usar para crear un subconjunto de una lista, filtrando eficazmente la lista según una condición.

El siguiente ejemplo obtiene todos los colores cuyos nombres terminan en e.

```
let colors = [red orange yellow green blue purple]
$colors | where ($it | str ends-with 'e')
# El bloque pasado a `where` debe evaluarse como un booleano.
# Esto imprime la lista [orange blue purple].
```

En este ejemplo, mantenemos solo los valores superiores a 7.

```
let scores = [7 10 8 6 7]
$scores | where $it > 7 # [10 8]
```

El comando `reduce` calcula un solo valor a partir de una lista. Utiliza un bloque que toma 2 parámetros: el elemento actual (convencionalmente llamado `elt`) y un acumulador (convencionalmente llamado `acc`). Para especificar un valor inicial para el acumulador, usa la opción `--fold` (`-f`). Para cambiar `elt` para que tenga valores de `index` y `item`, se usa el filtro `enumerate`. Por ejemplo:

```
let scores = [3 8 4]
$"total = ($scores | reduce { |elt, acc| $acc + $elt })" # total = 15

$"total = ($scores | math sum)" # enfoque más fácil, mismo resultado

$"product = ($scores | reduce --fold 1 { |elt, acc| $acc * $elt })" # product = 96

$scores | enumerate | reduce --fold 0 { |elt, acc| $acc + $elt.index * $elt.item } # 0*3 +
```

9.8.5. Accediendo a la Lista

Para acceder a un elemento de la lista en un índice dado, se usa la forma `$name.index` donde `$name` es una variable que contiene una lista.

Por ejemplo, se puede acceder al segundo elemento de la lista a continuación con `$names.1`.

```
let names = [Mark Tami Amanda Jeremy]
$names.1 # da Tami
```

Si el índice está en alguna variable `$index`, podemos usar el comando `get` para extraer el elemento de la lista.

```
let names = [Mark Tami Amanda Jeremy]
let index = 1
$names | get $index # da Tami
```

El comando `length` devuelve el número de elementos de una lista. Por ejemplo, `[red green blue] | length` imprime 3.

El comando `is-empty` determina si una cadena, lista o tabla está vacía. Se puede usar con listas de la siguiente manera:

```
let colors = [red green blue]
$colors | is-empty # false

let colors = []
$colors | is-empty # true
```

Los operadores `in` y `not-in` se utilizan para probar si un valor está en una lista. Por ejemplo:

```
let colors = [red green blue]
'blue' in $colors # true
'yellow' in $colors # false
'gold' not-in $colors # true
```

El comando `any` determina si algún elemento de una lista coincide con una condición dada. Por ejemplo:

```
let colors = [red green blue]
# £Algún nombre de color termina con "e"?
$colors | any {|elt| $elt | str ends-with "e" } # true

# £La longitud de algún nombre de color es inferior a 3?
$colors | any {|elt| ($elt | str length) < 3 } # false

let scores = [3 8 4]
# £Alguna puntuación es superior a 7?
$scores | any {|elt| $elt > 7 } # true

# £Alguna puntuación es impar?
$scores | any {|elt| $elt mod 2 == 1 } # true
```

El comando `all` determina si todos los elementos de una lista coinciden con una condición dada. Por ejemplo:

```
let colors = [red green blue]
# £Todos los nombres de color terminan con "e"?
$colors | all {|elt| $elt | str ends-with "e" } # false

# £La longitud de todos los nombres de color es mayor o igual a 3?
$colors | all {|elt| ($elt | str length) >= 3 } # true

let scores = [3 8 4]
# £Todas las puntuaciones son superiores a 7?
$scores | all {|elt| $elt > 7 } # false

# £Todas las puntuaciones son pares?
$scores | all {|elt| $elt mod 2 == 0 } # false
```

9.8.6. Convirtiendo la Lista

El comando `flatten` crea una nueva lista a partir de una lista existente agregando elementos de listas anidadas a la lista de nivel superior. Esto se puede llamar varias veces para aplanar listas anidadas a cualquier profundidad. Por ejemplo:

```
[1 [2 3] 4 [5 6]]
[1 [2 3] 4 [5 6]] | table -e
[1 [2 3] 4 [5 6]] | flatten # [1 2 3 4 5 6]

[[1 2] [3 [4 5 [6 7 8]]]]
[[1 2] [3 [4 5 [6 7 8]]]] | table -e
[[1 2] [3 [4 5 [6 7 8]]]] | flatten

[[1 2] [3 [4 5 [6 7 8]]]] | flatten | flatten | flatten # [1 2 3 4 5 6 7 8]
```

El comando `wrap` convierte una lista en una tabla. Cada valor de la lista se convertirá en una fila separada con una sola columna:

```
let zones = [UTC CET Europe/Moscow Asia/Yekaterinburg]

# Mostrar el reloj mundial para las zonas horarias seleccionadas
$zones
| wrap 'Zone'
| upsert Time {|row|
   (date now | date to-timezone $row.Zone | format date '%Y.%m.%d%H:%M')
  }
```

9.9. Trabajando con Registros (records)

Los registros son aproximadamente equivalentes a las filas individuales de una tabla. Esencialmente son una tabla de una fila. Por lo tanto, la mayoría de los comandos que operan en una fila de tabla *también* operan en un registro. Por ejemplo, `update` se puede usar con registros:

```
let my_record = {
   name: "Sam"
   age: 30
   }
$my_record | update age { $in + 1 }
```

9.9.1. Creando registros

Un registro es una colección de cero o más asignaciones de pares clave-valor. Es similar a un objeto JSON y se puede crear utilizando la misma sintaxis:

```
# Nushell
{ "apples": 543, "bananas": 411, "oranges": 0 }

 apples   543
 bananas  411
 oranges  0

# JSON
'{ "apples": 543, "bananas": 411, "oranges": 0 }' | from json
```

En Nushell, los pares clave-valor de un registro también se pueden separar utilizando espacios o saltos de línea.

Como los registros pueden tener muchos campos, de forma predeterminada, se muestran verticalmente en lugar de de izquierda a derecha. Para mostrar un registro de izquierda a derecha, de debe convertir a un nuon. Por ejemplo:

```
{
   name: "Sam"
   rank: 10
} | to nuon
```

9.9.2. Actualizando Registros

Al igual que con las listas, se pueden insertar valores en los registros. Por ejemplo:

```
{ "apples": 543, "bananas": 411, "oranges": 0 } | insert pears { 21 }
```

También se pueden actualizar valores:

```
{ "apples": 543, "bananas": 411, "oranges": 0 } | update oranges { 100 }
```

Para hacer una copia de un registro con nuevos campos, se puede:

- Usar el comando `merge`:

```
let first_record = { name: "Sam", rank: 10 }
$first_record | merge { title: "Mayor" }
```

- Usar el operador de propagación (`...`) para expandir el primer registro dentro de un nuevo registro:

```
let first_record = { name: "Sam", rank: 10 }
{
   ...$first_record
   title: "Mayor"
 }
```

9.9.3. Iterando sobre un Registro

Se puede iterar sobre los pares clave-valor de un registro transponiéndolo primero a una tabla:

```
{ "apples": 543, "bananas": 411, "oranges": 0 }
| transpose fruit count
| each {|f| $"We have ($f.count) ($f.fruit)" }
```

9.10. Navegando y Accediendo a Datos Estructurados

Dado el fuerte soporte de Nushell para datos estructurados, algunas de las tareas más comunes involucran navegar y acceder a esos datos.

9.10.1. Antecedentes

Para los ejemplos y descripciones a continuación, se debe tener en cuenta varias definiciones con respecto a los datos estructurados:

- ***Lista:*** Las listas contienen una serie de cero o más valores de cualquier tipo. Una lista con cero valores se conoce como una lista vacía.
- ***Registro:*** Los registros contienen cero o más pares de claves con nombre y su valor correspondiente. Los datos en el valor de un registro también pueden ser de cualquier tipo. Un registro con cero pares clave-valor se conoce como un registro vacío.
- ***Datos Anidados:*** Los valores contenidos en una lista, registro o tabla pueden ser de un tipo básico o datos estructurados en sí mismos. Esto significa que los datos se pueden anidar en múltiples niveles y en múltiples formas:
  - Los valores de la lista pueden contener tablas, registros e incluso otras listas
    Tabla: Las tablas son una lista de registros
  - Los valores de registro pueden contener tablas, listas y otros registros
    Esto significa que los registros de una tabla también pueden contener tablas anidadas, listas y otros registros

Debido a que una tabla es una lista de registros, cualquier comando o sintaxis que funcione en una lista también funcionará en una tabla. Lo contrario no es necesariamente el caso; hay algunos comandos y sintaxis que funcionan en tablas pero no en listas.

9.10.2. Rutas de Celda

Una ruta de celda (*cellpath*) es la forma principal de acceder a valores dentro de datos estructurados. Esta ruta se basa en un concepto similar al de una hoja de cálculo, donde las columnas tienen nombres y las filas tienen números. Los nombres e índices de las rutas de celda están separados por puntos.

9.10.2.1. Registros Para un registro, la ruta de celda especifica el nombre de una clave, que es una `string`.

Por ejemplo, para acceder a un Valor de Registro:

```
let my_record = {
    a: 5
    b: 42
  }

$my_record.b + 5
```

9.10.2.2. Listas Para una lista, la ruta de celda especifica la posición (índice) del valor en la lista. Este es un `int`:

Por ejemplo, para acceder a un Valor de Lista (recordar que los índices de lista comienzan en 0):

```
let scoobies_list = [ Velma Fred Daphne Shaggy Scooby ]

$scoobies_list.3
```

9.10.2.3. Tablas

- Para acceder a una columna, una ruta de celda usa el nombre de la columna, que es una `string`
- Para acceder a una fila, usa el número de índice de la fila, que es un `int`
- Para acceder a una sola celda, usa una combinación del nombre de la columna con el índice de la fila.

Los siguientes ejemplos usarán la siguiente tabla:

```
let data = [[date                        temps                                    condition
            [2022-02-01T14:30:00+05:00,  [ 38.24, 38.50, 37.99, 37.98, 39.10 ],  'summy'
            [2022-02-02T14:30:00+05:00,  [ 35.24, 35.94, 34.91, 35.24, 36.65 ],  'sunny'
            [2022-02-03T14:30:00+05:00,  [ 35.17, 36.67, 34.42, 35.76, 36.52 ],  'cloudy'
```

```
            [2022-02-04T14:30:00+05:00,  [ 39.24, 40.94, 39.21, 38.99, 38.80 ],  'rain'
          ]
```

```
$data
```

```
$data | table -e
```

Esto representa datos meteorológicos en forma de una tabla con tres columnas:

1. ***date***: Una `date` de Nushell para cada día
2. ***temps***: Una `list` de Nushell de 5 valores `float` que representan lecturas de temperatura en diferentes estaciones meteorológicas en el área
3. ***condition***: Una `string` de Nushell para la condición meteorológica de cada día para el área

9.10.2.3.1. Ejemplo - Acceder a una Fila de la Tabla (Registro) Acceder a los datos del segundo día como un registro:

```
> $data.1

 date       2 years ago

 temps       0  35.24
             1  35.94
             2  34.91
             3  35.24
             4  36.65

 condition  sunny
```

9.10.2.3.2. Ejemplo - Acceder a una Columna de la Tabla (Lista)

```
$data.condition
```

9.10.2.3.3. Ejemplo - Acceder a una Celda de la Tabla (Valor) La condición para el cuarto día:

```
$data.condition.3
```

```
$data | get condition | get 3
```

```
$data | get condition.3
```

9.10.3. Datos Anidados

Dado que los datos se pueden anidar, una ruta de celda puede contener referencias a múltiples nombres o índices.

9.10.3.1. Ejemplo - Accediendo a Datos de Tabla Anidados

Para obtener la temperatura en la segunda estación meteorológica el tercer día:

```
$data.temps.2.1
```

El primer índice `2` accede al tercer día, luego el siguiente índice `1` accede a la lectura de temperatura de la segunda estación meteorológica.

9.10.4. Usando `get` y `select`

Además de la sintaxis literal de la ruta de celda utilizada anteriormente, Nushell también proporciona varios comandos que utilizan rutas de celda. Los más importantes de estos son:

- `get` es equivalente a usar una ruta de celda literal pero con soporte para nombres de variables y expresiones. `get`, como los ejemplos de ruta de celda anteriores, devuelve el **valor** indicado por la ruta de celda.
- `select` es sutilmente, pero críticamente, diferente. Devuelve la **estructura de datos** especificada en sí misma, en lugar de solo su valor.
  - Usar `select` en una tabla devolverá una tabla de igual o menor tamaño
  - Usar `select` en una lista devolverá una lista de igual o menor tamaño
  - usar `select` en un registro devolverá un registro de igual o menor tamaño

Continuando con la tabla de muestra anterior:

9.10.4.1. Ejemplo - `get` vs. `select` una fila de la tabla

```
$data | get 1
```

```
$data | select 1
```

Observe que:

- `get` devuelve el mismo registro que el ejemplo `$data.1` anterior
- `select` devuelve una nueva tabla de una sola fila, incluidos los nombres de las columnas y los índices de las filas

Los índices de fila de la tabla resultante de `select` no son los mismos que los del original. La nueva tabla tiene su propio índice que comienza en 0.

Para obtener el índice original, puede usar el comando `enumerate`. Por ejemplo:

```
$data | enumerate | select 1
```

9.10.4.2. Ejemplo - `select` con múltiples filas y columnas

Debido a que `select` da como resultado una nueva tabla, es posible especificar varios nombres de columna, índices de fila o incluso ambos. Este ejemplo crea una nueva tabla que contiene las columnas de fecha y condición de la primera y segunda filas:

```
$data | select date condition 0 1
```

9.10.5. Nombres de Clave/Columna con espacios

Si un nombre de clave o un nombre de columna contiene espacios u otros caracteres que impiden que sea accesible como una cadena de palabra simple, entonces el nombre de la clave se puede especificar entre comillas.

Ejemplo:

```
let record_example = {
    "key x":12
    "key y":4
  }
$record_example."key x"

# o
$record_example | get "key x"
```

También se requieren comillas cuando un nombre de clave puede confundirse con un valor numérico.

Ejemplo:

```
let record_example = {
    "1": foo
    "2": baz
    "3": far
  }

$record_example."1"
```

No se debe confundir el nombre de la clave con un índice de fila en este caso. Aquí, al primer elemento se le *asigna* el nombre de clave `1` (una cadena). Si se convierte a una tabla usando el comando `transpose`, la clave `1` (`string`) estaría en el índice de fila `0` (un entero).

9.10.6. Manejo de Datos Faltantes

9.10.6.1. El Operador Opcional

De forma predeterminada, el acceso a la ruta de celda fallará si no puede acceder a la fila o columna solicitada. Para suprimir estos errores, puede agregar un `?` a un miembro de la ruta de celda para marcarlo como opcional:

9.10.6.1.1. Ejemplo - El Operador Opcional

Usando los datos de temperatura de arriba:

```
let cp: cell-path = $.temps?.1

$data | reject temps | get $cp
```

De forma predeterminada, las celdas faltantes se reemplazarán por `null` cuando se acceda a ellas a través del operador opcional.

9.10.6.2. Asignar un `default` para datos faltantes o `null`

El comando `default` se puede usar para aplicar un valor predeterminado al resultado de una columna faltante o nula.

```
let missing_value = [{a:1 b:2} {b:1}]
$missing_value

let with_default_value = ($missing_value | default 'n/a' a)
$with_default_value

$with_default_value.1.a
```

9.10.7. Otros comandos para acceder a datos estructurados

- `reject` es lo opuesto a `select`, eliminando las filas y columnas especificadas
- `range` especifica las filas de una lista o tabla para seleccionar usando un tipo `range`.

9.11. Trabajando con strings

Como con la mayoría de los lenguajes, los strings son una colección de 0 o más caracteres que representan texto. Esto puede incluir nombres de archivos, rutas de archivos, nombres de columnas, y mucho más. Las strings son tan comunes que Nushell ofrece múltiples formatos de string para que coincidan con cada caso de uso.

9.11.1. Formatos de String

Formato de string	Ejemplo	Escapes	Notas
String con comillas simples	`'[^\n]+'`	Ninguno	No puede contener comillas simples dentro del string
String con comillas dobles	`"The\nEnd"`	Escapes de barra invertida estilo C	Todas las barras invertidas literales deben tener escape
Strings raw	`r#'Raw string'#`	Ninguno	Puede incluir comillas simples

Formato de string	Ejemplo	Escapes	Notas
String de palabra simple	`ozymandias`	Ninguno	Solo puede contener caracteres de palabra; No se puede usar en la posición de comando
String con comillas invertidas	‘[^\n]+‘	Ninguno	String simple que puede incluir espacios en blanco. No puede contener comillas invertidas
Interpolación con comillas simples	`$'Captain ($name)'`	Ninguno	No puede contener ningún `'` o `()` no coincidentes
Interpolación con comillas dobles	`$Çaptain ($name)"`	Escapes de barra invertida estilo C	Todas las barras invertidas literales y `()` deben tener escape

9.11.2. Strings con Comillas Simples

El string más simple en Nushell es el string con comillas simples. Este string usa el carácter `'` para rodear algún texto. Por ejemplo:

```
'hello world'


'The
end'
```

Los strings con comillas simples no le hacen nada al texto que se les da, lo que las hace ideales para contener una amplia gama de datos de texto.

9.11.3. Strings con Comillas Dobles

Para strings más complejas, Nushell también ofrece strings con comillas dobles. Estas strings usan el carácter `"` para rodear el texto. También admiten la capacidad de escapar caracteres dentro del texto usando el carácter `\`.

Por ejemplo, se podría escribir un texto seguido de una nueva línea y luego otro texto, usando caracteres de escape y un string con comillas dobles:

```
"hello\nworld"
```

Los caracteres de escape permiten agregar rápidamente un carácter que de otro modo sería difícil de escribir.

Nushell actualmente admite los siguientes caracteres de escape:

- `\"` - carácter de comillas dobles
- `\'` - carácter de comillas simples
- `\\` - barra invertida
- `\/` - barra diagonal

- `\b` - retroceso
- `\f` - avance de formulario
- `\r` - retorno de carro
- `\n` - nueva línea (avance de línea)
- `\t` - tabulación
- `\u{X...}` - un solo carácter unicode, donde X son 1-6 dígitos hexadecimales (0-9, A-F)

9.11.4. Strings Raw

Los strings raw se comportan igual que las strings con comillas simples, excepto que las strings raw también pueden contener comillas simples. Esto es posible porque las strings raw están encerradas por un `r#'` inicial y un `'#` final.

```
r#'Raw strings can contain 'quoted' text.'#
```

Se pueden agregar símbolos `#` adicionales al principio y al final del string raw para encerrar uno menos que la misma cantidad de símbolos `#` junto a un símbolo `'` en el string. Esto se puede usar para anidar strings raw:

```
r###'r##'This is an example of a raw string.'##'###
```

9.11.5. Strings de Palabra Simple

Al igual que otros lenguajes de shell (pero a diferencia de la mayoría de los otros lenguajes de programación), las strings que constan de una sola palabra también se pueden escribir sin comillas:

```
print hello
print "hello"

[hello] | describe
```

Pero se debe tener cuidado: si usa una palabra simple directamente en la línea de comandos (es decir, no dentro de una estructura de datos o como un parámetro de comando) o dentro de paréntesis redondos `( )`, se interpretará como un comando externo:

```
hello
Error: nu::shell::external_command

  E External command failed
   [entry #5:1:1]
 1  hello
   ·
   ·    executable was not found

  help: program not found
```

Además, muchas palabras simples tienen un significado especial en nu, por lo que no se interpretarán como una string:

```
true | describe

[true] | describe

[trueX] | describe

trueX | describe
```

Entonces, si bien los strings simples son útiles para el uso informal de la línea de comandos, al programar más formalmente en nu, se recomienda usar comillas.

9.11.6. Strings con Comillas Invertidas

Las strings de palabras simples, por su naturaleza, no pueden incluir espacios ni comillas. Como alternativa, Nushell también incluye strings con comillas invertidas usando el carácter ‘. En la mayoría de los casos, estos deberían funcionar igual que una string de palabra simple.

Por ejemplo, al igual que con una palabra simple, una string entre comillas invertidas en la primera posición de una expresión se interpretará como un *comando* o *ruta*. Por ejemplo:

```
# Ejecutar el binario ls externo que se encuentra en la ruta
`ls`

# Subir un directorio
`..`
```

Las strings con comillas invertidas pueden ser útiles para combinar globs con archivos o directorios que incluyen espacios:

```
ls `./my dir/*`
```

9.11.7. Strings como comandos externos

Puede colocar el signo ^ delante de cualquier string (incluida una variable) para que Nushell ejecute la string como si fuera un comando externo:

```
^ls

^'ls'

let foo = 'ls'
^$foo
```

También se puede usar el comando `run-external` para este propósito, que proporciona indicadores y opciones adicionales.

9.11.8. Añadir y anteponer a strings

Hay varias formas de anteponer o añadir strings. Si se desea agregar algo al principio de cada string, se puede iterar:

```
['foo', 'bar'] | each {|s| '~/' ++ $s} # ~/foo, ~/bar
['foo', 'bar'] | each {|s| '~/' + $s} # ~/foo, ~/bar
```

También puede usar una expresión regular para reemplazar el principio o el final de una string:

```
['foo', 'bar'] | str replace -r '^' '~/' # ~/foo, ~/bar
['foo', 'bar'] | str replace -r '$' '~/' # foo~/, bar~/
```

Si se desea obtener un string a partir de una lista:

```
"hello" | append "world!" | str join " "

ls | get name | str join "/"
```

También se puede usar reduce:

```
1..10 | reduce -f "" {|elt, acc| $acc + ($elt | into string) + " + "}
```

Aunque en el caso de los strings, especialmente si no se tiene que operar en las strings, suele ser más fácil y correcto (observe el + adicional al final del ejemplo anterior) usar `str join`.

Finalmente, también se puede usar la interpolación de strings.

9.11.9. Interpolación de strings

Los casos de uso de strings más complejos también necesitan una nueva forma de string: interpolación de strings. Esta es una forma de construir texto a partir de texto sin formato y el resultado de ejecutar expresiones. La interpolación de strings combina los resultados, dándo una nueva string.

La interpolación de strings usa `$" "` y `$' '` como formas de encapsular texto interpolado.

Por ejemplo, si se tiene una variable llamada `$name` y queremos saludar al nombre de la persona contenida en esta variable:

```
let name = "Alice"

$"greetings, ($name)"

$"greetings, (ls | get name.2)"
```

Al encapsular expresiones en `()`, podemos ejecutarlas hasta su finalización y usar los resultados para ayudar a construir el string.

La interpolación de strings tiene una forma con comillas simples, `$' '`, y una con comillas dobles, `$" "`. Éstas corresponden a las strings con comillas simples y comillas dobles: la interpolación de strings con comillas simples no admite caracteres de escape, mientras que la interpolación de strings con comillas dobles sí.

A partir de la versión 0.61, las strings interpoladas admiten el escape de paréntesis, de modo que los caracteres `(` y `)` se pueden usar en una string sin que Nushell intente evaluar lo que aparece entre ellos:

```
$"2 + 2 is (2 + 2) \(adivinaste!)"
```

Los strings interpolados se pueden evaluar en tiempo de análisis, pero si incluyen valores cuyo formato depende de su configuración y su `config.nu` aún no se ha cargado, utilizarán la configuración predeterminada. Entonces, si se tiene algo como esto en su `config.nu`, `x` será `"2.0 KB"` incluso si su configuración dice que use `MB` para todos los tamaños de archivo (las fechas y horas utilizarán de manera similar la configuración predeterminada).

```
const x = $"(2kb)"
```

9.11.10. Dividir Strings

El comando `split row` crea una lista a partir de una string basándose en un delimitador.

```
"red,green,blue" | split row ","
```

El comando `split column` creará una tabla a partir de una string basándose en un delimitador. Esto aplica nombres de columna genéricos a la tabla.

```
"red,green,blue" | split column ","
```

Finalmente, el comando `split chars` dividirá una string en una lista de caracteres.

```
'aeiou' | split chars
```

9.11.11. El comando `str`

Muchas funciones de string son subcomandos del comando `str`. Puede obtener una lista completa usando `help str`.

Por ejemplo, puede buscar si una string contiene una subcadena (*substring*) en particular usando `str contains`:

```
"hello world" | str contains "o wo"
```

También se puede utilizar el operador `=~`.

9.11.11.1. Recortar Strings Se puede recortar los lados de una string con el comando `str trim`. De forma predeterminada, el comando `str trim` recorta los espacios en blanco de ambos lados de la string. Por ejemplo:

```
'       My   string   ' | str trim
```

Se puede especificar en qué lado se produce el recorte con las opciones `--right` y `--left`. (`-r` y `-l` son las opciones abreviadas respectivamente)

Para recortar un carácter específico, se utiliza `--char <Carácter>` o `-c <Carácter>` para especificar el carácter a recortar.

Ejemplo:

```
'=== Nu shell ===' | str trim -r -c '='
```

9.11.11.2. Substrings Los substrings son porciones de una string. Tienen un punto de inicio y un punto final

```
'Hello World!' | str index-of 'o'

'Hello World!' | str index-of 'r'

'Hello World!' | str substring 4..8
```

9.11.11.3. Relleno de Strings Con el comando `fill` puede agregar relleno a una string. El relleno agrega caracteres a la string hasta que tenga una cierta longitud. Por ejemplo:

```
'1234' | fill -a right -c '0' -w 10

'1234' | fill -a left -c '0' -w 10
```

9.11.11.4. Invertir Strings Esto se puede hacer fácilmente con el comando `str reverse`.

```
'Nushell' | str reverse

['Nushell' 'is' 'cool'] | str reverse
```

9.11.11.5. Análisis de Strings Con el comando `parse` se puede analizar una string en columnas. Por ejemplo:

```
'Nushell 0.80' | parse '{shell} {version}'

'where all data is structured!' | parse --regex '(?P<subject>\w*\s?\w+) is (?P<adjective>\w
```

Si se sabe que un string contiene datos separados por comas, separados por tabulaciones o separados por varios espacios, puede usar `from csv`, `from tsv` o `from ssv`, respectivamente:

```
"acronym,long\nAPL,A Programming Language" | from csv

"name  duration\nonestop.mid  4:06" | from ssv
```

```
"rank\tsuit\nJack\tSpades\nAce\tClubs" | from tsv
```

9.11.12. Comparación de Strings

Además de los operadores estándar `==` y `!=`, existen algunos operadores para comparar específicamente strings entre sí.

Para expresiones regulares:

```
'APL' =~ '^\w{0,3}$'
'FORTRAN' !~ '^\w{0,3}$'
```

Existen otros dos operadores para comparaciones más sencillas:

```
'JavaScript' starts-with 'Java'

'OCaml' ends-with 'Caml'
```

9.11.13. Convertir Strings

Hay varias formas de convertir strings a otros tipos y viceversa.

9.11.13.1. A string

1. Usando `into string`. Ej: `123 | into string`
2. Usando interpolación de strings. Ej: `$'(123)'`

9.11.13.2. De string

1. Usando `into <tipo>`. Ej: `'123' | into int`

9.11.14. Colorear Strings

Se pueden colorear strings con el comando `ansi`. Por ejemplo:

```
$'(ansi purple_bold)This text is a bold purple!(ansi reset)'
```

`ansi purple_bold` hace que el texto sea de color púrpura intenso `ansi reset` restablece el color al predeterminado. (Consejo: Siempre debe terminar las strings coloreadas con `ansi reset`)

9.12. El plugin polars de Nushell

Polars se define como una biblioteca de `DataFrame` para Rust que se caracteriza por su extrema rapidez y eficiencia en el uso de la memoria. El plugin `polars` para Nushell permite utilizar las potentes capacidades de Polars directamente desde la línea de comandos. Esto abre un mundo de posibilidades para el análisis y la manipulación de datos a alta velocidad, directamente en la terminal.

La filosofía de Polars se centra en aprovechar al máximo el hardware moderno, utilizando un motor de consulta paralelizado y optimizado para la ejecución en múltiples núcleos. Esto significa que se pueden procesar conjuntos de datos mucho más grandes que con las herramientas tradicionales de línea de comandos, y a menudo de forma mucho más rápida que con soluciones en memoria como Pandas.

9.12.1. Casos de uso

El plugin `polars` resulta ideal para:

- **Análisis exploratorio de datos (EDA)**: se pueden investigar y resumir rápidamente grandes conjuntos de datos.
- **Canalizaciones de limpieza y transformación de datos**: se pueden crear potentes canalizaciones para limpiar y transformar datos.
- **Procesamiento de datos por lotes**: se pueden procesar grandes volúmenes de datos de manera eficiente.
- **Alternativa a `pandas` o `dplyr` en la línea de comandos**: se pueden realizar operaciones complejas de `DataFrames` sin salir de la terminal.

9.12.2. Conceptos básicos

Los dos conceptos principales en Polars son el `DataFrame` y la `Serie`.

- `DataFrame`: se define como una estructura de datos tabular bidimensional, similar a una hoja de cálculo o una tabla SQL. Consta de filas y columnas, donde cada columna es una `Serie`.
- `Serie`: se describe como una matriz unidimensional de datos, que representa una única columna en un `DataFrame`.

9.12.2.1. Diferencia entre DataFrames de Polars y Tablas de Nushell Es importante entender la diferencia entre un `DataFrame` de Polars y una tabla de Nushell. Aunque a primera vista pueden parecer similares, se observan diferencias fundamentales en su diseño y uso:

- **Representación en Memoria:** Las tablas de Nushell se representan como una lista de filas, donde cada fila es una colección de pares clave-valor. Esto resulta ideal para la mayoría de las tareas de la línea de comandos. Los `DataFrames` de Polars, por otro lado, utilizan un formato columnar. Esto significa que los datos de cada columna se almacenan juntos en memoria, lo que permite un procesamiento y análisis de datos mucho más rápido, especialmente para conjuntos de datos grandes.
- **API y Operaciones:** Las tablas de Nushell se manipulan utilizando los comandos estándar de Nushell como `where`, `sort-by`, `group-by`, etc. Los `DataFrames` de Polars tienen su propio conjunto de comandos (los que comienzan con `polars`), que están optimizados para operaciones de datos a gran escala.
- **Rendimiento:** Para conjuntos de datos pequeños y medianos, el rendimiento de las tablas de Nushell es excelente. Sin embargo, para conjuntos de datos muy grandes, los `DataFrames`

de Polars ofrecen un rendimiento significativamente mayor debido a su diseño columnar y su capacidad para realizar operaciones en paralelo.

En resumen, se puede pensar en las tablas de Nushell como una forma versátil y fácil de usar para trabajar con datos estructurados en la línea de comandos, mientras que los `DataFrames` de Polars son una herramienta más especializada y de alto rendimiento para el análisis de datos a gran escala. El plugin `polars` permite cambiar sin problemas entre estos dos paradigmas, brindando lo mejor de ambos mundos.

9.12.3. Uso básico

Para utilizar el plugin `polars`, primero se deben cargar los datos en un `DataFrame`. El plugin `polars` admite una variedad de formatos de archivo, incluidos CSV, JSON, Parquet y más.

9.12.3.1. Carga de datos Para este ejemplo, se utilizará un conjunto de datos de ventas de videojuegos. Se pueden descargar los datos usando el siguiente comando:

```
http get https://raw.githubusercontent.com/rfordatascience/tidytuesday/master/data/2019/201
| save vgsales.csv
```

> Fuente: https://raw.githubusercontent.com/rfordatascience/tidytuesday/master/data/2019/2019-07-30/video_games.csv

Ahora, se puede cargar el archivo `vgsales.csv` en un DataFrame de Polars de la siguiente manera:

```
let df = polars open vgsales.csv
$df | explore
$df | describe
```

Esto crea un `DataFrame` de Polars que puede usarse para su posterior análisis.

> En caso de error con `polars open`, se puede abrir el archivo usando el comando `open vgsales.csv | polars into-df`.

9.12.3.2. Exploración de datos Una vez que se han cargado los datos en un `DataFrame`, se puede ver su esquema usando el comando `polars schema`. Esto mostrará los nombres de las columnas y sus tipos de datos.

```
$df | polars schema
```

Se observa que algunas variables aparecen de tipo `object`, que es el tipo de datos predeterminado para Polars cuando no se especifica un tipo de datos específico. En estos casos, se puede especificar el tipo de datos adecuado usando el comando `polars into-df` con la opción `--schema`.

```
let df = open vgsales.csv
  | polars into-df --schema {
      number: i64,
      game: str,
```

```
    release_date: str,
    price: str,
    owners: str,
    developer: str,
    publisher: str,
    average_playtime: str,
    median_playtime: str,
    metascore: str
  }
```

Cuando hay valores `NA`, se definen temporalmente las variables numéricas de tipo `str`.

9.12.3.3. Conversión de tipos de datos Para poder abrir el archivo, se tuvo que usar el tipo `str` para las columnas `number`, `price`, `owners`, `average_playtime`, `median_playtime`, y `metascore`.

Para convertir estas columnas al tipo apropiado se utilizan:

```
let df = $df
  | collect
  | polars with-column [
    (polars col price
        | polars str-replace-all --pattern "Free" --replace "0"
        | polars cast f64
    ),
    (polars col average_playtime | polars cast i64),
    (polars col median_playtime | polars cast i64),
    (polars col metascore | polars cast i64)
  ]

$df | polars schema
$df | polars summary | polars collect
$df | polars summary | polars select descriptor "price (f64)" "average_playtime (i64)" "me

$df | polars collect | explore
```

En este proceso:

- `collect`: Recolecta un flujo (*stream*) en un valor.
- `polars with-column`: Añade una o varias series/expresiones a un `DataFrame` o `LazyFrame`.
- `polars col`: Crea una expresión que referencia una columna existente (p. ej. `polars col price`).
- `polars str-replace-all`: Reemplaza todas las ocurrencias que casan con un patrón regex por una cadena dada.

- `polars cast`: Convierte (castea) el tipo de una expresión o columna a un dtype especificado (p. ej. `f64`, `i64`).

Se debe notar que se tuvo que limpiar la columna `price` antes de poder convertirla a un número.

La columna `price` original contenía la palabra **Free** (Gratis) en algunas filas. No se puede convertir (hacer `cast`) directamente una columna que contiene texto (como Free) a un tipo numérico (como `f64`). Intentar hacerlo fallaría porque el computador no sabe qué número corresponde a Free.

Para arreglar esto, se usa un proceso de dos pasos dentro del comando `with-column`:

1. `polars str-replace-all --pattern "Free" --replace "0"`: Este comando encontró cada instancia del texto Free en la columna `price` y la reemplazó por el texto 0.
2. `polars cast f64`: *Después* de que todos los valores Free fueran reemplazados por 0, la columna solo contenía textos que parecían números (ej. 1.99, 59.99, y ahora 0). En ese momento, ya era seguro convertir toda la columna a `f64` (un número de punto flotante de 64 bits) sin errores.

9.12.4. Manipulación de datos

El plugin `polars` proporciona una amplia gama de comandos para manipular `DataFrames`.

9.12.4.1. Selección de columnas Se pueden seleccionar columnas de un `DataFrame` usando el comando `polars select`. Por ejemplo, para seleccionar las columnas `game`, `price`, y `metascore` del `DataFrame` de ventas de videojuegos, se usaría el siguiente comando:

```
$df | polars select game price metascore
```

Se observa que se obtiene lo siguiente:

```
plan            SELECT [col("game"), col("price"), col("metascore")]
                  DF ["number", "game", "release_date", "price", ...]; PROJECT */10 COLUM
optimized_plan  DF ["number", "game", "release_date", "price", ...]; PROJECT["game", "pric
```

Este resultado es el **plan de consulta** de Polars. Es una mirada interna a cómo el motor de Polars procesará la solicitud. Se divide en dos partes:

- **`plan` (Plan Lógico):** Esta es la traducción directa del comando a una serie de operaciones. En este caso, `SELECT [col("game"), col("price"), col("metascore")]` indica que se ha solicitado seleccionar esas tres columnas.
- **`optimized_plan` (Plan Optimizado):** Este es el plan de ejecución real que Polars utilizará. El motor de Polars analiza el plan lógico y lo optimiza para obtener la máxima eficiencia. En este ejemplo, el plan optimizado muestra que realizará una operación de proyección para seleccionar solo las 3 columnas que se solicitaron de las 10 columnas totales del DataFrame.

Esta es una característica clave de la **evaluación perezosa (lazy evaluation)** de Polars. En lugar de ejecutar cada comando de inmediato, se construye un plan de todas las operaciones, se optimiza y luego se ejecuta de la manera más eficiente posible. Esto es lo que hace que Polars sea tan rápido, especialmente con grandes conjuntos de datos.

Para ver el resultado, se debe terminar la *pipeline* con `polars collect`. Esto ejecutará el plan optimizado y devolverá el resultado como un DataFrame.

```
$df | polars select game price metascore | polars collect
```

9.12.4.2. Filtrado de filas Se pueden filtrar filas de un `DataFrame` usando el comando `polars filter`. Por ejemplo, para filtrar el `DataFrame` de ventas de videojuegos para incluir solo los juegos cuyo nombre de desarrollador (*developer*) contiene la palabra Action, se usaría el siguiente comando:

```
#búsqueda Insensible a Mayúsculas/Minúsculas
let filter_condition = polars col developer | polars contains "(?i)Action"
$df | polars filter $filter_condition | polars collect

#búsqueda Sensible a Mayúsculas/Minúsculas
let filter_condition = polars col developer | polars contains "Action"
$df | polars filter $filter_condition | polars collect

#eliminar na in price
let filter_condition = polars col price | polars is-not-null
$df | polars filter $filter_condition | polars collect
```

Por otro lado, para eliminar todas las filas con valores nulos en todas las columnas, se usaría el siguiente comando:

```
let df_filtered = $df | polars drop-nulls
$df_filtered | polars collect
```

9.12.4.3. Ordenación de datos Se puede ordenar un `DataFrame` usando el comando `polars sort-by`. Por ejemplo, para ordenar el `DataFrame` de ventas de videojuegos por la columna `price` en orden descendente, se usaría el siguiente comando:

```
$df_filtered | polars sort-by price --reverse [true]
```

9.12.4.4. Agrupación y agregación Se pueden agrupar y agregar datos en un `DataFrame` usando los comandos `group-by` y `agg`. Por ejemplo, para agrupar el `DataFrame` de ventas de videojuegos por la columna `developer` y calcular el número total de juegos por cada desarrollador, se usaría el siguiente comando:

```
$df_filtered
| polars group-by developer
```

```
| polars agg (polars col game | polars count)
| polars sort-by game --reverse [true]
| polars collect
```

- Para estadísticas por `developer`:

```
$df_filtered
 | polars group-by developer
 | polars agg [
     (polars col price | polars mean | polars as "price_mean"),
     (polars col game  | polars count | polars as "n_games")
   ]
 | polars sort-by developer
 | polars collect
```

- Para estadísticas por año de `release_date`:

```
#se mira la columna release_date
$df_filtered | polars select release_date | polars collect

#primero se extrae año, luego se agrupa
let data_by_year = $df_filtered
| polars with-column [
    (polars col release_date
       | polars str-replace-all --pattern ".*,\\s*(\\d{4})$" --replace "$1" # extrae el año
       | polars str-replace-all --pattern "^\\s+|\\s+$" --replace ""        # trim
       | polars cast i64
       | polars as "year")
   ]
 | polars group-by year
 | polars agg [
     (polars col price | polars mean | polars as "price_mean"),
     (polars col game  | polars count | polars as "n_games")
   ]
 | polars sort-by year
 | polars collect

#se visualiza
$data_by_year
$data_by_year | describe
$data_by_year | polars into-nu #tabla nu
$data_by_year | polars into-nu | describe
```

9.12.5. Guardado y formatos

Para guardar un `DataFrame` en un archivo, se usaría el comando `polars save`. Por ejemplo, para guardar el `DataFrame` de ventas de videojuegos filtrado en un archivo CSV, se usaría el siguiente comando:

```
$df_filtered | polars save vgsales_filtered.parquet
$df_filtered | polars save vgsales_filtered.csv
$data_by_year | polars save data_by_year.csv
```

Si lo anterior no funciona, en algunos casos es necesario entregar la ruta completa del archivo. Por ejemplo:

```
$df_filtered | polars save /path/to/vgsales_filtered.csv
$df_filtered | polars save ($env.PWD | path join vgsales_filtered.csv)

$df_filtered | polars save /home/kira/Dropbox/Documentos/Clases/UBB/DataScienceAtTheCommanc

$data_by_year | polars save /home/kira/Dropbox/Documentos/Clases/UBB/DataScienceAtTheCommar
```

9.12.6. Pivoteo de columnas

El comando `polars pivot` en Nushell es una operación de transformación de datos que convierte un DataFrame del formato largo al formato ancho, similar a la función PIVOT en SQL o la función `pivot_wider` en R. Esta operación es fundamental para reorganizar datos tabulares, permitiendo que valores únicos de una columna se conviertan en nuevas columnas, mientras que los valores de otra columna se distribuyen como datos en estas nuevas columnas. Es especialmente útil para crear tablas de resumen, reportes y visualizaciones donde se necesita mostrar múltiples métricas organizadas por categorías específicas.

Dentro de sus opciones principales se encuentran:

- **`--index (-i)`**: Define las columnas que servirán como identificadores únicos de las filas en el resultado. Estas columnas mantienen su estructura original y no se transforman, funcionando como las claves que identifican cada fila en el formato ancho resultante.
- **`--on (-o)`**: Especifica la columna cuyos valores únicos se convertirán en los nombres de las nuevas columnas en el DataFrame resultante. Cada valor distinto en esta columna generará una nueva columna en el formato ancho.
- **`--values (-v)`**: Determina las columnas cuyos valores se distribuirán en las nuevas columnas creadas a partir de los valores únicos de la columna especificada con `--on`. Estas son las columnas que contienen los datos numéricos o categóricos que se desean reorganizar.

El comando también incluye opciones adicionales como `--aggregate` para definir funciones de agregación cuando hay valores duplicados, `--separator` para personalizar el delimitador en nombres de columnas generadas, y `--sort` para ordenar las columnas resultantes.

Para ilustración, se considera el siguiente ejemplo.

Primero, se preparan los datos para el pivoteo:

- Se repite el procesamiento anterior a `release_date`.
- Se extraen columnas `developer`, `price`, `year`.
- Se agrupa por `developer` y `year`.
- Se calcula promedio de `price` de acuerdo al agrupamiento.

```
let df_aggregated = $df_filtered
    | polars with-column [
        (polars col release_date
        | polars str-replace-all --pattern ".*,\\s*(\\d{4})$" --replace "$1"
        | polars str-replace-all --pattern "^\\s+|\\s+$" --replace ""
        | polars cast i64
        | polars as "year"
        )
    ]
    | polars select [
        (polars col developer)
        (polars col price)
        (polars col year)
    ]
    | polars group-by [developer year]
    | polars agg [(polars col price | polars mean | polars as price_mean)]

$df_aggregated | polars collect
$df_aggregated | polars sort-by year | polars collect
$df_aggregated | polars sort-by developer | polars collect
```

Se hace el pivoteo:

```
let $df_pivoted = $df_aggregated
| polars pivot --index [developer] --on [year] --values [price_mean] --sort
| polars sort-by developer

$df_pivoted | polars collect
$df_pivoted | polars into-nu | explore
```

9.12.7. Buenas prácticas y recomendaciones

- Siempre se deben nombrar explícitamente las columnas creadas con `polars as` dentro de expresiones complejas.
- Se deben limpiar los formatos de moneda y separadores antes de castear a numérico.
- Se debe llamar `collect` o `save` al final de una *pipeline*.
- Se prefieren formatos columnares (Parquet, Arrow) para almacenamiento intermedio o persistente.

- Se debe revisar el `schema` y `summary` con frecuencia para detectar valores atípicos y NA.
- Para desarrollos iterativos, usar `polars collect | polars into-nu` permite inspeccionar resultados en formato de tabla de Nushell y combinar con otras utilidades de la shell.

9.12.8. Conclusión

El plugin `polars` para Nushell combina la expresividad y conveniencia de la shell con el rendimiento de Polars. Permite construir pipelines de limpieza, transformación, agregación y guardado de datos de manera eficiente. Usando estrategias sencillas; limpieza de cadenas, uso de `polars as` para nombrar columnas, evaluación perezosa (lazyframes) y formatos columnares; se pueden crear flujos reproducibles y escalables directamente desde la terminal.

9.13. Programación en Nushell

Esta sección entra en más detalle de Nushell como un lenguaje de programación.

Al igual que la mayoría de los lenguajes de programación permiten definir funciones, Nushell usa comandos personalizados (*custom commands*) para este propósito.

Los *alias* de Nushell funcionan de manera similar a los alias de otras shell y son parte del lenguaje de programación, no solo una característica del shell.

Las operaciones comunes pueden, como la suma o la búsqueda regex, hacerse con *operadores*. No todas las operaciones son compatibles con todos los tipos de datos y Nushell se asegurará de hacerlo saber.

Se pueden almacenar resultados intermedios en *variables*. Las variables pueden ser inmutables, mutables o una constante de tiempo de análisis.

Los *scripts* son la forma más simple de organización del código: simplemente se escribe el código en un archivo y se ejecuta. Sin embargo, también se pueden ejecutar scripts como programas independientes con firmas de línea de comandos utilizando el comando `main`.

9.13.1. Operadores

Nushell soporta los siguientes operadores para operaciones comunes de matemáticas, lógica y cadenas:

Operador	Descripción
`+`	sumar
`-`	restar
`*`	multiplicar
`/`	dividir
`//`	división de piso
`mod`	módulo
`**`	exponenciación (potencia)

Operador	Descripción
`==`	igual
`!=`	no igual
`<`	menor que
`<=`	menor o igual que
`>`	mayor que
`>=`	mayor o igual que
`=~`	coincidencia de expresiones regulares / cadena contiene otra
`!~`	coincidencia inversa de expresiones regulares / cadena *no* contiene otra
`in`	valor en lista
`not-in`	valor no en lista
`not`	no lógico
`and`	y dos expresiones booleanas (cortocircuitos)
`or`	o dos expresiones booleanas (cortocircuitos)
`xor`	o exclusivo dos expresiones booleanas
`bit-or`	o bit a bit
`bit-xor`	xor bit a bit
`bit-and`	y bit a bit
`bit-shl`	desplazamiento bit a bit a la izquierda
`bit-shr`	desplazamiento bit a bit a la derecha
`starts-with`	cadena comienza con
`ends-with`	cadena termina con
`++`	agregar listas

Los paréntesis se pueden usar para agrupar para especificar el orden de evaluación o para llamar a comandos y usar los resultados en una expresión

9.13.1.1. Orden de Operaciones Para entender la precedencia de las operaciones, se puede ejecutar el comando: `help operators | sort-by precedence -r`.

Presentado en orden descendente de precedencia, el artículo detalla las operaciones de la siguiente manera:

- Paréntesis (`()`)
- Exponenciación/Potencia (`**`)
- Multiplicar (`*`), Dividir (`/`), División Entera/Suelo (`//`), y Módulo (`mod`)
- Sumar (`+`) y Restar (`-`)
- Desplazamiento de bits (`bit-shl`, `bit-shr`)
- Operaciones de comparación (`==`, `!=`, `<`, `>`, `<=`, `>=`), pruebas de pertenencia (`in`, `not-in`, `starts-with`, `ends-with`), coincidencia de expresiones regulares (`=~`, `!~`), y agregar a la lista (`++`)
- Bit a bit and (`bit-and`)

- Bit a bit xor (`bit-xor`)
- Bit a bit or (`bit-or`)
- Lógico and (`and`)
- Lógico xor (`xor`)
- Lógico or (`or`)
- Operaciones de asignación
- Lógico not (`not`)

Por ejemplo:

```
3 * (1 + 2)
```

9.13.1.2. Tipos No todos las operaciones tienen sentido para todos los tipos de datos. Si se intenta realizar una operación en tipos de datos no compatibles, se encontrará con un mensaje de error que debería explicar qué salió mal:

```
> "spam" - 1
Error: nu::parser::unsupported_operation (link)

  Tipos no coincidentes para la operación.
   [entry #49:1:1]
 1  "spam" - 1
   ů
   ů         int
   ů        no admite estos valores.
   ů     string

  help: Cambie string o int para que sean los tipos correctos e intente nuevamente.
```

Las reglas a veces pueden parecer un poco estrictas, pero por otro lado, debería haber menos efectos secundarios inesperados

9.13.1.3. Expresión Regular / Operadores string-contains Los operadores `=~` y `!~` proporcionan una forma conveniente de evaluar expresiones regulares. No se necesita saber de expresiones regulares para usarlos; también son una forma fácil de comprobar si una cadena contiene otra.

- `string =~ pattern` devuelve **true** si `string` contiene una coincidencia para `pattern`, y **false** en caso contrario.
- `string !~ pattern` devuelve **false** si `string` contiene una coincidencia para `pattern`, y **true** en caso contrario.

Por ejemplo:

```
"foobarbaz" =~ "bar" # devuelve true
"foobarbaz" !~ "bar" # devuelve false
```

```
ls | where name =~ ^nu # devuelve todos los archivos cuyos nombres comienzan con "nu"
```

9.13.1.4. Sensibilidad a Mayúsculas y Minúsculas Los operadores suelen ser sensibles a mayúsculas y minúsculas cuando operan con cadenas. Hay algunas maneras de trabajar sin distinción entre mayúsculas y minúsculas:

1. En los operadores de expresiones regulares, especifique el modificador de modo insensible a mayúsculas y minúsculas `(?i)`:

```
"FOO" =~ "foo" # devuelve falso
"FOO" =~ "(?i)foo" # devuelve verdadero
```

2. Utilice la opción `--ignore-case` del comando `str contains`:

```
"FOO" | str contains --ignore-case "foo"
```

3. Convierta las cadenas a minúsculas con `str downcase` antes de comparar:

```
("FOO" | str downcase) == ("Foo" | str downcase)
```

9.13.1.5. Operador de propagación Nushell tiene un operador de propagación (`...`) para desempaquetar listas y registros. Se puede expandir listas o registros en lugares donde se esperan múltiples valores o pares clave-valor.

Hay tres lugares donde puede usar el operador de propagación:

- En literales de lista.
- En literales de registro.
- En llamadas de comando.

9.13.1.5.1. En literales de Lista Supongamos que se tienen varias listas que desea concatenar, pero también quiere intercalar algunos valores individuales. Esto se puede hacer con `append` y `prepend`, pero la propagación el operador puede permitir hacerlo más fácilmente.

```
let dogs = [Spot, Teddy, Tommy]
let cats = ["Mr. Humphrey Montgomery", Kitten]
[
    ...$dogs
    Polly
    ...($cats | each { |elt| $"($elt) \(cat\)" })
    ...[Porky Bessie]
    ...Nemo
]
```

El siguiente código es una versión equivalente usando `append`:

```
$dogs
| append Polly
```

```
|append ($cats | each { |elt| $"($elt) \(cat\)" })
| append [Porky Bessie]
| append ...Nemo
```

Considerar que cada llamada a `append` da como resultado la creación de una nueva lista, lo que significa que en este segundo ejemplo, se crean 3 listas intermedias innecesarias. Este no es el caso con el operador de propagación, por lo que puede haber beneficios de rendimiento (muy menores) al usar `...` si se está uniendo muchas listas grandes juntas, una y otra vez.

Puede haber notado que el último elemento de la lista resultante anterior es `"...Nemo"`. Esto se debe a que dentro de los literales de lista, solo se puede usar para difundir listas, no cadenas. Como tal, dentro de los literales de lista, solo se puede usar antes de variables (`...$foo`), subexpresiones (`...(foo)`), y literales de lista (`...[foo]`).

El `...` tampoco se reconocerá como el operador de propagación si hay algún espacio en blanco entre él y la siguiente expresión:

```
[ ... [] ]
```

Esto es principalmente para que `...` no se confunda con el operador de propagación en comandos como `mv ... $dir`.

9.13.1.5.2. En literales de Registro Digamos que se tiene un registro con alguna información de configuración y desea agregar más campos a este registro:

```
let config = { path: /tmp, limit: 5 }
```

Se puede crear un nuevo registro con todos los campos de `$config` y algunas nuevas adiciones usando la propagación operador. Se puede usar la propagación de múltiples registros dentro de un solo literal de registro.

```
{
    ...$config,
    users: [alice bob],
    ...{ url: example.com },
    ...(sys mem)
}
```

De manera similar a las listas, dentro de los literales de registro, el operador de propagación solo se puede usar antes de las variables (`...$foo`), subexpresiones (`...(foo)`), y literales de registro (`...{foo:bar}`). Aquí también, no es necesario que haya espacio en blanco entre el `...` y la siguiente expresión para que se reconozca como el operador de propagación.

9.13.1.5.3. En llamadas de Comando También se pueden propagar argumentos a un comando, siempre que tenga un parámetro `rest` o sea un comando externo.

Aquí hay un ejemplo de comando personalizado que tiene un parámetro `rest`:

```
def foo [ --flag req opt? ...args ] { [$flag, $req, $opt, $args] | to nuon }
```

Tiene una opción (`--flag`), un parámetro posicional requerido (`req`), un parámetro posicional opcional (`opt?`), y el parámetro rest (`args`).

Si se tiene una lista de argumentos para pasar a `args`, se puede propagar de la misma manera que propagaría una lista dentro de un literal de lista. Se aplican las mismas reglas: el operador de propagación solo se reconoce antes de variables, subexpresiones y literales de lista, y no se permiten espacios en blanco en el medio.

```
foo "bar" "baz" ...[1 2 3] # Con ..., los números se tratan como argumentos separados
[false, bar, baz, [1, 2, 3]]
foo "bar" "baz" [1 2 3] # Sin ..., [1 2 3] se trata como un solo argumento
[false, bar, baz, [[1, 2, 3]]]
```

Una forma más útil de usar el operador de propagación es si tiene otro comando con un parámetro `rest` y desea que reenvíe sus argumentos a `foo`:

```
def bar [ ...args ] { foo --flag "bar" "baz" ...$args }
bar 1 2 3
[true, bar, baz, [1, 2, 3]]
```

Puede propagar varias listas en una sola llamada y también intercalar argumentos individuales:

```
foo "bar" "baz" 1 ...[2 3] 4 5 ...(6..9 | take 2) last
[false, bar, baz, [1, 2, 3, 4, 5, 6, 7, last]]
```

Las opciones/argumentos con nombre pueden ir después de un argumento de propagación, al igual que pueden ir después de argumentos `rest` regulares:

```
foo "bar" "baz" 1 ...[2 3] --flag 4
[true, bar, baz, [1, 2, 3, 4]]
```

Si un argumento de propagación viene antes de un parámetro posicional opcional, ese parámetro opcional se trata como omitido:

```
foo "bar" ...[1 2] "not opt"
# El nulo significa que no se dio ningún argumento para opt
[false, bar, null, [1, 2, "not opt"]]
```

9.13.2. Variables

Los valores de Nushell se pueden asignar a variables con nombre usando las palabras clave `let`, `const` o `mut`. Después de crear una variable, se puede referir a ella usando `$` seguido de su nombre.

9.13.2.1. Tipos de Variables

9.13.2.1.1. Variables Inmutables Una variable inmutable no puede cambiar su valor después de la declaración. Se declaran usando la palabra clave `let`,

```
let val = 42

$val

$val = 100
```

```
Error: nu::shell::assignment_requires_mutable_variable

  × Assignment to an immutable variable.
   [entry #10:1:1]
 1  $val = 100
   ·
   ·    needs to be a mutable variable
```

Sin embargo, las variables inmutables pueden ser sombreadas. Sombrear significa que se vuelven a declarar y su valor inicial ya no se puede usar dentro del mismo alcance.

```
let val = 42                    # declare a variable
do { let val = 101;  $val }     # in an inner scope, shadow the variable: 101
$val                            # in the outer scope the variable remains unchanged: 42
let val = $val + 1              # now, in the outer scope, shadow the original variable
$val                            # in the outer scope, the variable is now shadowed,
                                # and its original value is no longer available: 43
```

9.13.2.1.2. Variables Mutables Una variable mutable puede cambiar su valor por asignación. Estas se declaran usando la palabra clave `mut`.

```
mut val = 42
$val += 27
$val
```

Hay un par de operadores de asignación usados con variables mutables

Operador	Descripción
`=`	Asigna un nuevo valor a la variable
`+=`	Suma un valor a la variable y hace que la suma sea su nuevo valor
`-=`	Resta un valor de la variable y hace que la diferencia sea su nuevo valor
`*=`	Multiplica la variable por un valor y hace que el producto sea su nuevo valor
`/=`	Divide la variable por un valor y hace que el cociente sea su nuevo valor
`++=`	Agrega una lista o un valor a una variable

Nota:

1. `+=`, `-=`, `*=` y `/=` solo son válidos en los contextos donde se espera que sus operaciones raíz funcionen. Por ejemplo, `+=` usa la suma, por lo que no se puede usar para contextos donde la suma normalmente fallaría
2. `++=` requiere que la variable **o** el argumento sea una lista.

Más sobre Mutabilidad

Los Cierres y las `def` anidadas no pueden capturar variables mutables de su entorno. Por ejemplo

```
# método ingenuo para contar el número de elementos en una lista
mut x = 0

[1 2 3] | each { $x += 1 }   # error: $x es capturado en un cierre
```

Para usar variables mutables para tal comportamiento, se recomienda usar los bucles.

9.13.2.1.3. Variables Constantes Una variable constante es una variable inmutable que puede ser completamente evaluada en tiempo de análisis. Estas son útiles con comandos que necesitan saber el valor de un argumento en tiempo de análisis, como `source`, `use` y `plugin use`. Se declaran usando la palabra clave `const`:

```
const script_file = 'path/to/script.nu'
source $script_file
```

9.13.2.2. Escogiendo entre variables mutables e inmutables Se debe tratar de usar variables inmutables para la mayoría de los casos de uso.

Muchos, si no la mayoría, de los casos de uso de variables mutables en Nushell tienen una solución funcional que:

- Solo usa variables inmutables, y como resultado
- Tiene mejor rendimiento
- Soporta streaming
- Puede soportar características adicionales como `par-each`

Por ejemplo, los contadores de bucles son un patrón común para las variables mutables y están integrados en la mayoría de los comandos de iteración. Por ejemplo, se puede obtener cada elemento y el índice de cada elemento usando `each` con `enumerate`:

```
ls
| enumerate
| each { |elt|
    $"Item #($elt.index) is size ($elt.item.size)"
}
```

También se puede usar el comando `reduce` para trabajar de la misma manera que podrías mutar una variable en un bucle. Por ejemplo, si se quisiera encontrar la cadena más grande en una lista de cadenas, se podría hacer:

```
[one, two, three, four, five, six] | reduce {|current_item, max|
    if ($current_item | str length) > ($max | str length) {
        $current_item
    } else {
        $max
    }
  }
```

Mientras que `reduce` procesa listas, el comando `generate` se puede usar con fuentes arbitrarias como API REST externas, también sin requerir variables mutables. A continuación, un ejemplo que recupera datos meteorológicos locales cada hora y genera una lista continua a partir de esos datos. El comando `each` se puede usar para consumir cada nuevo elemento de la lista a medida que esté disponible.

```
generate khot {|weather_station|
  let res = try {
    http get -ef $'https://api.weather.gov/stations/($weather_station)/observations/latest
  } catch {
    null
  }
  sleep 1hr
  match $res {
    null => {
      next: $weather_station
    }
    _ => {
      out: ($res.body? | default '' | from json)
      next: $weather_station
    }
  }
}
| each {|weather_report|
    {
      time: ($weather_report.properties.timestamp | into datetime)
      temp: $weather_report.properties.temperature.value
    }
}
```

9.13.2.2.1. Consideraciones de Rendimiento Usar comandos de filtro con variables inmutables es a menudo mucho más eficiente que las variables mutables con sentencias tradicionales de

control de flujo como `for` y `while`. Por ejemplo:

- Usando una sentencia `for` para crear una lista de 50,000 números aleatorios:

```
timeit {
  mut randoms = []
  for _ in 1..50_000 {
    $randoms = ($randoms | append (random int))
  }
}
```

Resultado: 1min 4sec 191ms 135ţs 90ns

- Usando `each` para hacer lo mismo:

```
timeit {
  let randoms = (1..50_000 | each {random int})
}
```

Resultado: 19ms 314ţs 205ns

- Usando `each` con 10,000,000 iteraciones:

```
timeit {
  let randoms = (1..10_000_000 | each {random int})
}
```

Resultado: 4sec 233ms 865ţs 238ns

Como con muchos filtros, la sentencia `each` también transmite sus resultados, lo que significa que la siguiente etapa de la tubería puede continuar procesando sin esperar a que los resultados se recopilen en una variable.

Para tareas que pueden ser optimizadas por paralelización, como se mencionó anteriormente, `par-each` puede tener ganancias de rendimiento aún más drásticas.

9.13.2.3. Nombres de Variables Los nombres de variables en Nushell vienen con algunas restricciones en cuanto a los caracteres que pueden contener. En particular, no pueden contener estos caracteres:

```
.  [  (  {  +  -  *  ^  /  =  !  <  >  &  |
```

Es común que algunos scripts declaren variables que comienzan con `$`. Esto está permitido, y es equivalente a que el `$` no esté allí en absoluto.

```
> let $var = 42ls
# identical to `let var = 42`
```

9.13.3. Comandos personalizados

La capacidad de Nu para componer *pipelines* largos permite mucho control sobre los datos y sistema, pero tiene el precio de escribir mucho. Idealmente, se podrían guardar las *pipelines* bien elaborados para usarlos una y otra vez.

Aquí es donde entran los comandos personalizados.

Un ejemplo de definición de un comando personalizado:

```
def greet [name:string] {
  ['hello' $name]
}
```

En esta definición, se define el comando `greet`, que toma un solo parámetro `name`. Siguiendo a este parámetro está el bloque que representa lo que sucederá cuando se ejecute el comando personalizado. Cuando se llama, el comando personalizado establecerá el valor pasado para `name` como la variable `$name`, que estará disponible para el bloque.

Para ejecutar lo anterior, podemos llamarlo como llamaríamos a los comandos integrados:

```
greet "world"
```

> El valor producido por la última línea de un comando se convierte en el valor devuelto del comando. En este caso, se devuelve una lista que contiene la cadena `'hello'` y `$name`. Para evitar esto, se puede colocar `null` (o el comando `ignore` al final de la *pipeline*, así: `['hello' $name] | null`. También se debe tener en cuenta que la mayoría de los comandos del sistema de archivos, como `save` o `cd`, siempre generan `null`.
>
> Si se quiere generar una sola cadena de texto, se puede usa la sintaxis de interpolación de cadenas para incrustar `$name` en ella:

```
def greet [name:string] {
  $"hello ($name)"
}

greet nushell
```

9.13.4. Nombres de Comandos

En Nushell, un nombre de comando es una cadena de caracteres. Aquí hay algunos ejemplos de nombres de comandos válidos: `greet`, `get-size`, `mycommand123`, `my command`, y .

> Es una práctica común en Nushell separar las palabras del comando con `-` para una mejor legibilidad._ Por ejemplo `get-size` en lugar de `getsize` o `get_size`.

9.13.5. Subcomandos

También se pueden definir subcomandos a comandos usando un espacio. Por ejemplo, si se quisiera agregar un nuevo subcomando a `str`, se puede crear nombrando el subcomando de forma tal que comience con str. Por ejemplo:

```
def "str mycommand" [] {
  "hello"
}
```

Se puede llamar al comando personalizado como si fuera un subcomando incorporado de `str`:

```
str mycommand
```

Por supuesto, los comandos con espacios en sus nombres se definen de la misma manera:

```
def "custom command" [] {
  "this is a custom command with a space in the name!"
}
```

9.13.6. Tipos de Parámetros

Cuando se definen comandos personalizados, se puede nombrar y, opcionalmente, establecer el tipo para cada parámetro. Por ejemplo, se puede escribir lo anterior como:

```
def greet [name: string] {
  $"hello ($name)"
}
```

Los tipos de parámetros son opcionales. Nushell permite omitirlos y tratar el parámetro como `any` si es así. Si se selecciono un tipo en un parámetro, Nushell comprobará este tipo cuando llame a la función.

Por ejemplo, supongamos que necesitamos un dato tipo `int` en su lugar:

```
def greet [name: int] {
  $"hello ($name)"
}

greet world
```

Si intentamos ejecutar lo anterior, Nushell nos indica que los tipos no coinciden:

```
error: Type Error
   shell:6:7

5  greet world
          ^^^^^ Expected int
```

Esto puede ayudar a guiar a los usuarios de sus definiciones para que las llamen sólo con los tipos admitidos.

Los tipos actualmente aceptados son:

- `any`
- `binary`
- `bool`
- `cell-path`
- `closure`
- `datetime`
- `directory`
- `duration`
- `error`
- `filesize`
- `float`
- `glob`
- `int`
- `list`
- `nothing`
- `number`
- `path`
- `range`
- `record`
- `string`
- `table`

9.13.7. Parámetros con un Valor Predeterminado

Para hacer un parámetro opcional y directamente proporcionar un valor predeterminado para él, se puede proporcionar un valor predeterminado en la definición del comando.

```
def greet [age, name = "nushell"] {
  $"hello ($name) + ($age)"
}
```

Se puede llamar a este comando sin el parámetro o con un valor para anular el valor predeterminado:

```
greet


greet world
```

También puede combinar un valor predeterminado con un requisito de tipo:

```
def congratulate [age: int = 18] {
  $"Happy birthday! You are ($age) years old now!"
}
```

Si se desea comprobar si un parámetro opcional está presente o no y no solo confiar en un valor predeterminado, se pueden utilizar parámetros posicionales opcionales en su lugar.

9.13.8. Parámetros Posicionales Opcionales

Por defecto, los parámetros posicionales son requeridos. Si un parámetro posicional no se entrega, se encontrará un error:

```
  Error: Missing required positional argument.
    [entry #23:1:1]
 1  greet
    ^^^^^
    ^^^^^       missing name

  help: Usage: greet <name>
```

Podemos en cambio marcar un parámetro posicional como opcional poniendo un signo de interrogación (`?`) después de su nombre. Por ejemplo:

```
def greet [name?: string] {
  $"hello ($name)"
}

greet
greet world
```

Hacer un parámetro posicional opcional no cambia su nombre cuando se accede en el cuerpo. Como el ejemplo de arriba muestra, todavía se accede con `$name`, a pesar del sufijo `?` en la lista de parámetros.

Cuando un parámetro opcional no se entrega, su valor en el cuerpo del comando es igual a `null`. Podemos usar esto para actuar en el caso donde un parámetro no fue entregado:

```
def greet [name?: string] {
  if ($name == null) {
    "hello, I don't know your name!"
  } else {
    $"hello ($name)"
  }
}

greet
```

Si solo se quiere establecer un valor por defecto cuando el parámetro falta, es más simple usar un valor por defecto en cambio.

Si se usan parámetros posicionales requeridos y opcionales juntos, entonces los parámetros requeridos deben aparecer en la definición primero.

9.13.9. Opciones (*flags*)

Además de pasar parámetros posicionales, también puede pasar parámetros con nombre definiendo opciones para los comandos personalizados.

Por ejemplo:

```
def greet [
  name: string
  --age: int
] {
  [$name $age]
}
```

En la definición de `greet` anterior, se define el parámetro posicional `name`, así como una opción llamada `age`. Esto permite a quien llama a `greet` pasar opcionalmente también el parámetro `age`.

Se puede llamar a lo anterior usando:

```
greet world --age 10
```

O:

```
greet --age 10 world
```

O incluso omitir la opción por completo:

```
greet world
```

Las opciones también se pueden definir para tener una versión abreviada. Esto permite pasar una opción de forma más simple, así como una opción larga-y-más-fácil-de-leer.

Extendemos el ejemplo anterior para usar una opción abreviada para el valor de `age`:

```
def greet [
  name: string
  --age (-a): int
] {
  [$name $age]
}
```

> *Nota:* Las opciones se nombran por su nombre largo, por lo que el ejemplo anterior necesitaría usar `$age` y no `$a`.

Ahora, se puede llamar a esta definición actualizada usando la la opción abreviada:

```
greet -a 10 hello
```

Las opciones también se pueden usar como interruptores (*switches*) básicos. Esto significa que su presencia o ausencia se toma como un argumento para la definición. Extendiendo el ejemplo anterior:

```
def greet [
  name: string
  --age (-a): int
  --twice
] {
  if $twice {
    [$name $name $age $age]
  } else {
    [$name $age]
  }
}
```

Y la definición se puede llamar como:

```
greet -a 10 --twice hello
```

O simplemente sin la opción del switch:

```
greet -a 10 hello
```

También se puede asignar un valor de verdadero/falso para habilitar/deshabilitar la opción:

```
> greet -a 10 --switch=false
> greet -a 10 --switch=true
```

Las opciones pueden contener guiones. Se puede acceder a ellas reemplazando el guion con un guion bajo:

```
def greet [
  name: string
  --age (-a): int
  --two-times
] {
  if $two_times {
    [$name $name $age $age]
  } else {
    [$name $age]
  }
}
```

9.13.10. Parámetros rest

Puede haber casos en los que se desee definir un comando que tome cualquier número de argumentos posicionales. Podemos hacer esto con un parámetro te tipo `rest`, usando la siguiente sintaxis `...`:

```
def greet [...name: string] {
  print "hello all:"
  for $n in $name {
    print $n
  }
}

greet earth mars jupiter venus
```

Se puede llamar a la definición anterior del comando `greet` con cualquier número de argumentos, incluyendo ninguno en absoluto. Todos los argumentos se recopilan en `$name` como una lista.

Los parámetros rest se pueden usar junto con los parámetros posicionales:

```
def greet [vip: string, ...name: string] {
  print $"hello to our VIP ($vip)"
  print "and hello to everybody else:"
  for $n in $name {
    print $n
  }
}

greet moon earth mars jupiter venus
```

Para pasar una lista a un parámetro `rest`, puede usar el operador de propagación `...`:

```
let planets = [earth mars jupiter venus]

greet moon ...$planets
```

9.13.11. Documentando un Comando

Para ayudar mejor a los usuarios acerca del uso de los comandos personalizados, también se pueden documentar con descripciones adicionales para los comandos y parámetros.

Tomando el ejemplo anterior:

```
def greet [
  name: string
  --age (-a): int
] {
  [$name $age]
}
```

Una vez definido, se puede ejecutar `help greet` para obtener la información de ayuda para el comando:

```
Usage:
  > greet <name> {flags}

Parameters:
  <name>

Flags:
  -h, --help: Display this help message
  -a, --age <integer>
```

Se puede ver el parámetro y la opción que se definió, así como la bandera de ayuda `-h` que tienen todos los comandos.

Para mejorar esta ayuda, se pueden agregar descripciones a la definición que aparecerán en la ayuda:

```
# A greeting command that can greet the caller
def greet [
  name: string      # The name of the person to greet
  --age (-a): int   # The age of the person
] {
  [$name $age]
}
```

Los comentarios que se incluyen en la definición y sus parámetros aparecen como descripciones dentro del `help` del comando.

Ahora, si se ejecuta `help greet`, se entrega un texto de ayuda más útil:

```
A greeting command that can greet the caller

Usage:
  > greet <name> {flags}

Parameters:
  <name> The name of the person to greet

Flags:
  -h, --help: Display this help message
  -a, --age <integer>: The age of the person
```

9.13.12. Salida de la *pipeline*

Los comandos personalizados transmiten su salida al igual que los comandos incorporados. Por ejemplo:

```
ls | get name
```

Hacemos una copia de `ls` a un comando personalizado:

```
def my-ls [] { ls }
```

Se puede usar la salida de este comando tal como se haría con `ls`.

```
my-ls | get name
```

Esto nos permite construir fácilmente comandos personalizados y procesar su salida. Tenga en cuenta que no usamos sentencias de retorno como otros lenguajes. En cambio, construimos tuberías que generan flujos de datos que se pueden conectar a otras tuberías.

9.13.13. Entrada de la *pipeline*

Los comandos personalizados también pueden tomar entrada desde la *pipeline*, al igual que otros comandos. Esta entrada se pasa automáticamente al bloque que utiliza el comando personalizado.

Consideremos el siguiente comando que calcula el doble para cada elemento de una lista:

```
def double [] {
  each { |elt| 2 * $elt }
}
```

Y puede utilizarse de la siguiente forma:

```
[1 2 3] | double
```

También se puede almacenar la entrada para su uso posterior utilizando la variable `$in`:

```
def nullify [...cols] {
  let start = $in
  $cols | reduce --fold $start { |col, df|
    $df | upsert $col null
  }
}
```

9.13.14. Control de flujo

Nushell proporciona varios comandos que ayudan a determinar cómo se ejecutan diferentes grupos de código. En los lenguajes de programación esta funcionalidad a menudo se conoce como *control de flujo*.

9.13.14.1. Elección (Condicionales) Los siguientes comandos ejecutan código basado en una condición dada.

Los comandos de elección/condicionales son expresiones, por lo que devuelven valores, a diferencia de los otros comandos en esta página. Esto significa que lo siguiente funciona.

```
> 'foo' | if $in == 'foo' { 1 } else { 0 } | $in + 2
3
```

9.13.14.1.1. `if` `if` evalúa de código ramificados basados en los resultados de una o más condiciones similares a la funcionalidad if en otros lenguajes de programación. Por ejemplo:

```
if $x > 0 { 'positivo' }
```

Devuelve `'positivo'` cuando la condición es `true` (`$x` es mayor que cero) y `null` cuando la condición es `false` (`$x` es menor o igual a cero).

Se puede agregar una rama `else` al `if` después del primer bloque que se ejecuta y devuelve el valor resultante del bloque `else` cuando la condición es `false`. Por ejemplo:

```
if $x > 0 { 'positivo' } else { 'no-positivo' }
```

Esta vez devuelve `'positivo'` cuando la condición es `true` (`$x` es mayor que cero) y `'no-positivo'` cuando la condición es `false` (`$x` es menor o igual a cero).

También podemos se pueden anidar múltiples `if`s de la siguiente manera:

```
if $x > 0 { 'positive' } else if $x == 0 { 'zero' } else { "negative" }
```

o bien:

```
if $x > 0 {
   'positive'
} else if $x == 0 {
   'zero'
} else {
   "negativo"
}
```

Cuando la primera condición es `true` (`$x` es mayor que cero) devolverá `'positivo'`, cuando la primera condición es `false` y la siguiente condición es `true` (`$x` es igual a cero) devolverá `'zero'`, de lo contrario devolverá `'negativo'` (cuando `$x` es menor que cero).

9.13.14.1.2. `match` `match` ejecuta una de varias ramas condicionales basadas en el valor dado para coincidir. También puede hacer alguna coincidencia de patrones para desempaquetar valores en tipos compuestos como listas y registros.

El uso básico de `match` puede ejecutar condicionalmente diferentes códigos como una sentencia switch común en otros lenguajes. `match` comprueba si el valor después de la palabra `match` es igual al valor al comienzo de cada rama antes del `=>` y si lo es, ejecuta el código después del `=>` de esa rama.

```
match 2 {
    1 => 'one',
```

```
    2 => {
        let w = 'w'
        't' + $w + 'o'
    },
    3 => 'three',
    4 => 'four'
}
```

Las ramas pueden devolver un solo valor o, como se muestra en la segunda rama, pueden devolver los resultados de un bloque.

Rama comodín

También puede tener una condición comodín para cuando el valor dado no coincide con ninguna de las otras condiciones al tener una rama cuyo valor coincidente es `_`.

```
let foo = match 7 {
    1 => 'one',
    2 => 'two',
    3 => 'three',
    _ => 'other number'
}

$foo
```

> Notar que , `match` es una expresión, por lo que podemos asignar el resultado a una variable.

Coincidencia de patrones

Se pueden desempaquetar valores de tipos como listas y registros con coincidencia de patrones. Luego se puede asignar variables a las partes que se desea desempaquetar y usarlas en las expresiones coincidentes.

```
let foo = { name: 'bar', count: 7 }

match $foo {
    { name: 'bar', count: $it } => ($it + 3),
    { name: _, count: $it } => ($it + 7),
    _ => 1
}
```

El `_` en la segunda rama significa que coincide con cualquier registro con el campo `name` y `count`, no solo con aquellos donde `name` es `'bar'`.

Guardias

También puede agregar una condición adicional a cada rama llamada guardia (*guard*) para deter-

minar si la rama debe coincidir. Para hacerlo, después del patrón coincidente, se utiliza un `if` y luego la condición antes del `=>`.

```
let foo = { name: 'bar', count: 7 }

match $foo {
    { name: 'bar', count: $it } if $it < 5 => ($it + 3),
    { name: 'bar', count: $it } if $it >= 5 => ($it + 7),
    _ => 1
}
```

9.13.14.2. Bucles Los comandos de bucle permiten repetir un bloque de código varias veces.

9.13.14.2.1. Bucles y Otros Comandos de Iteración La funcionalidad de los comandos de bucle es similar a los comandos que aplican un cierre sobre elementos en una lista o tabla como `each` o `where` y muchas veces se puede lograr lo mismo con cualquiera. Por ejemplo:

```
mut result = []
for $it in [1 2 3] {
   $result = ($result | append ($it + 1))
}

$result


[1 2 3] | each { $in + 1 }
```

Si bien puede ser tentador usar bucles si se está familiarizado con ellos en otros lenguajes, se considera más en el estilo Nushell (idiomático) usar comandos que aplican cierres cuando se puede resolver un problema de cualquier manera. La razón de esto es debido a una desventaja bastante grande con el uso de bucles.

Desventajas del Bucle

La mayor desventaja de los bucles es que son sentencias, a diferencia de `each` que es una expresión. Las expresiones, como `each` siempre resultan en algún valor de salida, sin embargo, las sentencias no.

Esto significa que no funcionan bien con variables inmutables y el uso de variables inmutables se considera más estilo Nushell. Sin una variable mutable declarada de antemano en el ejemplo de la sección anterior, sería imposible usar `for` para obtener la lista de números con números incrementados, o cualquier valor en absoluto.

Las sentencias tampoco funcionan en las *pipelines* de Nushell, que requieren alguna salida. De hecho, Nushell dará un error si se intentas:

```
[1 2 3] | for x in $in { $x + 1 } | $in ++ [5 6 7]
Error: nu::parser::unexpected_keyword

  × Statement used in pipeline.
   [entry #5:1:1]
 1  [1 2 3] | for x in $in { $x + 1 } | $in ++ [5 6 7]
   ·
   ·             not allowed in pipeline

  help: 'for' keyword is not allowed in pipeline. Use 'for' by itself, outside of a pipeli
```

Debido a que Nushell está muy orientado a la *pipeline*, esto significa que usar comandos de expresión como `each` es típicamente más natural que las sentencias de bucle.

Ventajas del Bucle

Si los bucles tienen una desventaja tan grande, £por qué existen? Bueno, una razón es que los cierres (*clousures*), como los que usa `each`, no pueden modificar variables mutables en el entorno circundante. Si se intentas modificar una variable mutable en un cierre, se obtendrá un error:

```
mut foo = []

[1 2 3] | each { $foo = ($foo | append ($in + 1)) }
Error: nu::parser::expected_keyword

  × Capture of mutable variable.
   [entry #8:1:1]
 1  [1 2 3] | each { $foo = ($foo | append ($in + 1)) }
   ·
   ·                      capture of mutable variable
```

Si se modifica una variable de entorno en un cierre, se puede, pero solo se modificará dentro del alcance del cierre, dejándola sin cambios en cualquier otro lugar. Los bucles, sin embargo, usan bloques lo que significa que pueden modificar una variable mutable regular o una variable de entorno dentro del alcance mayor.

```
mut result = []

for $it in [1 2 3] { $result = ($result | append ($it + 1)) }

$result
```

9.13.14.2.2. `for` `for` recorre un rango o colección como una lista o una tabla.

```
for x in [1 2 3] { $x * $x | print }
```

Alternativas de Comandos de Expresión

- `each`
- `par-each`
- `where`
- `reduce`

9.13.14.2.3. `while` `while` repite el mismo bloque de código hasta que la condición dada es `false`.

```
mut x = 0;

while $x < 10 { $x = $x + 1 }

$x
```

Alternativas de Comandos de Expresión

Los comandos until y otros comandos while

- `take until`
- `take while`
- `skip until`
- `skip while`

9.13.14.2.4. `loop` `loop` repite un bloque infinitamente. Se puede usar `break` (como se describe en la siguiente sección) para limitar cuántas veces se repite. También puede ser útil para ejecutar scripts continuamente, como un indicador interactivo.

```
mut x = 0; loop { if $x > 10 { break }; $x = $x + 1 }; $x
```

9.13.14.2.5. `break` `break` detendrá la ejecución del código en un bucle y reanudará la ejecución después del bucle. Efectivamente rompiendo el bucle.

```
for x in 1..10 { if $x > 3 { break }; print $x }
```

9.13.14.2.6. `continue` `continue` detendrá la ejecución del bucle actual, omitiendo el resto del código en el bucle, e irá al siguiente bucle. Si el bucle normalmente terminaría, como si `for` ha iterado a través de todos los elementos dados, o si la condición de `while` ahora es falsa, no se repetirá de nuevo y la ejecución continuará después del bloque de bucle.

```
mut x = -1
while $x <= 6 {
    $x = $x + 1;
    if $x mod 3 == 0 {
        continue
```

```
    }
    print $x
}
```

9.13.14.2.7. Errores `error make`

`error make` crea un error que detiene la ejecución del código y cualquier código que lo haya llamado, hasta que sea manejado por un bloque `try`, o termine el script y muestre el mensaje de error. Esta funcionalidad es la misma que excepciones en otros lenguajes.

```
error make { msg: 'Some error info' }; print 'unprinted'
```

```
Error:   𝔼 Some error info
   [entry #9:1:1]
 1  print 'printed'; error make { msg: 'Some error info' }; print 'unprinted'
   ů
   ů                         originates from here
```

El registro que se le pasa proporciona información al código que lo captura o al mensaje de error resultante.

`try`

`try` capturará los errores creados en cualquier lugar del bloque de código de `try` y reanudará la ejecución del código después del bloque.

```
try { error make { msg: 'Some error info' }}; print 'Resuming'
```

Esto incluye la captura de errores incorporados.

```
try { 1 / 0 }; print 'Resuming'
```

El valor resultante será `nothing` si ocurre un error y el valor devuelto del bloque si no ocurre un error.

Si se incluye un bloque `catch` después del bloque `try`, ejecutará el código en el bloque `catch` si ocurrió un error en el bloque `try`.

```
try { 1 / 0 } catch { 'An error happened!' } | $in ++ ' And now I am resuming.'
```

No se ejecutará el bloque `catch` si no se produce un error.

```
try { 1 / 0 } catch { |e| $e.msg }
```

9.13.14.2.8. Otros `return`

`return` finaliza un cierre o comando anticipadamente donde se llama, sin ejecutar el resto del comando/cierre, y devuelve el valor dado. No es a menudo necesario ya que el último valor en un cierre o comando también se devuelve, pero a veces puede ser conveniente.

```
def 'positive-check' [it] {
    if $it > 0 {
        return 'positive'
    };

    'non-positive'
}

positive-check 3

positive-check (-3)

let positive_check = {|elt| if $elt > 0 { return 'positive' }; 'non-positive' }

do $positive_check 3

do $positive_check (-3)
```

9.13.15. Ejemplo de desarrollo de comando

En esta sección, utilizaremos la API de Google Maps para desarrollar un comando que interactue con ella y nos entregue informacion.

9.13.15.1. Obtener API Key Se debe crear una API key en el sitio web de Google Developers:

1. Ingrese al sitio.
2. Cree un nuevo proyecto.
3. Dirigirse al dashboard. Luego seleccionar el proyecto creado.
4. Dirigirse al APIs overview.
5. Activar API.
6. Busque y seleccion Directions API.
7. Dirigirse a API y Servicios, Credenciales.
8. Crear Credencial y seguir instrucciones.
9. Guardar la API Key obtenida.

9.13.15.2. Exploracion de la API De acuerdo a la información en el sitio web de la documentación de la API, entre otras cosas, se puede obtener una dirección a partir de coordenadas latitud-longitud, y viceversa.

Primeramente, almacenamos la api key en una variable para su posterior uso:

```
let mapsAPIkey = "the_api_key"
```

a su vez como la dirección de testeo:

```
let address = "Avenida Collao 1202, Concepcion, Chile"
```

De acuerdo la documentación de la API, para poder obtener las coordenadas asociadas a la dirección de testeo, debemos consultar la siguiente url: https://maps.google.com/maps/api/geocode/json?address=Avenida+Collao+1202%2C+Concepcion%2C+Chile&key=the_api_key

Esta url tiene las siguientes partes:

- **Scheme:** Esto indica el *protocolo* utilizado para acceder al recurso. Le dice al navegador *cómo* comunicarse con el servidor. Los esquemas comunes incluyen:
  - `http://` (Hypertext Transfer Protocol): El protocolo estándar para transferir páginas web. Sin cifrar.
  - `https://` (Hypertext Transfer Protocol Secure): La versión segura de HTTP. Cifrada para una comunicación más segura.
  - `ftp://` (File Transfer Protocol): Utilizado para transferir archivos entre computadoras.
  - `mailto:` Utilizado para crear enlaces de correo electrónico. Al hacer clic, se abre su cliente de correo electrónico predeterminado.
  - `file://` Utilizado para acceder a archivos locales en su computadora.
- **Host:** Esto especifica la *dirección del servidor* donde se encuentra el recurso. Puede ser un nombre de dominio (como `www.example.com`) o una dirección IP (como `192.168.1.1`). Le dice al navegador *dónde* encontrar el recurso. A menudo, el host incluye un subdominio (por ejemplo, `blog.example.com` donde `blog` es el subdominio). También puede incluir un número de puerto después de dos puntos (por ejemplo, `www.example.com:8080`), aunque los puertos predeterminados para `http` (80) y `https` (443) generalmente se omiten.
- **Path:** Esto identifica el *recurso* específico en el servidor. Es como una ruta de archivo en su computadora. Sigue al host y comienza con una barra diagonal `/`. Por ejemplo, en `https://www.example.com/articles/technology/new-gadgets.html`, la ruta es `/articles/technology/new-gadgets.html`.
- **Parámetros (Query Parameters):** Estos son opcionales y proporcionan *información adicional* al servidor. Vienen después de un signo de interrogación `?` y tienen el formato `key=value`. Múltiples parámetros están separados por símbolos de unión `&`. Por ejemplo, en `https://www.example.com/search?q=programming&language=python`, los parámetros son `q=programming` y `language=python`. Le dicen al servidor que está buscando programming en el idioma python. Estos se utilizan a menudo para filtrar, ordenar o pasar datos a aplicaciones web.

El comando `url join` nos permite separar una url en sus partes y generar una url correcta:

```
let url = {
   scheme: "https"
```

```
   host: "maps.google.com"
   path: "/maps/api/geocode/json"
   params: {
     address: $address
     key: $mapsAPIkey
   }
} | url join
```

Hacemos la consulta mediante `http get`:

```
let response = http get $url
```

Y procedemos a explorar la respuesta:

```
$response

$response.results
$response.results | table -e
$response.results.geometry
$response.results.geometry.location
```

Podemos ahora compilar las operaciones en un solo comando:

```
ox maps.nu #o el editor de su preferencia
```

y copiamos lo siguiente:

```
export def loc-from-address [address:string] {
  let mapsAPIkey = "the_api_key"

  let url = {
    scheme: "https"
    host: "maps.google.com"
    path: "/maps/api/geocode/json"
    params: {
        address: $address
        key: $mapsAPIkey
    }
  } | url join

  return (http get $url | get results.geometry.location)
}
```

Guardamos el resultado y en nushell ejecutamos:

```
use maps.nu *
loc-from-address "Avenida Collao 1202, Concepcion, Chile"
```

De forma similar podemos crear un comando que obtenga la dirección a partir de las coordenadas latitud y longitud:

```
ox maps.nu
```

Y escribimos el siguiente código:

```
export def address-from-loc [
   latitude:number
   longitude:number

   ] {
  let mapsAPIkey = "the_api_key"

  let url = {
    scheme: "https"
    host: "maps.google.com"
    path: "/maps/api/geocode/json"
    params: {
        latlng: $"($latitude),($longitude)"
        key: $mapsAPIkey
    }
  } | url join

  let response = http get $url

  if $response.status != OK {
    error make -u {msg: "address not found!"}
  }

  return $response.results.0.formatted_address
}
```

Guardamos y en nushell ejecutamos:

```
use maps.nu *

address-from-loc -36.8221967 -73.0120093
```

que debería dar como resultado:

```
Collao 1202, Concepción, Bío Bío, Chile
```

10. Para seguir explorando

Como este curso trata sobre la intersección de la línea de comandos y la ciencia de datos, muchos temas relacionados solo se han mencionado superficialmente. Las siguientes subsecciones proporcionan una lista de temas y recursos sugeridos para consultar.

10.1. La Línea de Comandos

- *The Linux Command Line: A Complete Introduction, 2nd Edition* Por William E. Shotts, Jr. (No Starch Press, 2019)
- *Unix Power Tools, 3rd Edition* por Jerry Peek, Shelley Powers, Tim OReilly, y Mike Loukides (OReilly Media, 2002)
- *Learning the Vi and Vim Editors, 7th Edition* por Arnold Robbins, Elbert Hannah, y Linda Lamb (OReilly Media, 2008)

10.2. Programación de Shell

- *Classic Shell Scripting* por Arnold Robbins y Nelson H.F. Beebe (OReilly Media, 2005)
- *Wicked Cool Shell Scripts, 2nd Edition* por Dave Taylor y Brandon Perry (No Starch Press, 2017)
- *Bash Cookbook* por Carl Albing JP Vossen (OReilly Media, 2018)
- *Nushell Book*: https://www.nushell.sh/book/

10.3. Python, R y SQL

- *Learn Python 3 the Hard Way* por Zed A. Shaw (Addison-Wesley Professional, 2017)
- *Python for Data Analysis, 2nd Edition* por Wes McKinney (OReilly Media, 2017)
- *Data Science from Scratch, 2nd Edition* por Joel Grus (OReilly Media, 2019)
- *R for Data Science* por Garrett Grolemund y Hadley Wickham (OReilly Media, 2016)
- *R for Everyone, 2nd edition* por Jared Lander (Addison-Wesley Professional, 2017)
- *Sams Teach Yourself SQL in 10 Minutes a Day, 5th Edition* por Ben Forta (Sams, 2020)

10.4. APIs

- *Mining the Social Web, 3rd Edition* por Matthew A. Russell y Mikhail Klassen (OReilly Media, 2019)
- *Data Source Handbook* por Pete Warden (OReilly Media, 2011)

10.5. Aprendizaje Automático

- *Python Machine Learning, 3rd Edition* por Sebastian Raschka y Vahid Mirjalili (Packt Publishing, 2019)
- *Pattern Recognition and Machine Learning* por Christopher M. Bishop (Springer, 2006)

- *Information Theory, Inference, and Learning Algorithms* por David MacKay (Cambridge University Press, 2003)